

软件工程概论-UML

张璇 云南大学

软件建模技术UML

内容取自西南林业大学 周华

几个问题？？？

■ 什么是对象？什么是类？

■ 什么是封装？

■ 什么是继承？什么是单继承？什么是多继承？

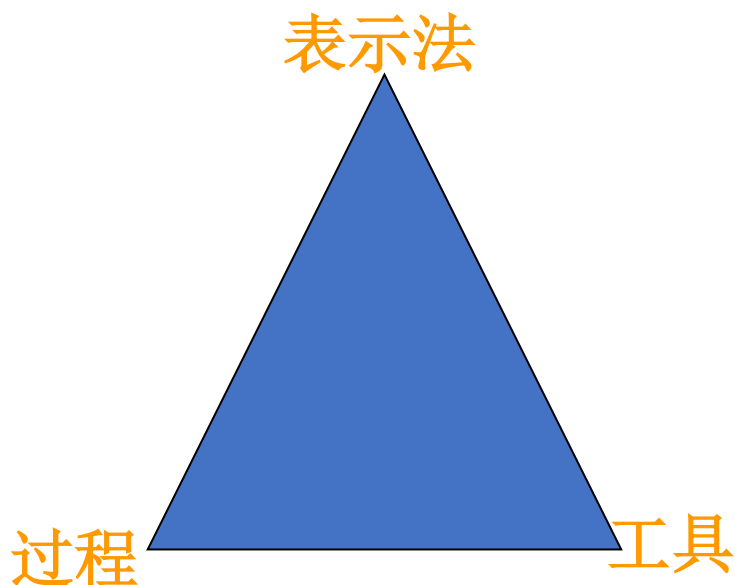
■ 什么是多态？

■ 什么是消息？

■ 什么是模型？什么是UML？

● 模型的概念

模型是对现实系统的简化或模拟,是对现实系统本质特征的一种简化、直观、类比和抽象的描述。



● **过程** — 建立模型的过程,只有确切知道建模的过程才能够成功建立一个模型。

● **表示法** — 模型的表示,也就是建模语言,是建立模型的关键,例如UML就是软件的建模语言。

● **工具** — 通过建模工具来方便建立模型。

软件建模的必要性

● 软件的特性

智能性

软件是人智力劳动的产物，具有极高的智能特性。

无形性

软件没有物理实体的物理形态，而是智能、逻辑和算法的描述。

抽象性

软件具有高度抽象性。

复杂性

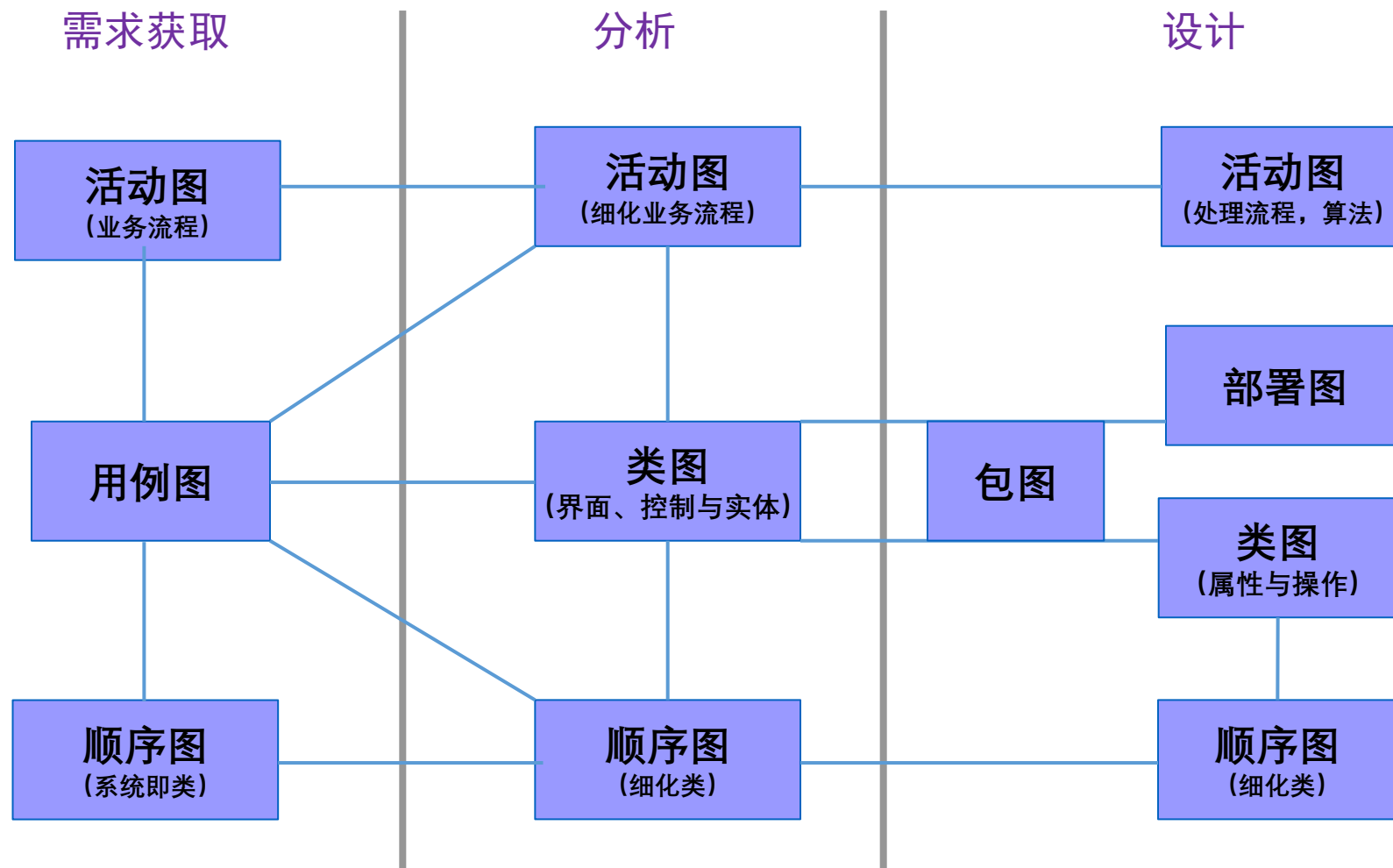
软件开发是高度复杂性工作，软件的规模越来越大，功能越来越复杂，服务领域越来越广泛。

综上

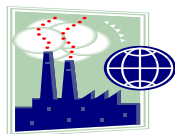
软件的开发和生产必须借助于模型，来反映和描述软件中间的开发过程和开发状态。

UML共定义了9种图:

- 用例图
- 状态图
- 类图
- 活动图
- 对象图
- 构件图
- 顺序图
- 部署图
- 协作图
- 包图



UML中的图



业务模型

业务用例图, 活动图, 状态图等



需求模型

用例图, 活动图, 状态图等



逻辑模型

类图, 交互图, 活动图, 顺序图, 状态图等



设计模型

类图, 交互图, 活动图, 顺序图, 状态图等



实现模型

构件图, 部署图等



测试模型

测试用例, 类图, 交互图等

UML支持工具

- Rational Rose（功能强大，市场占有率高）
- Together（纯Java开发）
- Argo UML（开放源代码）
- Visio（MS office 企业版套件）
- Visual UML（易用，功能强大，与Rose生成的模型可相互转换）
- Archware（基于体系结构的UML建模工具）

■基于UPS物流系统，完成如下UML建模作业

用例图

活动图

类图

顺序图

部署图

UML 用例图 (功能模型)

用例图

- ① 用例用于描述系统的功能，这个功能是外部使用者看到的系统功能，不反映功能的实现方式。
- ② 用例描述用户提出的一些可见需求，对应一个具体的用户目标。
- ③ 用例是对系统功能的描述，属于需求建模。

参与者的三种类型

- ①. 人：客户，读者，库管员
- ②. 设备：计算机，磁盘，读卡机等
- ③. 外部系统：上层系统等



储蓄系统

✓

开户

✓

存款

✓

取款

✓

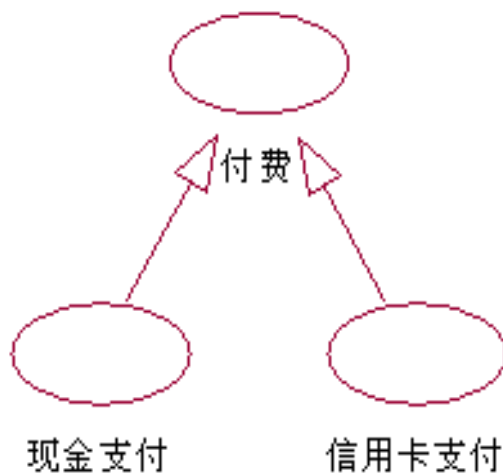
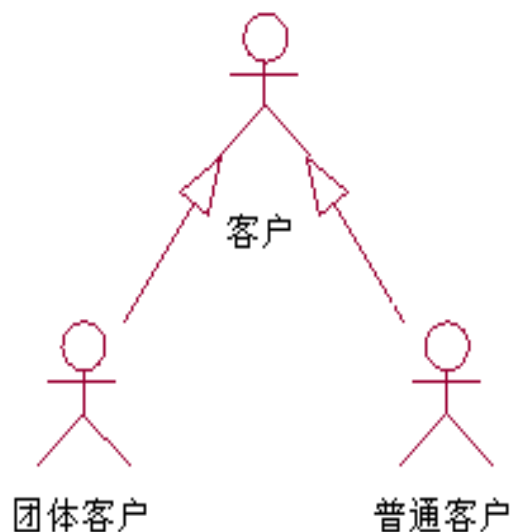
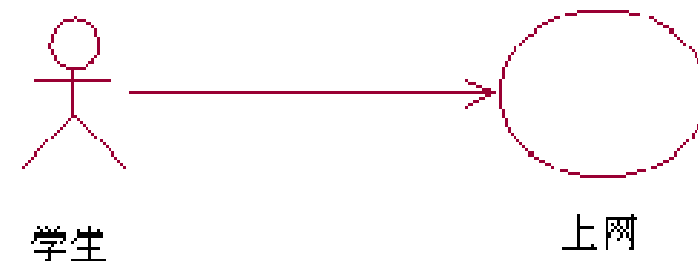
转帐

×

数据上传

1. 关联关系

参与者与用例之间是关联关系，表示参与者与用例之间具有使用，交互信息的关联。



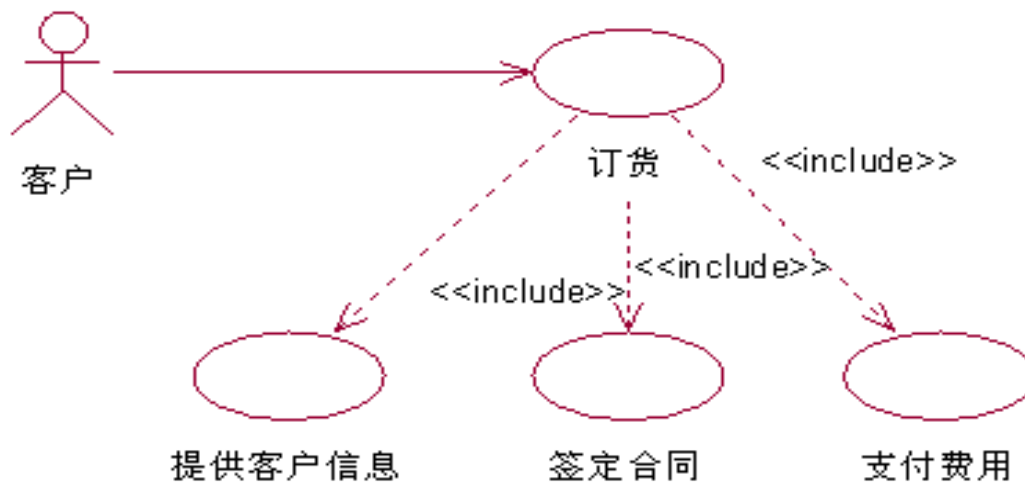
2. 泛化关系

参与者与参与者之间，用例与用例之间存在一般与特殊的关系。

3. 包含关系

一个用例(基本用例)的行为包含了另外一个用例(包含用例)的行为。

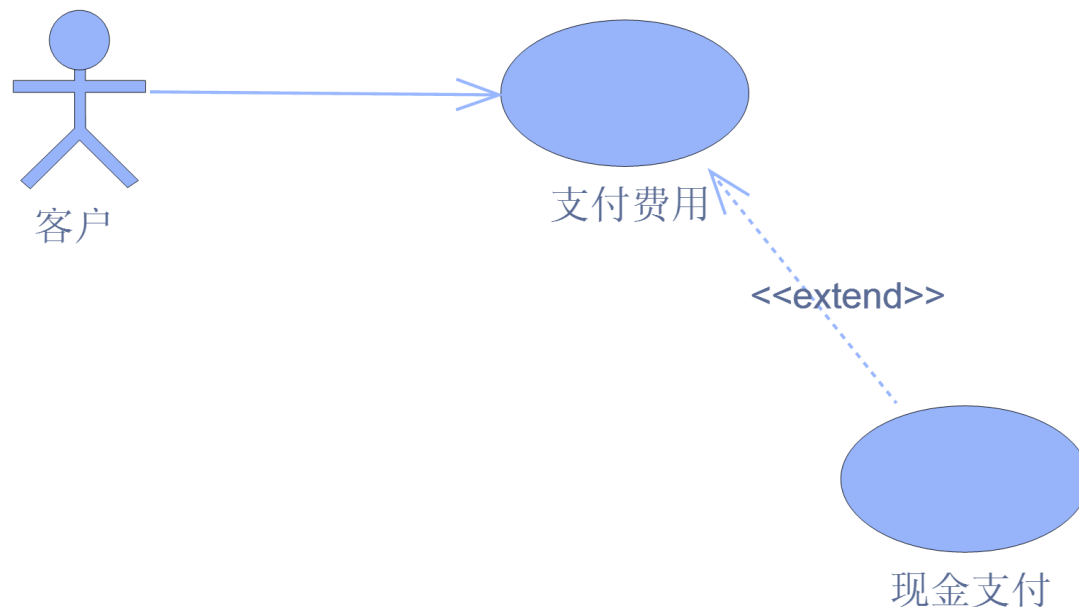
包含关系用依赖关系的`<<include>>`构造型来表示, 箭头指向被包含的用例。



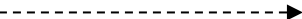
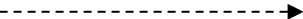
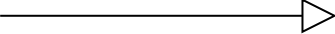
4. 扩展关系

扩展关系表示基本用例在扩展点要增加新的行为或功能，以扩展到新用例。

扩展关系用依赖关系的<<extend>>构造型来表示，箭头指向被扩展的用例。

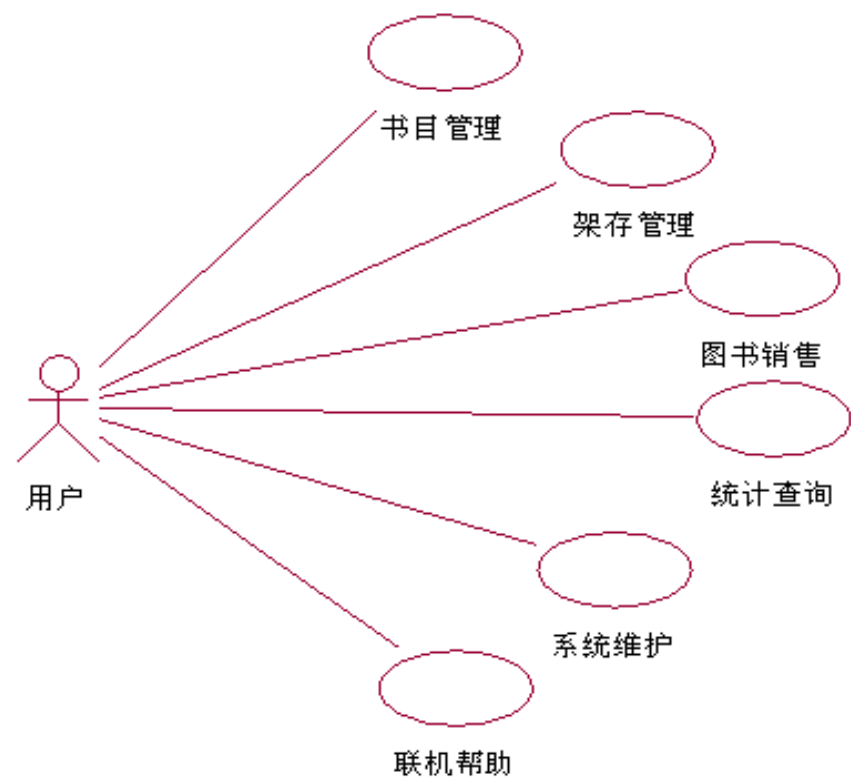


用例图中的关系及解释

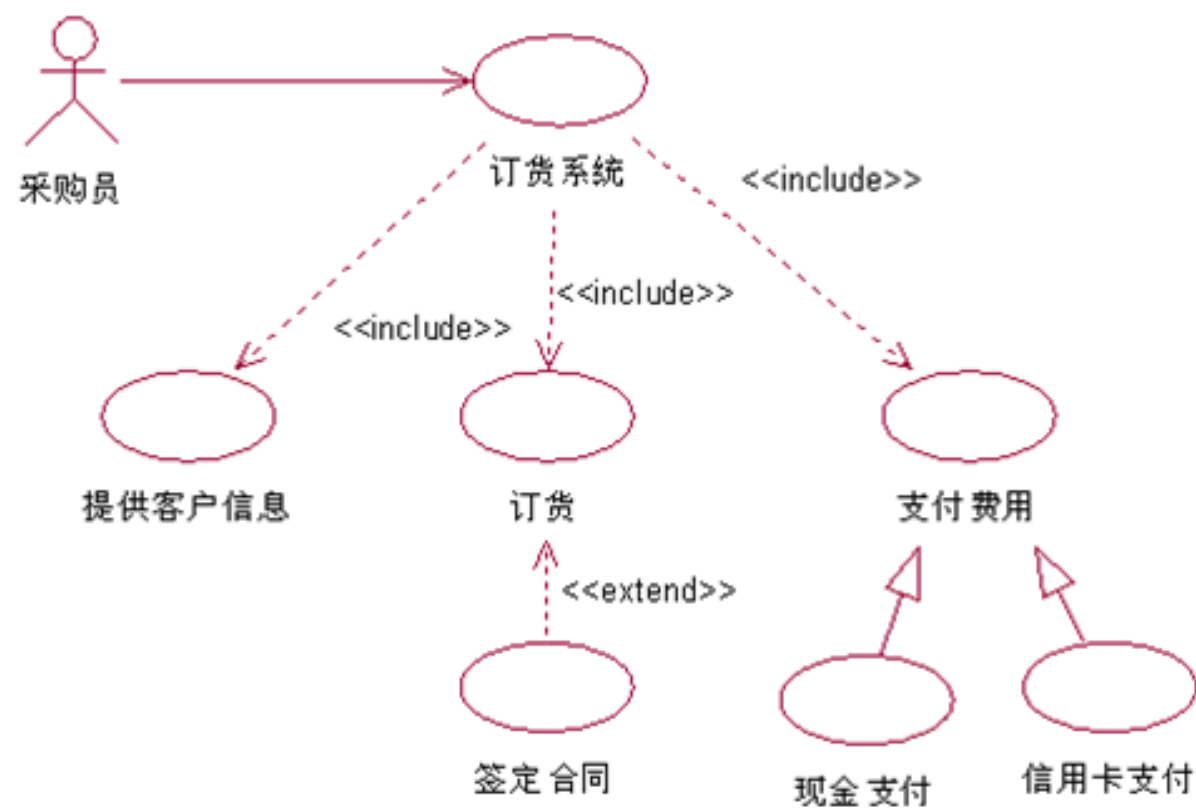
关系		解释	图
参与者与用例之间的关系	关联	表示参与者与用例之间的交互，通信途径。 (关联有时候也用带箭头的实线来表示，这样的表示能够显示地表明发起用例的是参与者。)	
用例之间的关系	包含	箭头指向的用例为被包含的用例，称为包含用例；箭头出发的用例为基用例。包含用例是必选的，如果缺少包含用例，基用例就不完整；包含用例必须被执行，不需要满足某种条件；其执行并不会改变基用例的行为。	《include》 
	扩展	箭头指向的用例为被扩展的用例，称为扩展用例；箭头出发的用例为基用例。扩展用例是可选的，如果缺少扩展用例，不会影响到基用例的完整性；扩展用例在一定条件下才会执行，并且其执行会改变基用例的行为。	《extend》 
参与者之间或用例之间的关系	泛化	发出箭头的事物“is a”箭头指向的事物。泛化关系是一般和特殊关系，发出箭头的一方代表特殊的一方，箭头指向的一方代表一般一方。特殊一方继承了一般方的特性并增加了新的特性。	

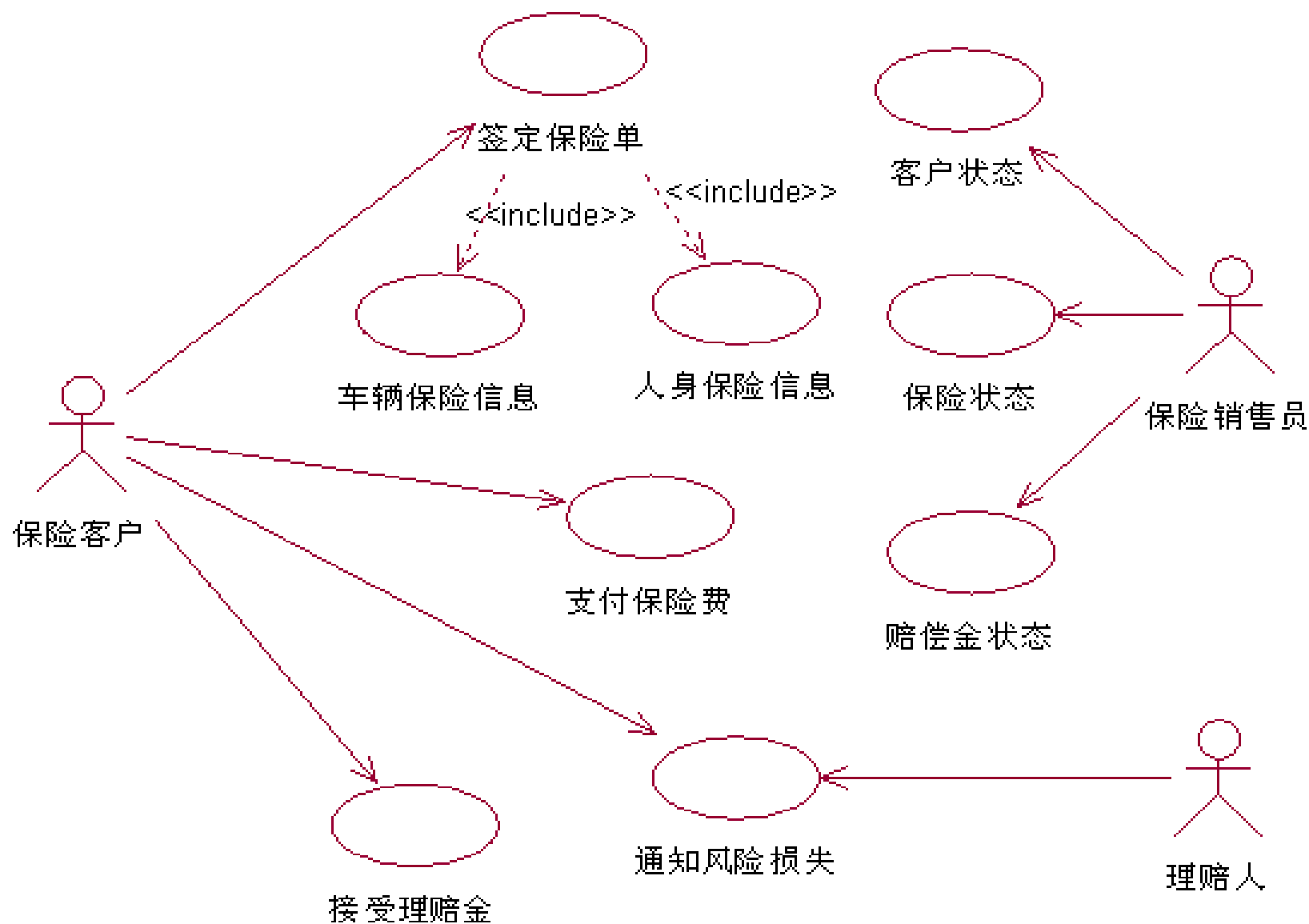
用例图

书务管理系统



订货管理





用例说明

描述项	说明
用例名称	用例的用途
标识符*	唯一，在文档别处可引用
用例描述	简单描述
参与者表	与此用例相关的参与者列表
优先级	数字越小，优先级越高。以此决定用例开发的顺序
状态*	进行中、等待审查、通过审查或未通过审查
前置条件	条件列表，访问用例之前应得到满足
后置条件	条件列表，访问用例之后得到满足
基本操作流	描述用例中各项工作正常进行是用例工作方式
可选操作流	描述变更工作方式、出现异常所遵循的工作路径
被泛化用例表	该用例的特化用例列表
被包含用例表	该用例所包含用例列表
被扩展用例表	该用例的扩展用例列表
修改历史记录*	关于用例的修改时间、原因和修改人信息等
问题*	与此用例开发相关的问题列表
决策*	关键决策列表
频率*	参与者访问该用例的频率

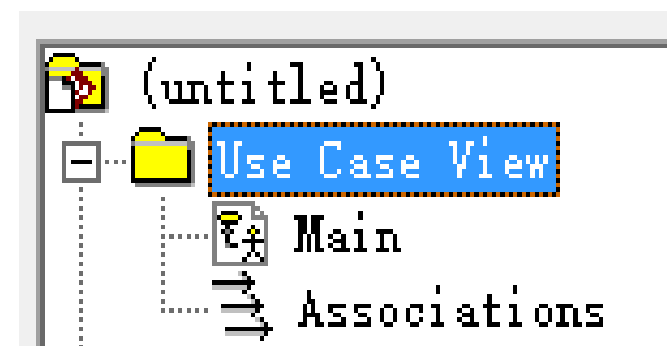
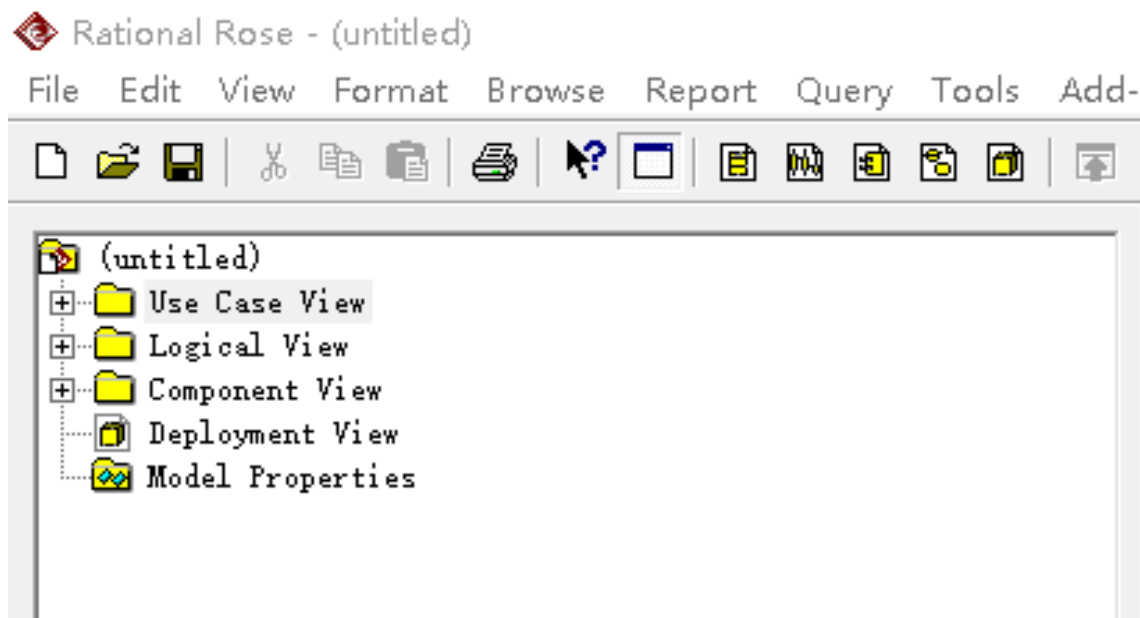
- 用例：取款
- ●参与者：储户
- ●操作流：
 - ① 通过读卡机，储户插入ATM卡
 - ② ATM系统从卡上读取银行ID、帐号、并验证帐号。
 - ③ 储户键入密码，系统检验密码。
 - ④储户按确认键，输入取款金额。
 - ⑤ATM把帐号和取款金额传递给银行系统，取回确认信息和帐户余额。
 - ⑥ ATM输出现金，并显示帐户余额。
 - ⑦ATM记录事务到日志文件。

- 发现用例的一般方法：

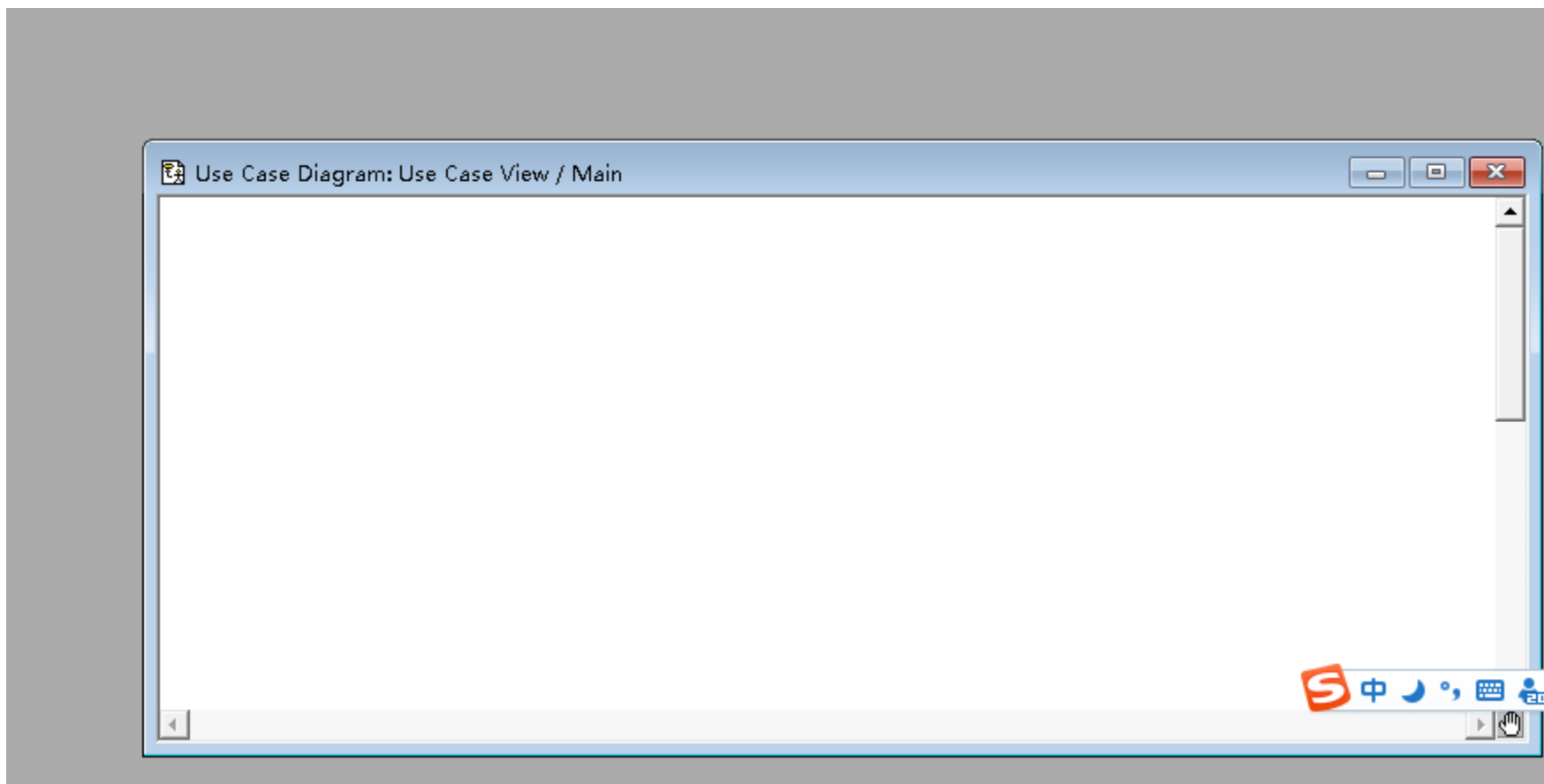
- ① 找出系统外部参与者，确定系统边界和范围。
- ② 确定各参与者所期望的系统行为。
- ③ 把这些系统行为命名为用例。
- ④ 确定各用例之间的关系(泛化，包含，扩展)。
- ⑤ 绘制用例图。
- ⑥ 编制用例说明。
- ⑦ 对异常流程确定单独用例。
- ⑧ 优化用例图，解决用例之间的冲突和重复。

案例

以自动提款机（Automated Teller Machine, ATM）系统为例，讨论使用面向对象技术进行需求分析的相关过程、概念和方法。



建立画布



ATM系统的需求说明

- 一个ATM系统的硬件设备包含了读卡器、客户交互控制台、出钞口、打印机和钥匙开关。
- 自动取款机服务一次只为一名顾客提供服务。
- 通过自动取款机客户能够进行如下交易：取款交易、存款交易、转账交易、查询交易。

建立用例模型

在进行面向对象需求分析时，我们首先确定软件系统的最终用户，并请他们叙述系统需要完成哪些事物。需求分析人员用用例模型来记录用户的需求，这些需求便是从系统外部看到的系统功能，同时它描述了用户提出的一些可见的需求，以及一些具体的用户目标。在UML中，用例模型的基本元素有参与者、用例和关系。

确定参与者(1)

参与者（Actor）是指与系统交互的人或事物，可以分为3类，即系统的**使用者**、**外部系统**和**时间**。其图形化的表示是一个类似人的图形符号。通过考察下面的四个方面来确定参与者：

- 标识系统范围和边界环境；
- 现有系统的文档和用户手册；
- 项目会议和研讨会的记录；
- 现有的需求文档、工作手册等。

确定参与者(2)

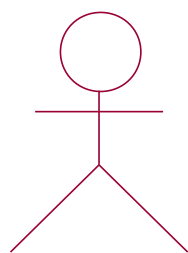
可以通过提出以下问题，明确参与者：

- 谁或者什么为系统提供输入？
- 谁或者什么接收系统输出？
- 需要与其他系统连接吗？
- 是否在预定时间自动触发事件？
- 谁将维护系统信息？

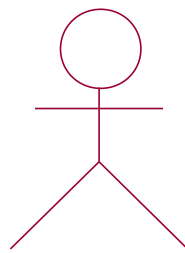
确定参与者时，从用户的角度出发，并使用用户的词汇给出参与者的文字定义。应该使用名词或名词词组来命名参与者。在3类参与者中系统使用者是最重要的角色。

ATM系统参与者

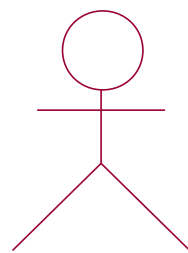
根据ATM系统分析则该系统对应着3个参与者，
即客户、管理员和银行计算机系统。



用户

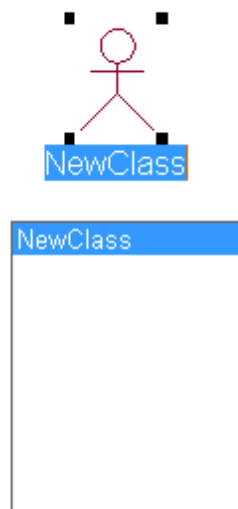
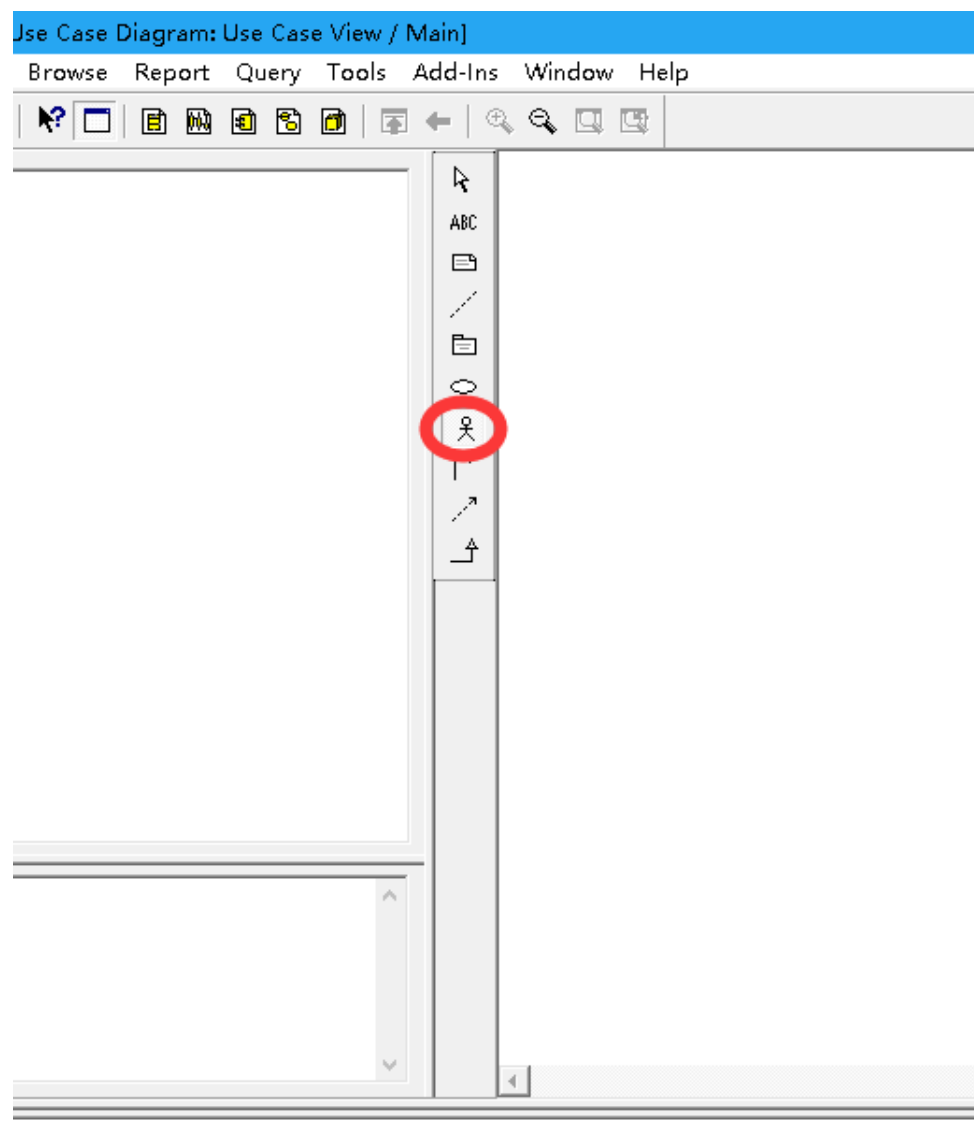


管理员



银行信息系统

添加参与者



The screenshot shows the 'Class Specification for 用户' dialog box. The 'Name' field is circled in red and contains the text '用户'. The 'Parent' field is set to 'Use Case'.

Class Specification for 用户			
Relations	Components	Nested	Files
General	Detail	Operations	Attributes
Name: 用户		Parent: Use Case	

确定业务需求用例

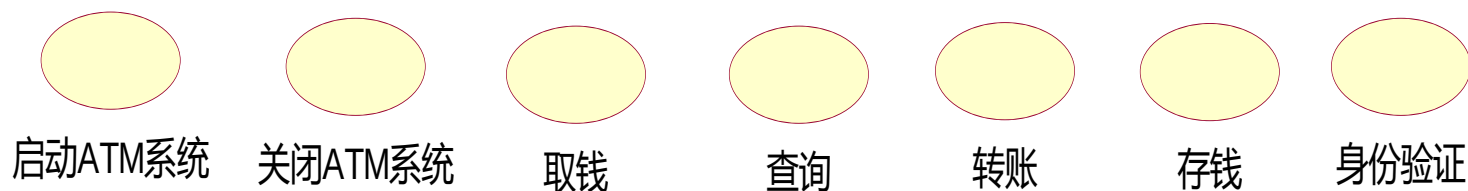
从用户的视角看，一个用例是参与者和系统之间的一次典型的交互作用。从系统内部的视角出发，一个用例代表系统执行的一系列动作，动作的执行结果能够被外部的参与者所察觉。

以下问题有助于更好地标识系统的用例：

- 每个参与者的特定任务是什么；
- 是否每个参与者都要从系统中创建、存储、改变、移动或者读取信息；
- 是否任何参与者都需要通知系统有关突发性的、外部的改变；
- 哪些用例支持或维护系统；
- 目前的用例是否覆盖了所有的功能需求。

ATM系统用例

在ATM系统中客户可以完成取钱、存钱、转账和查询任务，因此，在系统中必定存在取钱、存钱、转账和查询4个用例，银行信息系统除了与上述4个用例有关外还需要完成身份验证事务，因此银行信息系统对应着身份验证用例。同样，管理员对可以改变ATM系统的状态，因此对应着开启ATM系统和关闭ATM系统两个用例。因此，ATM系统总共对应着7个用例。



添加用例



Use Case Specification for NewUseCase

General | Diagrams | Relations | Files

Name: 查询 Package: Use Case View

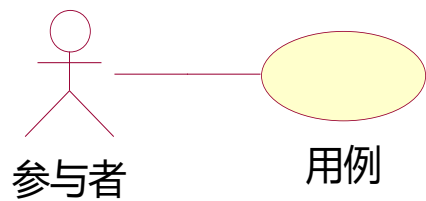
Stereotype:

Rank: ☐ Abstract

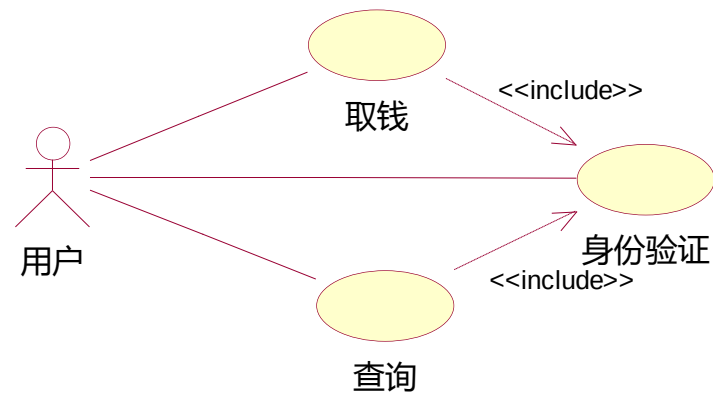
Documentation

OK Cancel Apply Browse Help

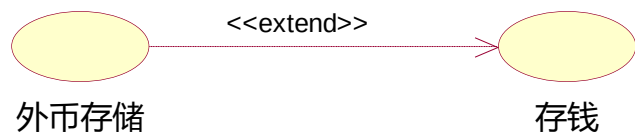
确定用例关系



关联关系



包含关系

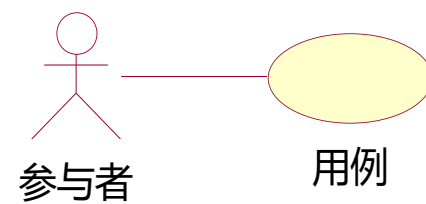
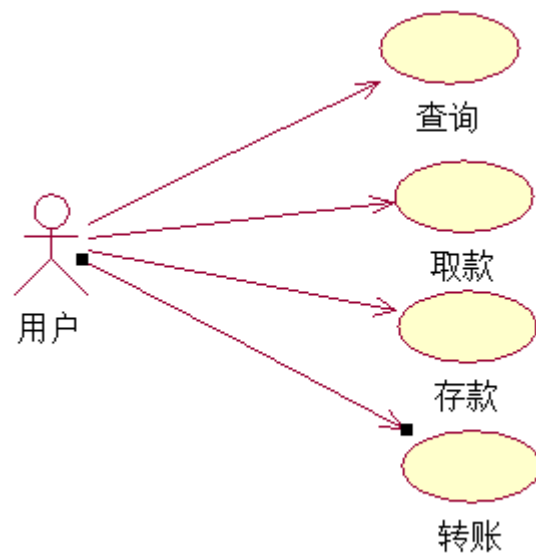


扩展关系

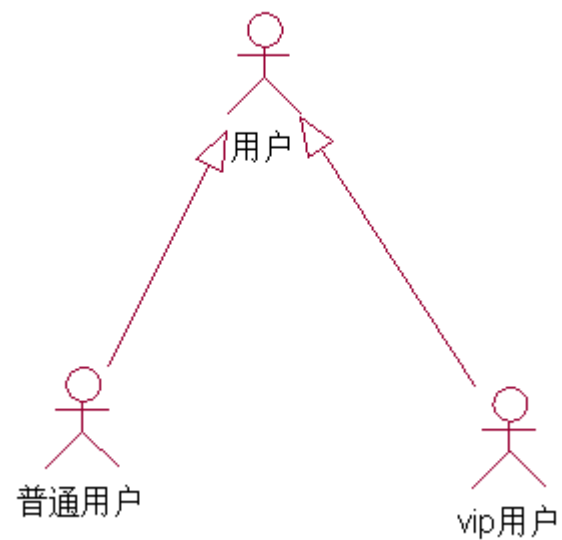
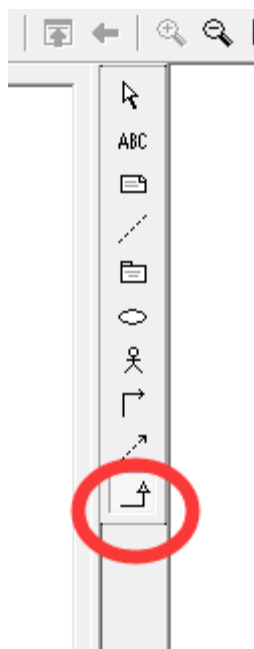


泛化关系

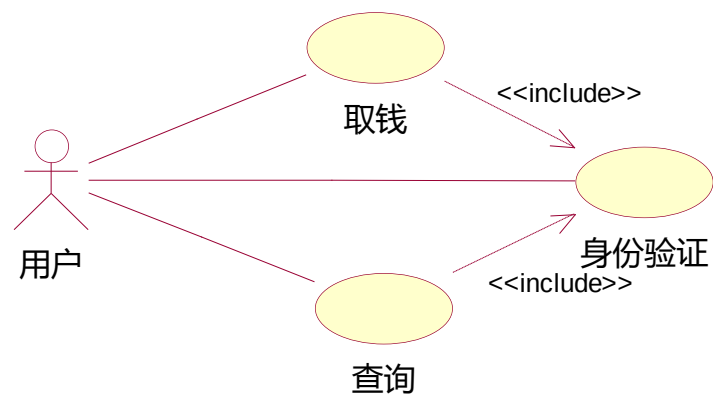
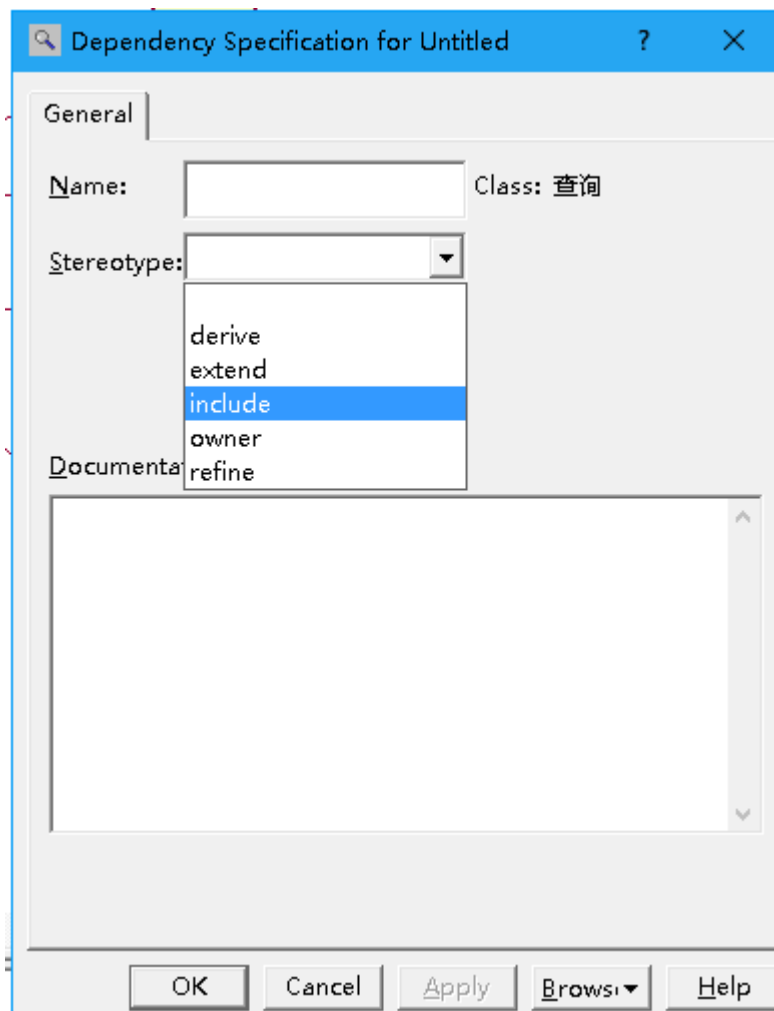
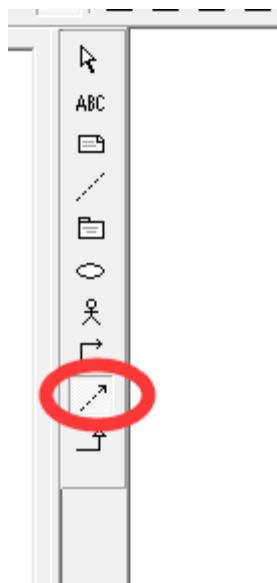
关联关系



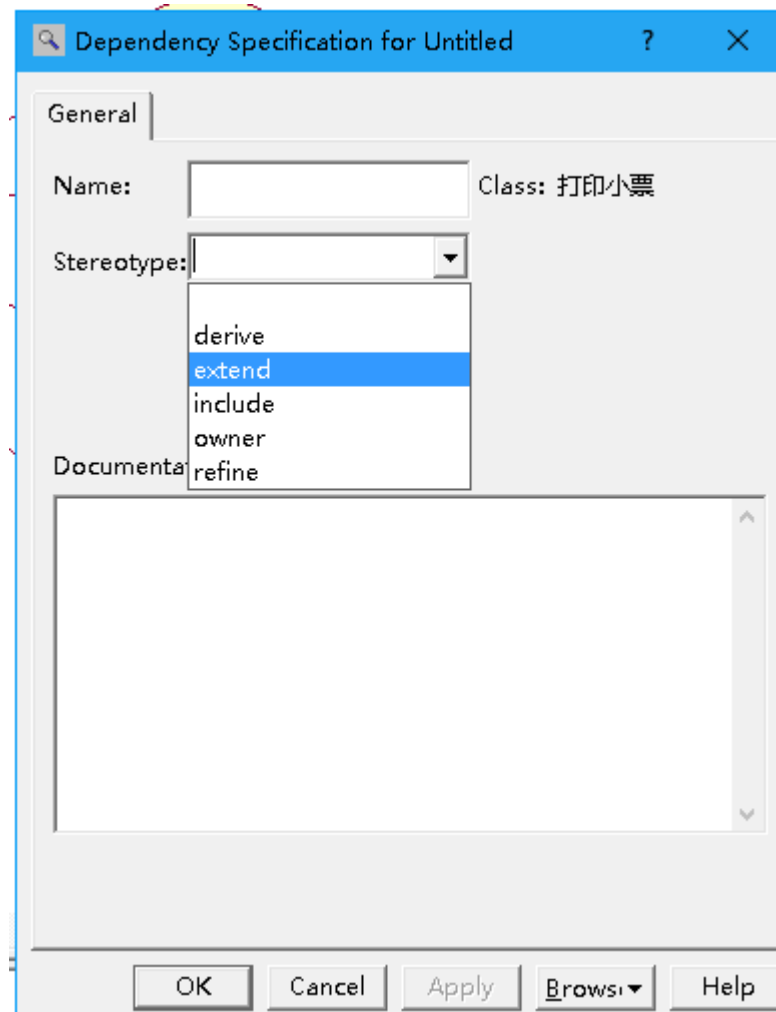
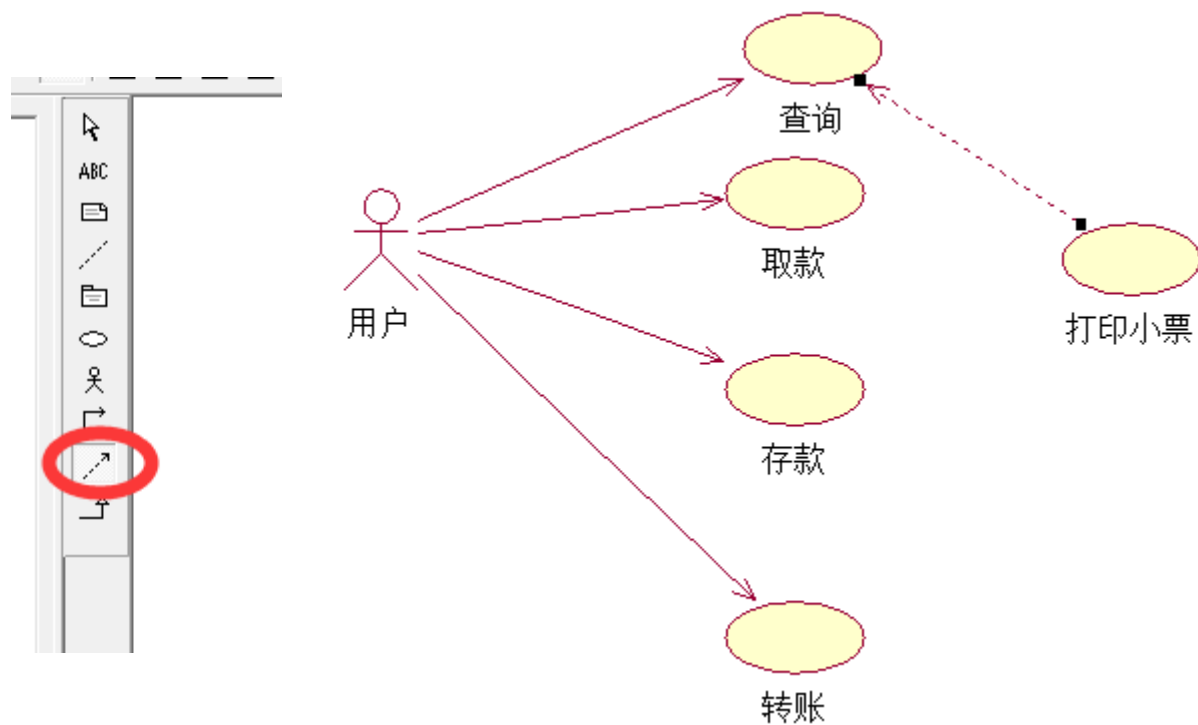
泛化关系



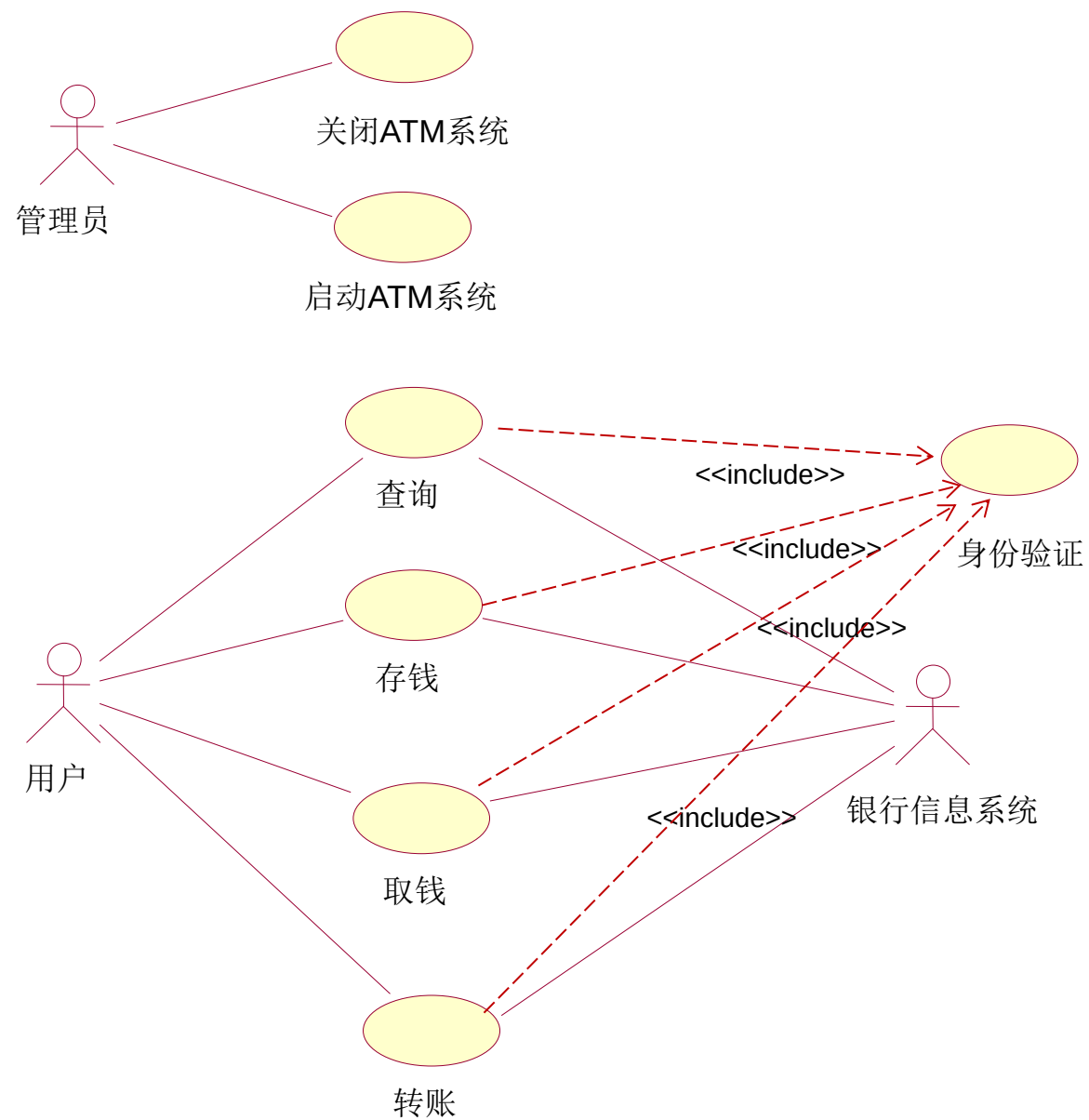
包含关系



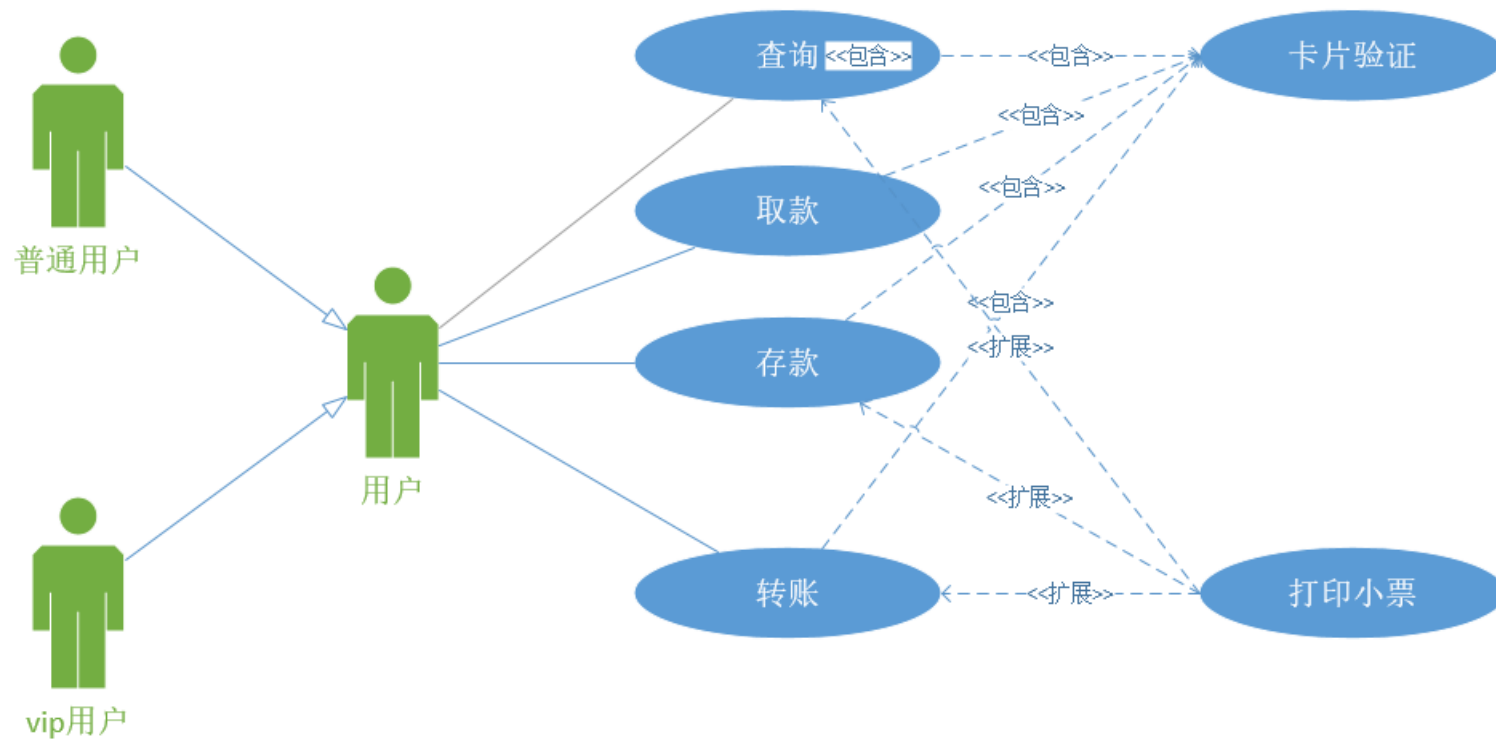
扩展关系



整个ATM系统的用例图



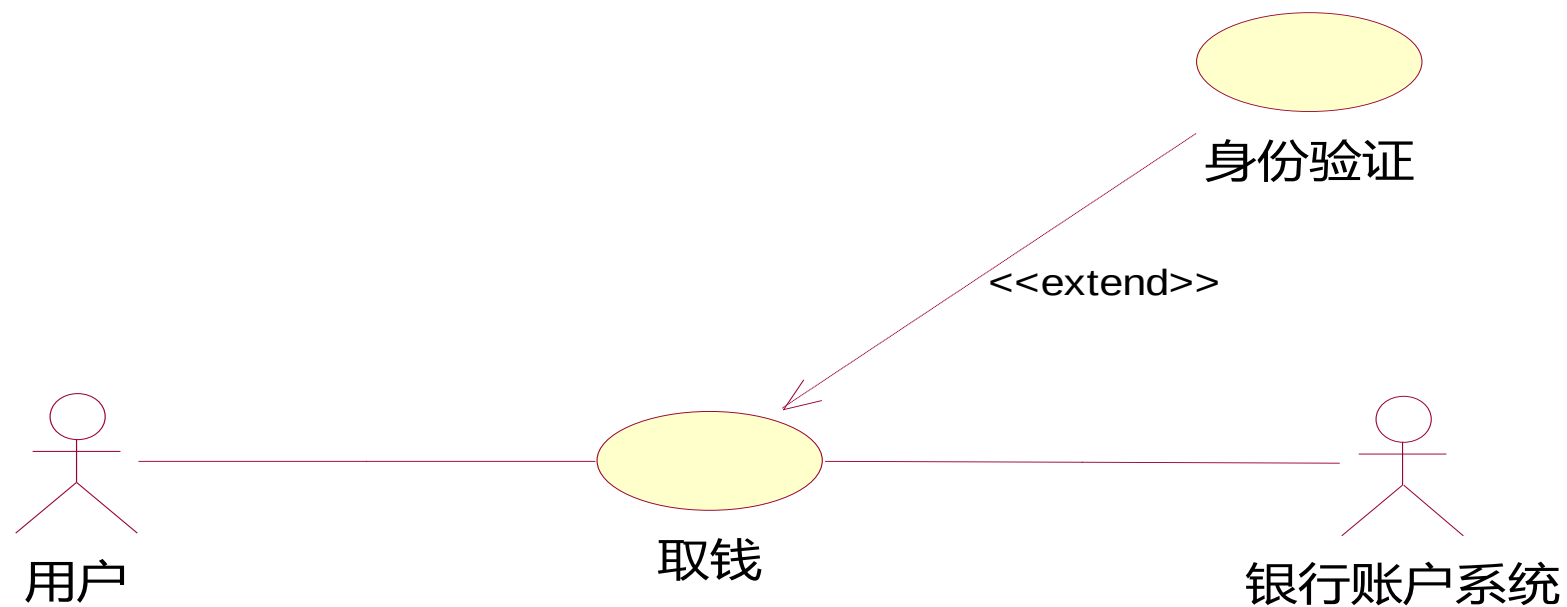
Visio绘制结果



建立用例描述——用例模板

ID	用例的标识，通常会结合用例的层次结构使用 X.YZ 的方式给出。
名称	对用例内容的高度概括，体现了用例描述的任务，通常是“动词+名词”的模式。
用例属性	包括创建者、创建日期、更新历史等。
参与者	描述系统的主要参与者、辅助参与者和每个参与者的目标。
描述	简要描述用例产生的原因，大概过程和输出结果。
优先级	用例描述的需求的优先级。
触发条件	标识启动用例的事件，可能是系统外部调用，也可能是系统内部的事件，还有可能是正常流程的第一个步骤。
前置条件	用例能够正常启动和工作的系统状态条件。
后置条件	用例执行完成后的系统状态条件。
正常流程	在常见和符合预期的条件下，系统与外界的行为交互序列。
分支流程	用例中可能发生的非常常见的其他合理场景。
异常流程	在非预期的错误条件发生时，系统与外界进行的行为交互序列。
相关用例	记录和该用例存在关系的其他用例。
业务规则	可能会影响用例执行的业务规则。
特殊需求	和用例相关的其他特殊需求，尤其是非功能性需求。
假设	在建立用例时所做的假设。

ATM系统取钱用例



ATM系统取钱用例

ID↵	Usecase.Withdraw↵
名称↵	取钱↵
用例属性↵	创建者：张三↵ 创建日期：2010年8月1日↵ 更新历史：2011年7月1日↵
参与者↵	用户↵ 银行信息系统↵
描述↵	本用例完成用户的取钱交易。输入为储蓄卡信息和与储蓄卡相对应的密码，输出为用户期望的现金和收据↵
优先级↵	优先↵
触发条件↵	用户身份验证成功，并且取钱金额小于账户余额↵
前置条件↵	ATM系统处于开启状态，并且用户身份验证成功↵
后置条件↵	从出钞口吐出现金，账户余额信息进行修改↵
正常流程↵	1).- 向读卡器插入储蓄卡，然后从键盘输入与储蓄卡对应的密码；↵ 2).- 用户从显示器选择取钱服务；↵ 3).- 系统提示用户输入想要取钱的金额；↵ 4).- 用户从键盘输入取钱的金额；↵ 5).- ATM机从出钞口吐出现金，并打印收据。↵
分支流↵	1).- 若用户身份验证失败，则在屏幕上给出提示信息↵ 2).- 若用户取钱金额是否大于账户余额，则在屏幕上给出错误提示，并提示用户是否继续进行取钱交易↵
相关用例↵	身份验证↵

ATM系统查询账户用例

用例名:	查询账户 (Query Account)
概述:	客户得知一个有效银行账户的余额
参与者:	ATM客户
依赖:	包含了“验证PIN码”(Validate PIN)用例
前置条件:	ATM机是空闲的，显示着欢迎信息
主序列:	
可替换序列:	步骤3:如果系统确定所选账户号无效，那么系统显示错误信息并弹出ATM卡
后置条件:	客户查询了账户。

ATM系统查询账户用例

用例名:	查询账户 (Query Account)
概述:	客户得知一个有效银行账户的余额
参与者:	ATM客户
依赖:	包含了“验证PIN码”(Validate PIN)用例
前置条件:	ATM机是空闲的，显示着欢迎信息
主序列:	1.包含“验证PIN码”用例的步骤。 2.客户选择“查询”选项，并输入账户号码。 3.系统读取账户余额。 4.系统打印凭条，显示交易号、交易类型和账户余额信息。 5.系统弹出ATM卡。 6.系统显示欢迎信息。
可替换序列:	步骤3:如果系统确定所选账户号无效，那么系统显示错误信息并弹出ATM卡
后置条件:	客户查询了账户。

ATM系统转账用例

用例名:	转账 (Transfer Funds)
概述:	客户将一定数额的资金从一个有效账户转到另一个有效账户
参与者:	ATM客户
依赖:	包含了“验证PIN码”(Validate PIN)用例
前置条件:	ATM机是空闲的，显示着欢迎信息
主序列:	
可替换序列:	
后置条件:	客户资金从指定的“转出账户”转到指定的“转入账户”。

ATM系统转账用例

用例名:	转账 (Transfer Funds)
概述:	客户将一定数额的资金从一个有效账户转到另一个有效账户
参与者:	ATM客户
依赖:	包含了“验证PIN码”(Validate PIN)用例
前置条件:	ATM机是空闲的，显示着欢迎信息
主序列:	1.包含“验证PIN码”用例的步骤 2.客P选择“转账”选项并输入转账 “金额” (amount),“转出账户” (from account)和“转入账户”(to account) 3.如果系统确定“转出账户”里有足够的金额， 那么系统进行转账 4.系统打印凭条， 显示交易号、交易类型、转账金额和账户余额信息。 5.系统弹出ATM卡 6.系统显示欢迎信息.
可替换序列:	步骤3: 如果系统确定“转出账户”号码无效.那么系统显示错误信息并弹出ATM卡。 步骤3:如果系统确定“转入账户”号码无效， 那么系统显示错误信息并弹出ATM卡。 步骤3:如果系统确定客户的“转出账户”上没有足够的金额， 那么系统显示道歉信息并弹出 ATM卡。
后置条件:	客户资金从指定的“转出账户”转到指定的“转入账户”。

UML 活动图 (过程模型)

活动图具有广泛地用途,在软件建模中,活动图可以用来:

1. 描述 workflow

workflow:也被称为业务流程,属于业务活动的过程,可以用活动图来描述。

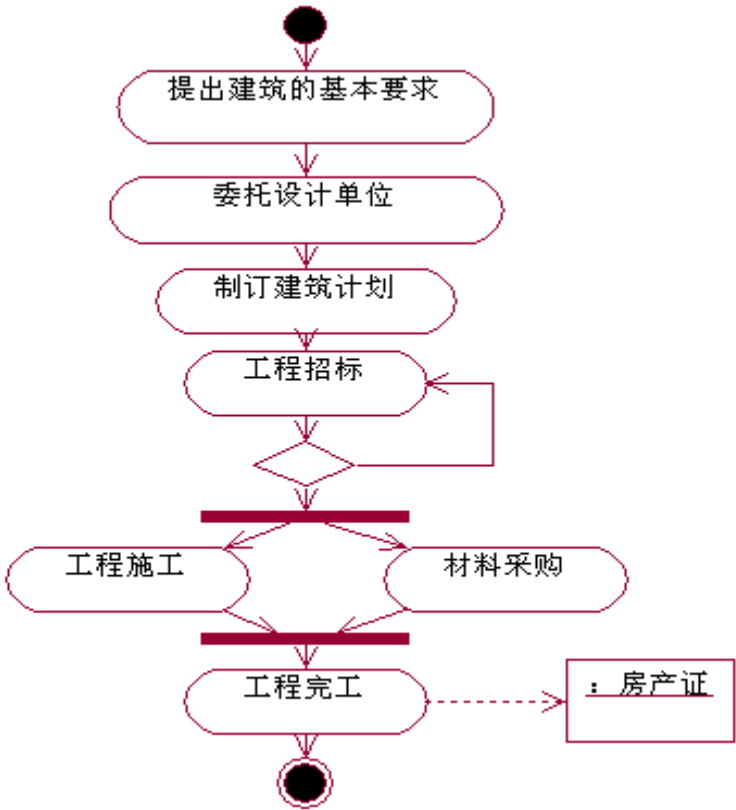
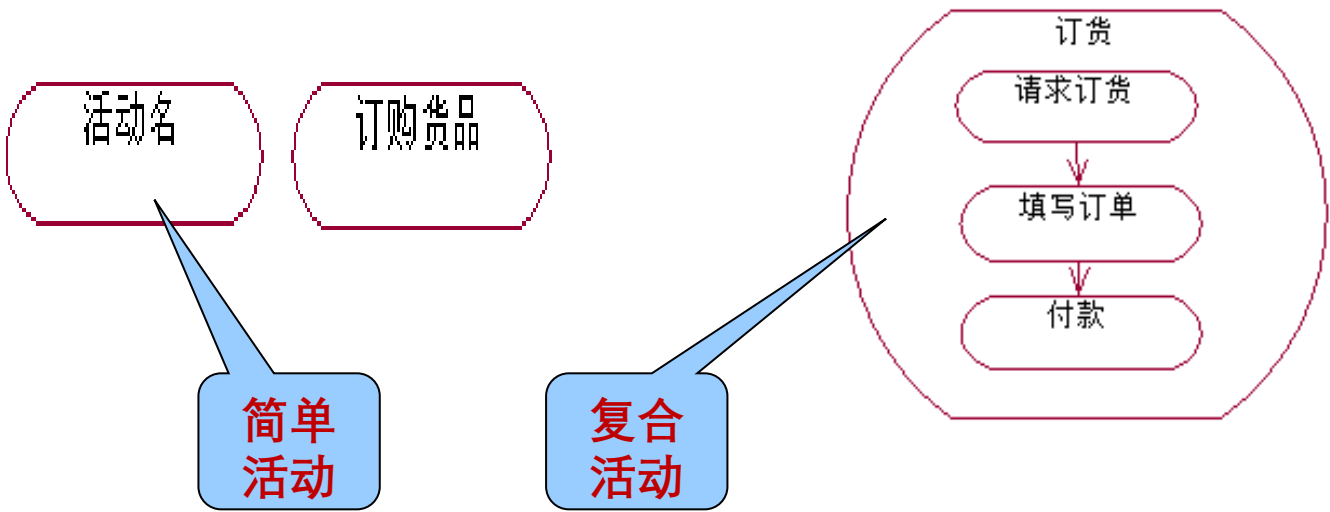
2. 描述工程组织过程

各种工程组织过程,管理过程均可以用活动来描述。

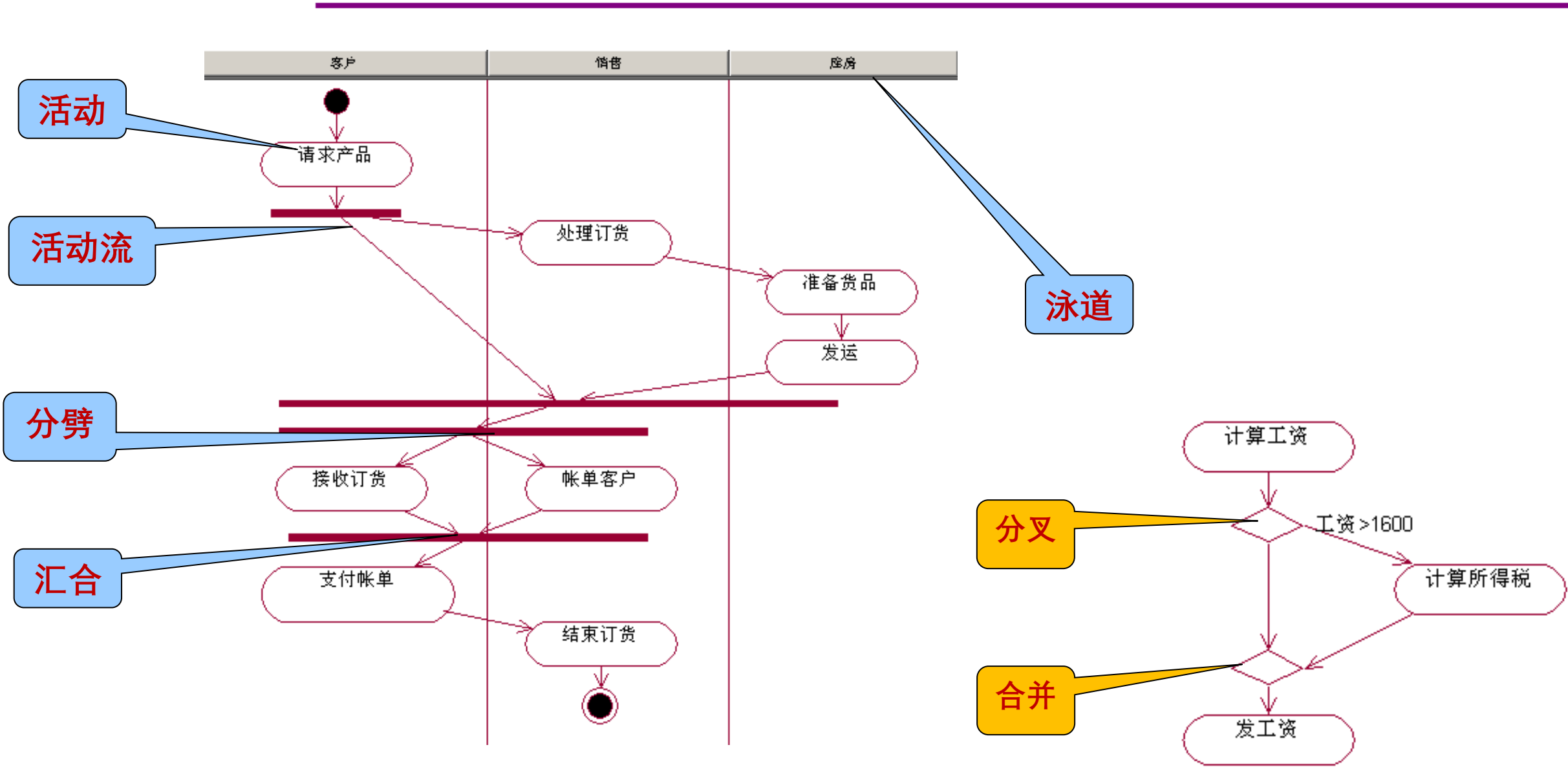
3. 描述算法流程

1. 活动图的概念

活动图(activity diagram)是UML的动态视图之一，用来描述事物或对象的活动变化流程。



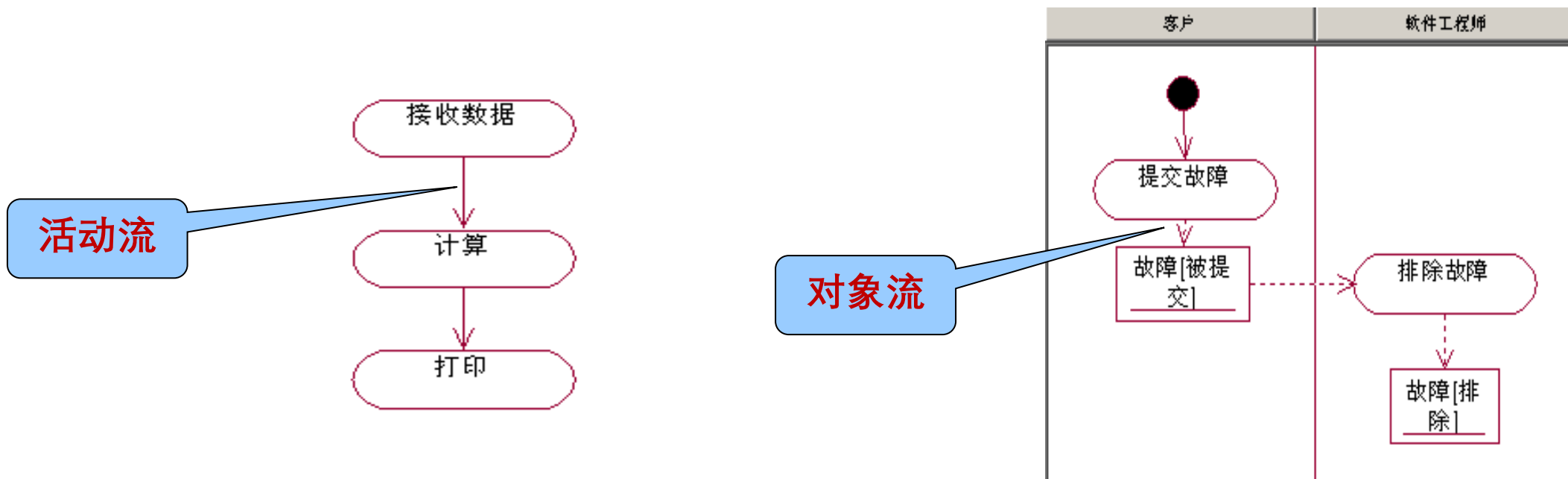
活动图



活动图

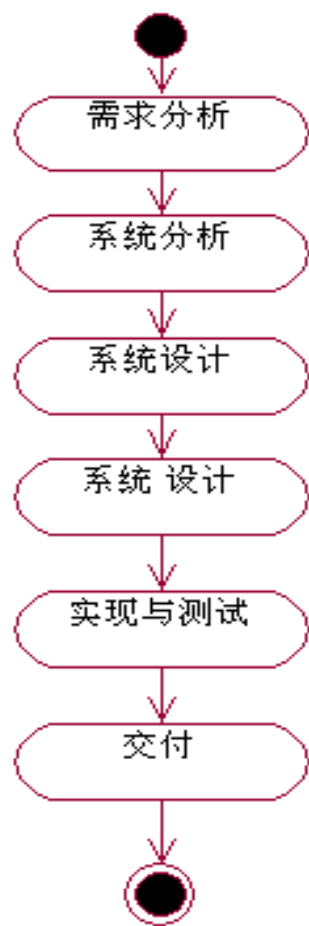
活动流(ActionFlow): 描述活动之间的有向关系，反映一个活动向另外一个活动之间的转移。用带箭头的实线表示。

对象流: 反映活动与对象之间的依赖关系，表示对象对活动的作用或活动对对象的影响，用依赖关系表示。

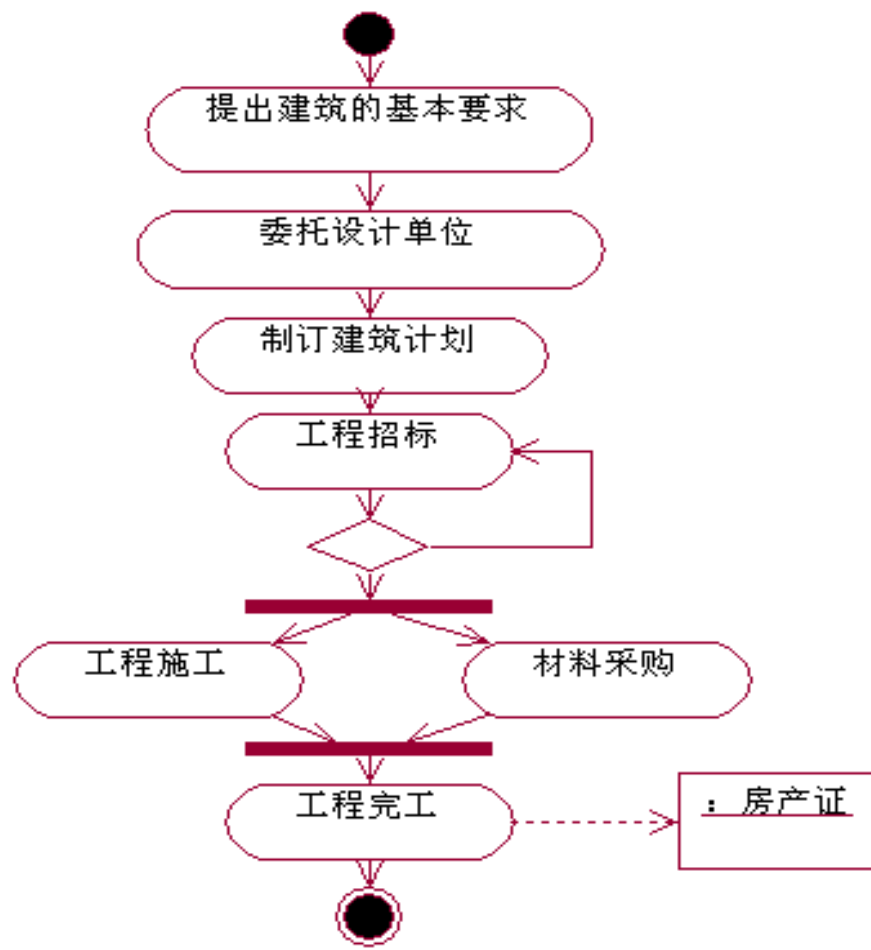
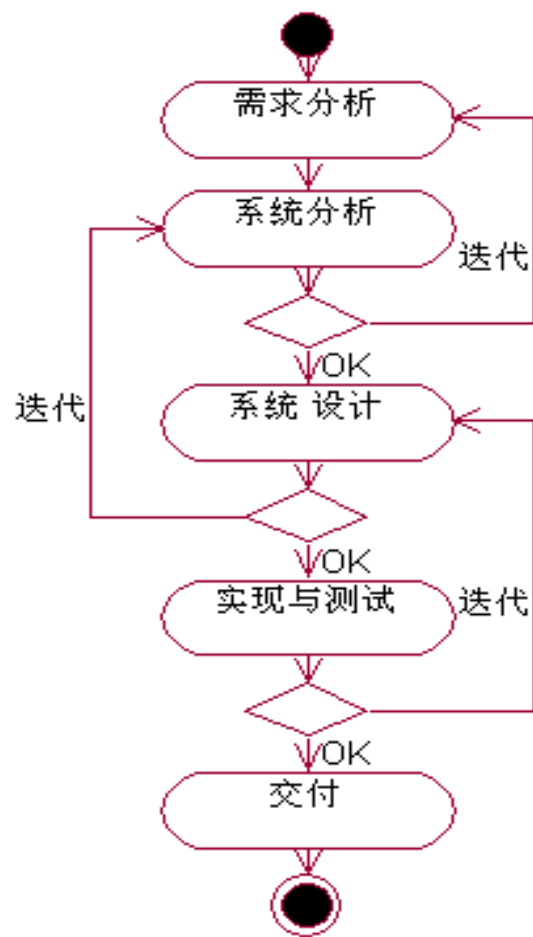


活动图

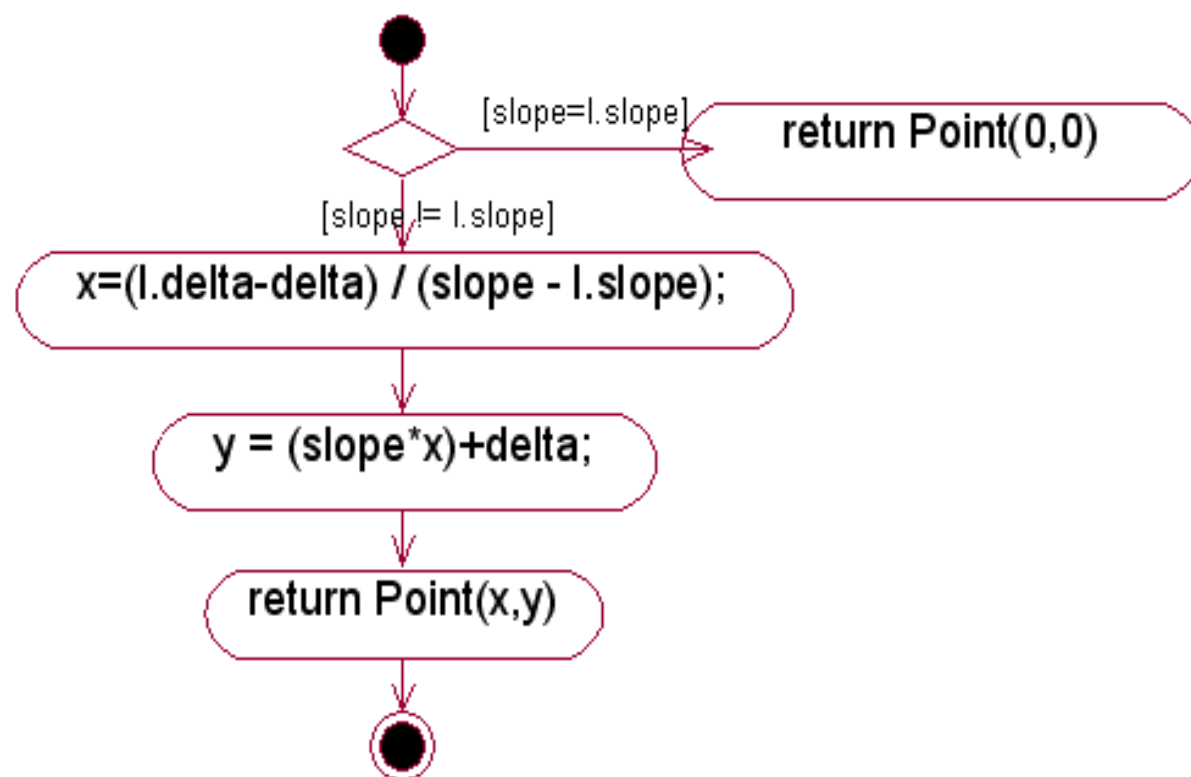
非迭代软件开发过程



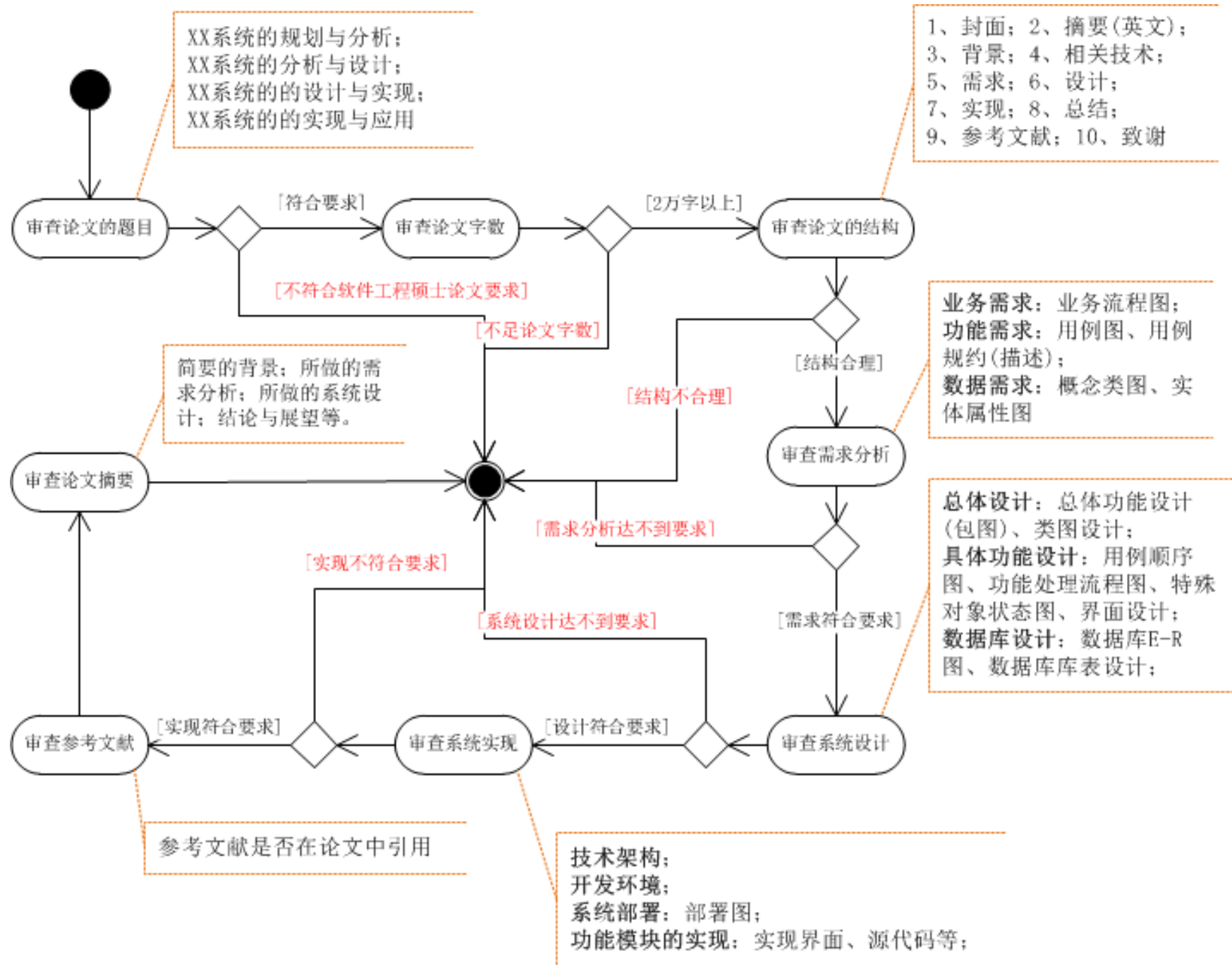
迭代软件开发过程



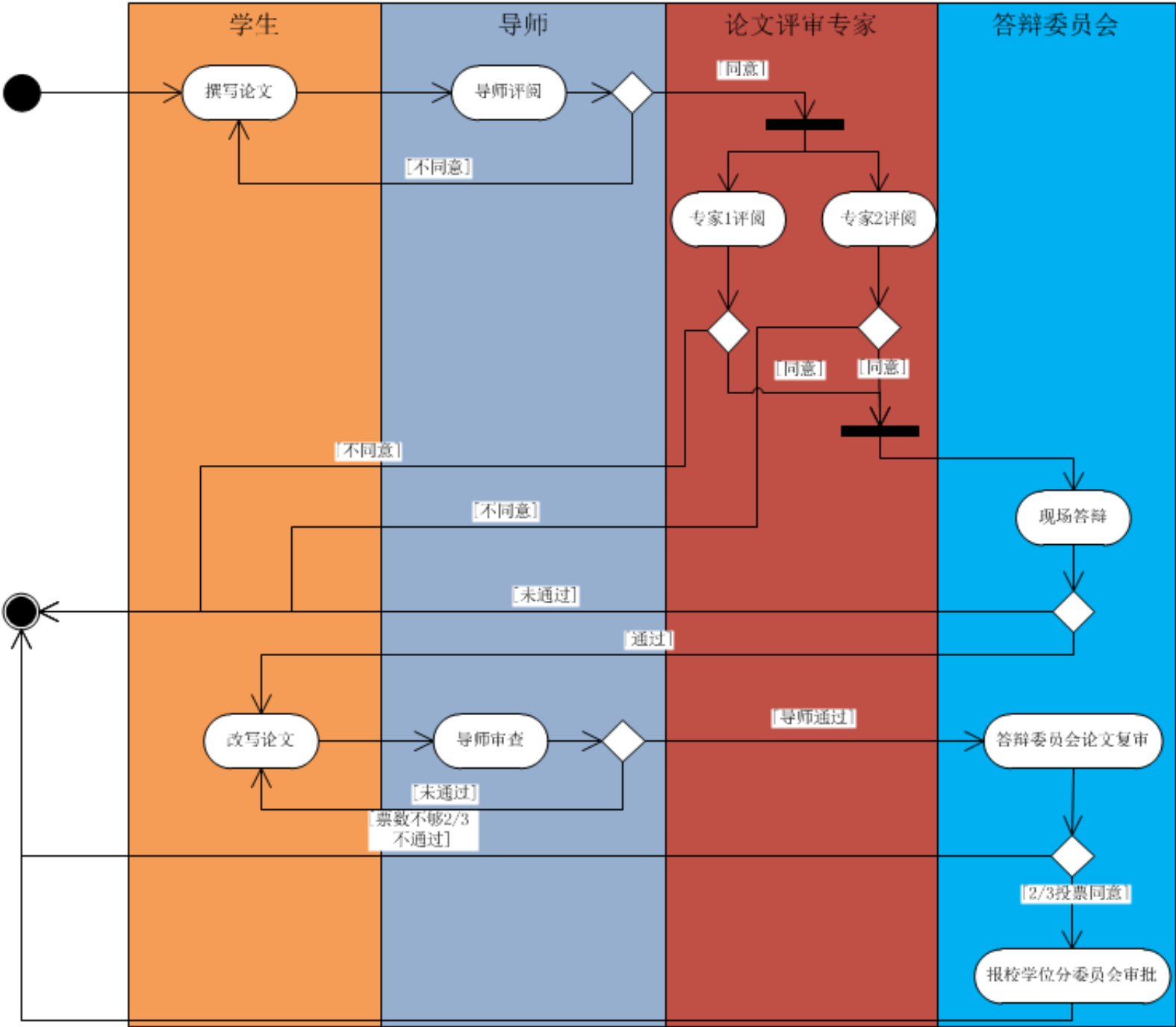
活动图



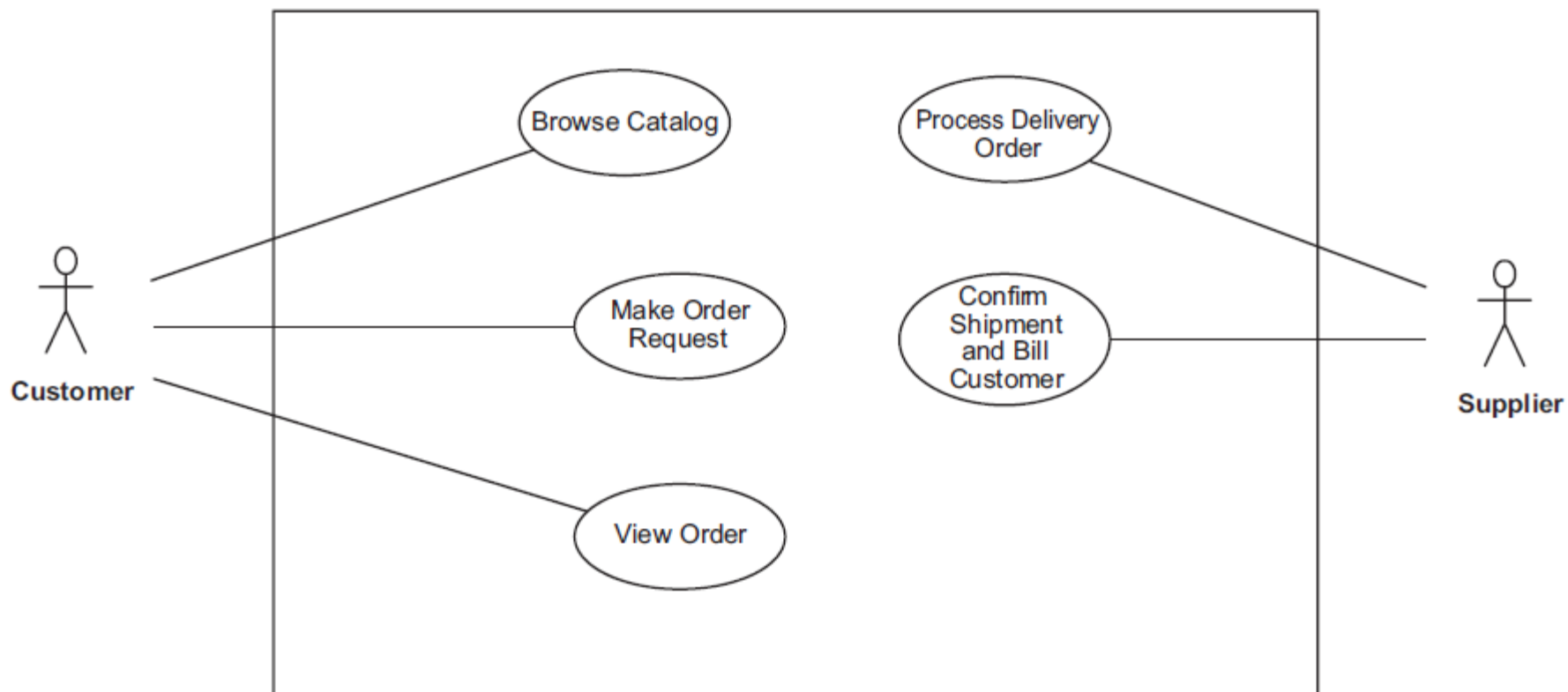
活动图



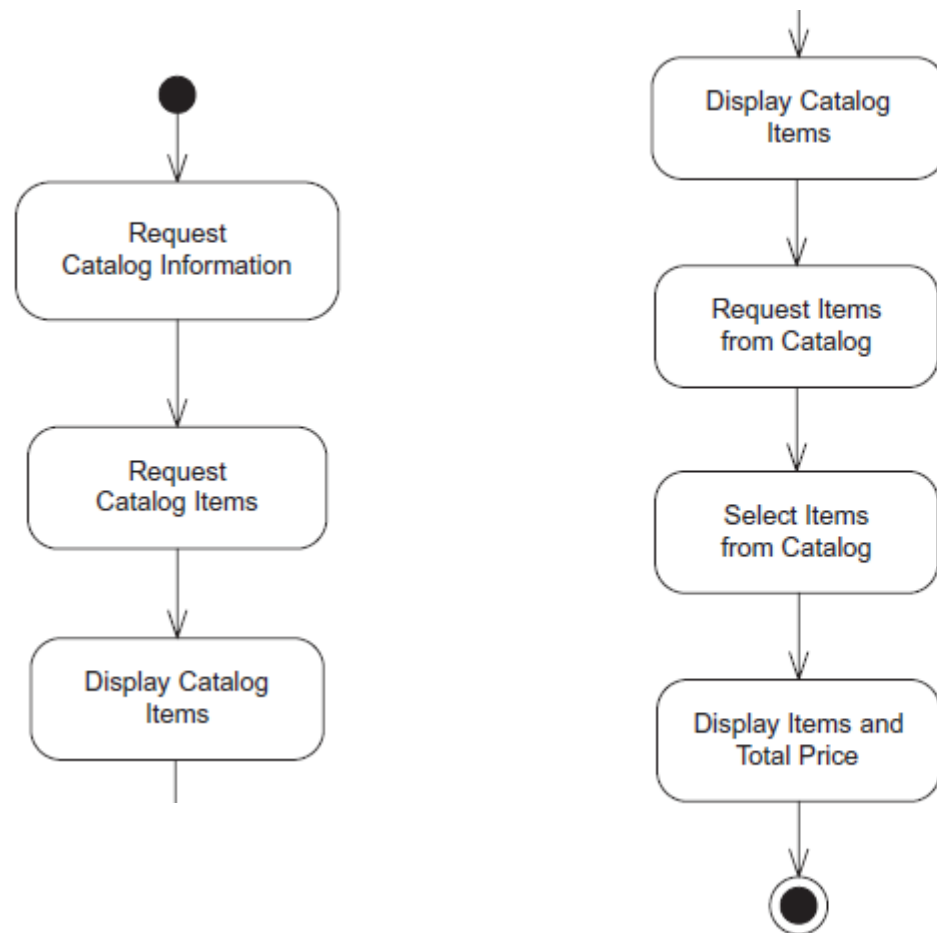
活动图



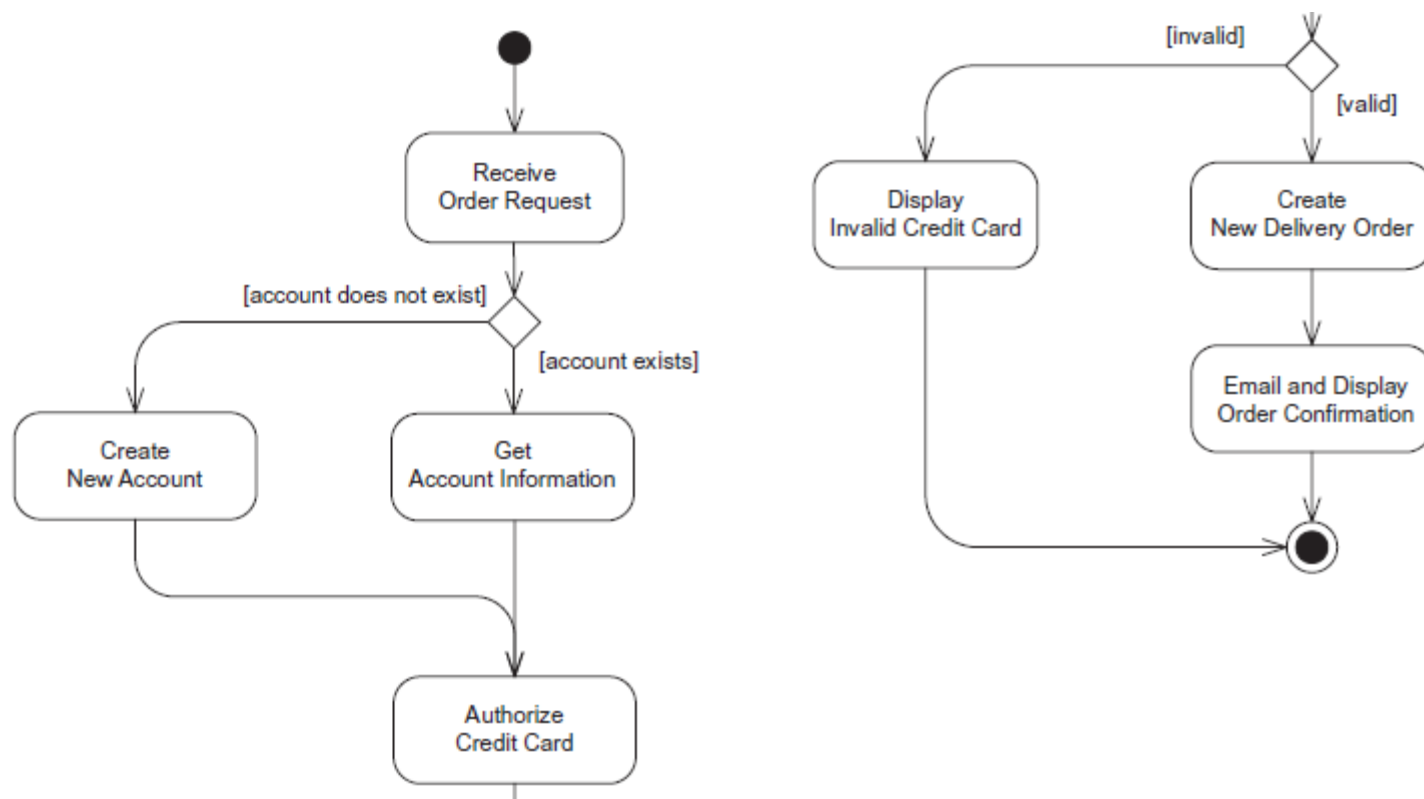
面向服务的体系结构案例研究：在线购物系统



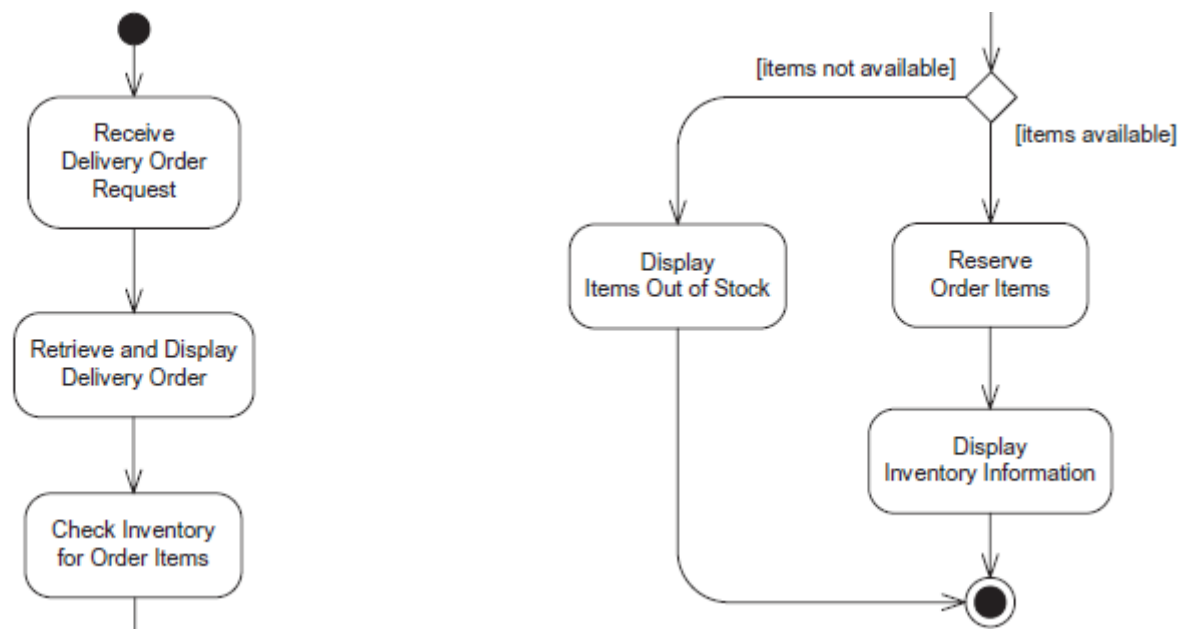
浏览目录用例活动图



下单请求用例活动图



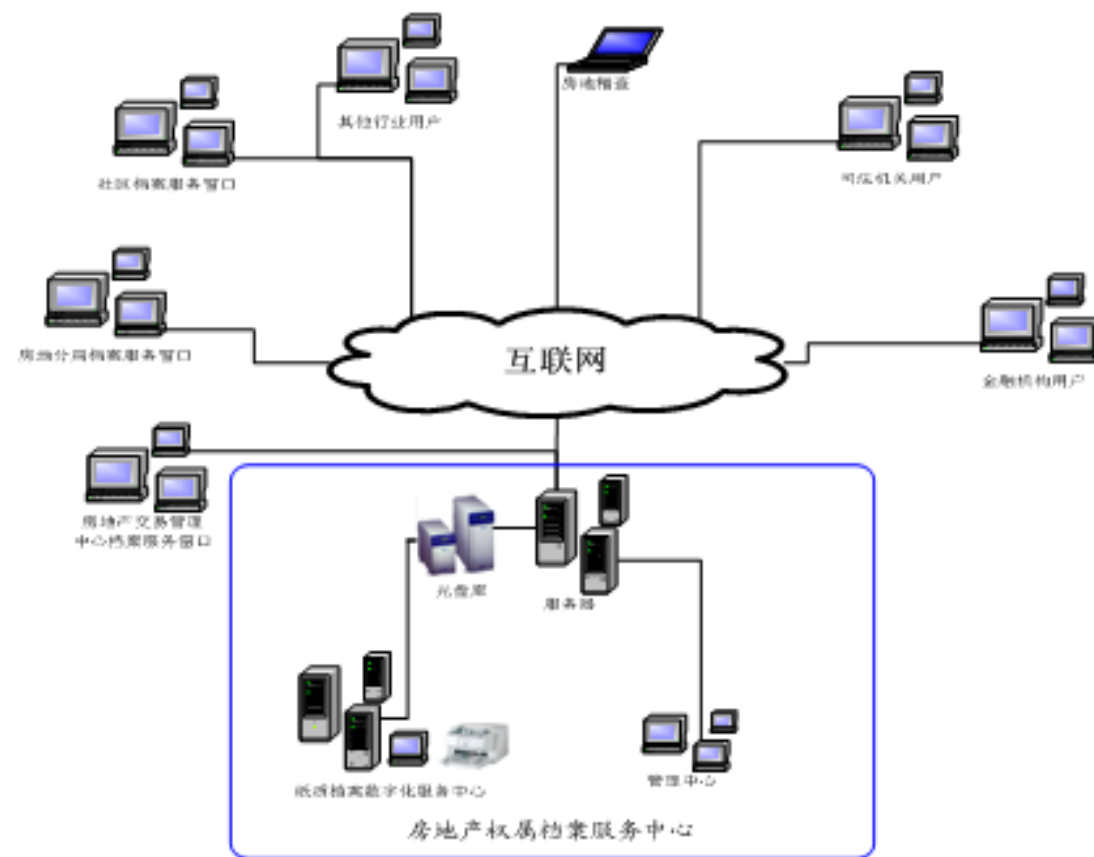
处理配送订单用例活动图



UML 部署图 (物理架构)

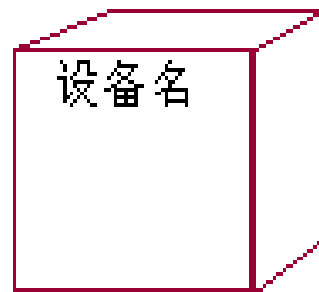
部署图(deployment diagram):
用来描述系统中计算结点的拓扑
结构和通信路径与结点上运行的
软件构件等。

一般一个系统仅有一个部署
图。



西安市房地产权属档案的数字化档案服务平台示意图

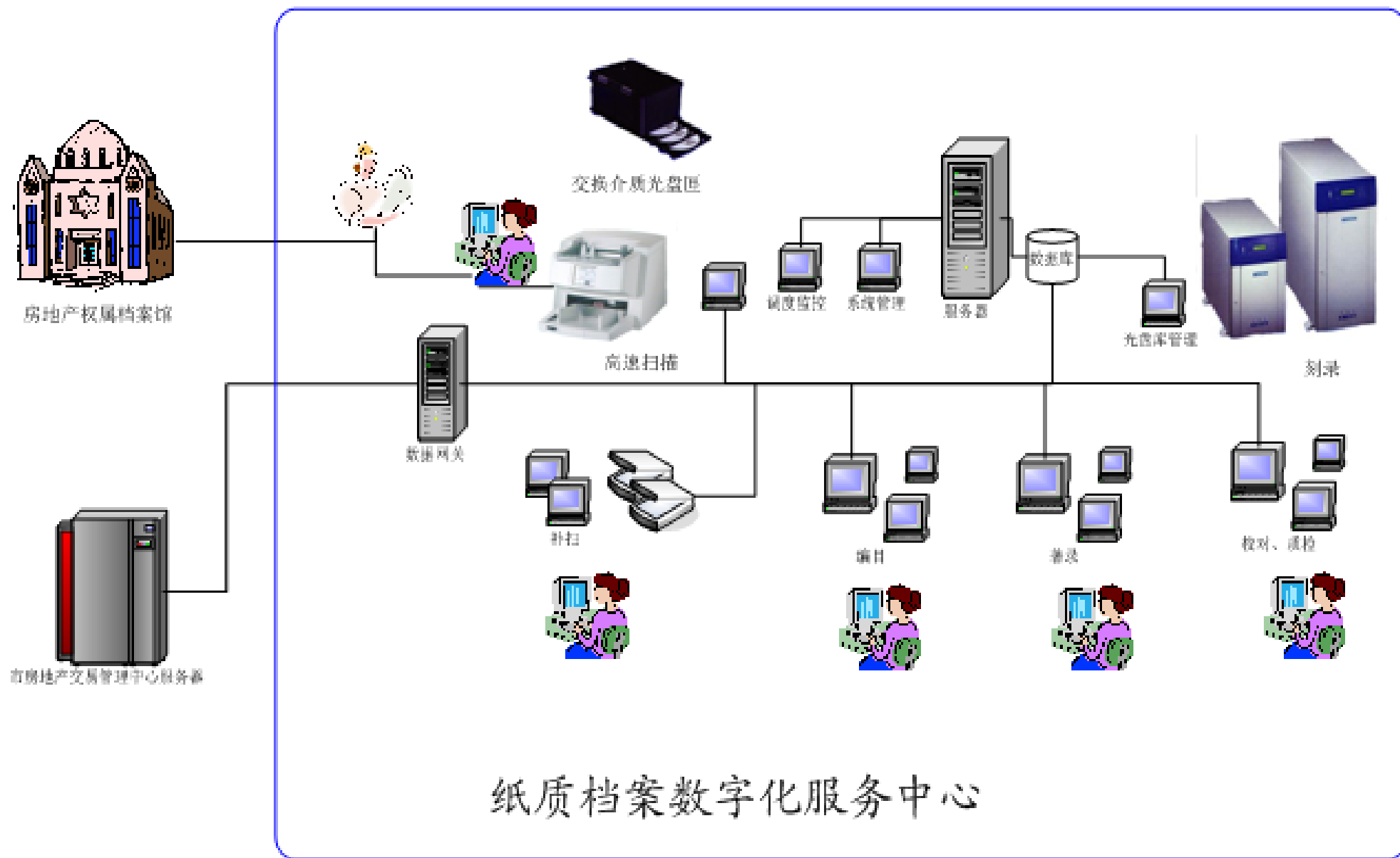
- **结点 (node)** 表示独立计算资源的物理设备，可以分为处理机 (processor) 和设备 (device) 两类。
- 处理机：主机，服务器，客户机等；
- 设备：打印机，传感器，终端等。



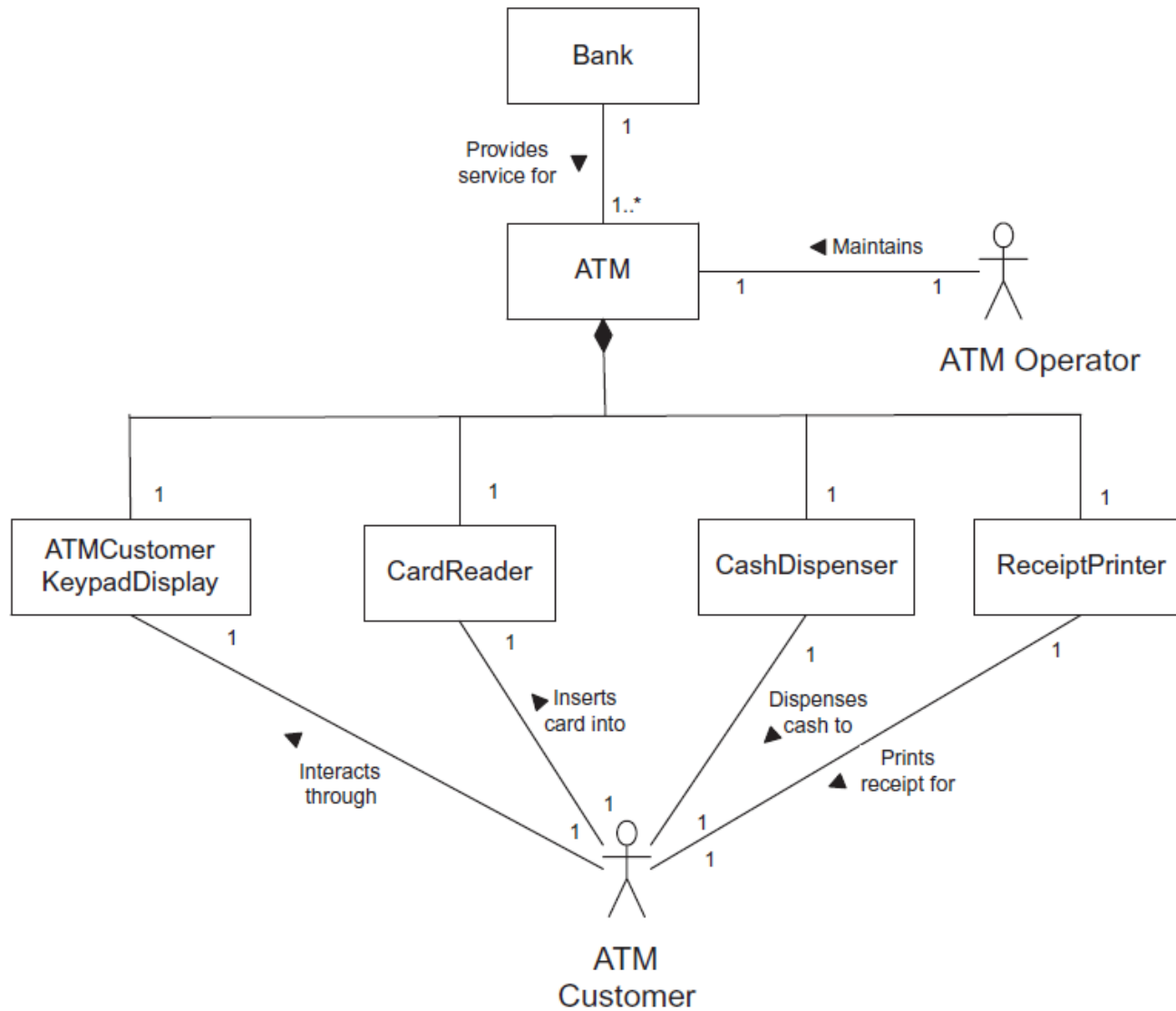
- **连接**表示两个结点之间的物理连接关系，用直线表示，在连接上可以加多重性、角色、约束等。



部署图



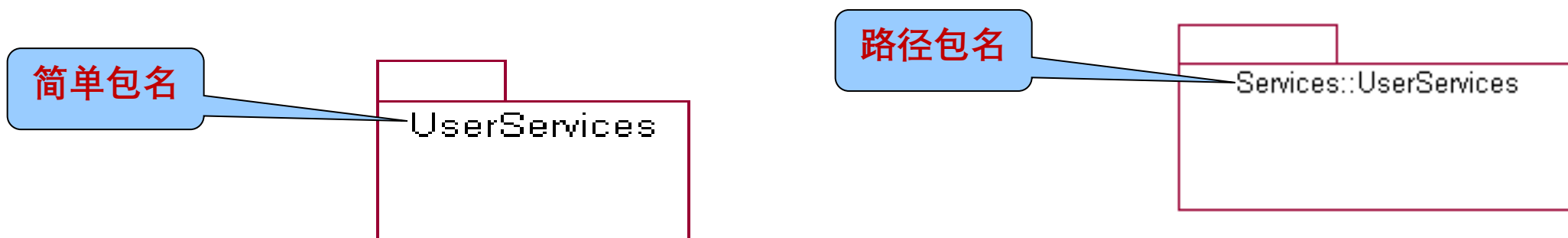
案例-ATM



UML 包图 (功能模型)

包(Package): 是UML用来组织模型元素的模型元素。包中可以包含类、接口、构件、用例、结点、活动、状态、包等其他模型元素。包是对软件模型进行分解、组织的有效模型元素。

包图由包和包之间的联系构成，包图中的图形元素是包，包之间用依赖关系或泛化关系连接。包图是对系统结构建模的重要工具。



包的设计原则

1、重用等价原则

把类放到包中时，尽量把包可以作为可重用的单元。包是可重用的包，包中的类是可重用的类。

2、共同闭包原则

把可能同时修改，同时维护的类放到一个包中，以便于维护，和升级。

3、共同重用原则

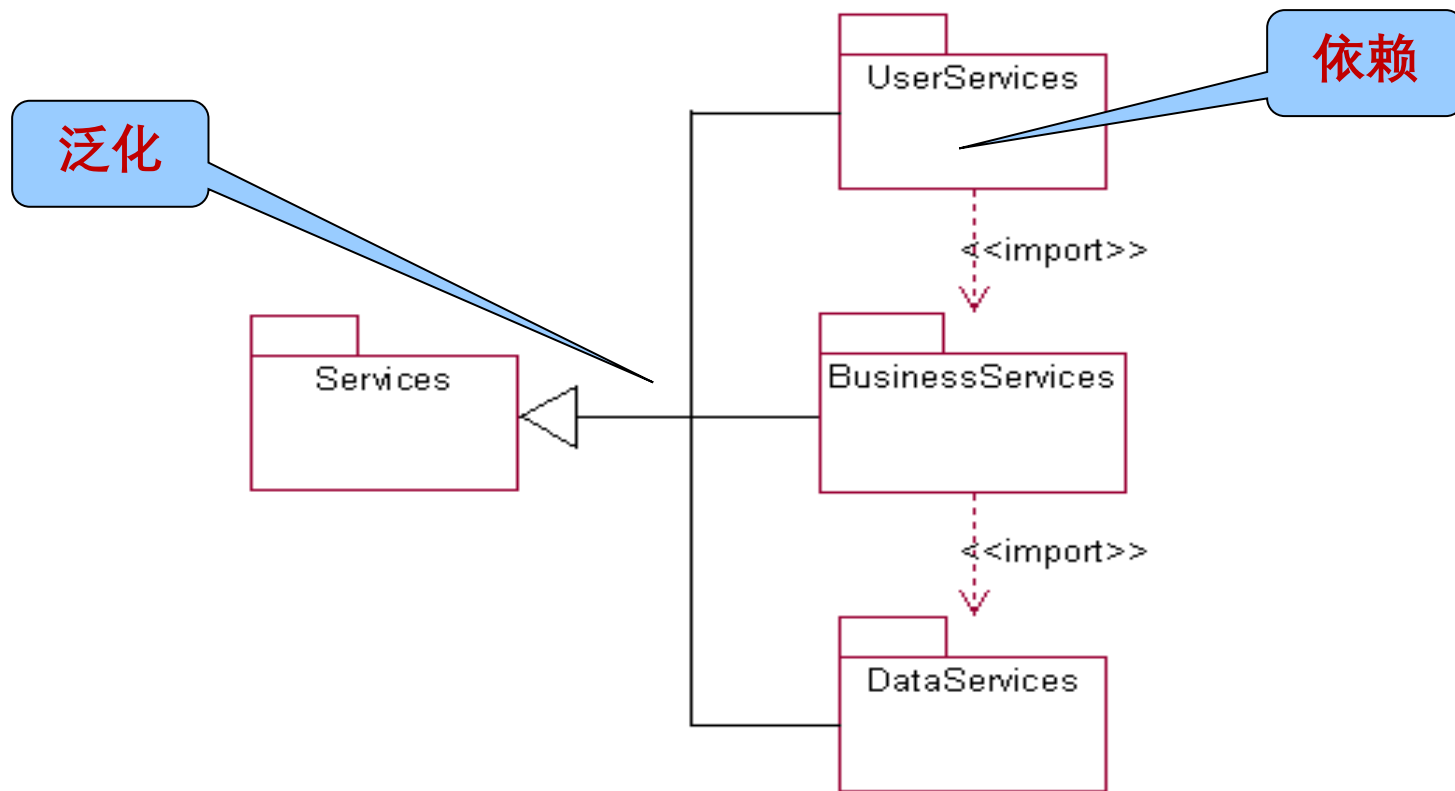
把将会同时，或使用时间相隔不长的建模元素放到同一个包中。

4、非循环依赖原则

包之间的依赖关系不要形成循环：即A依赖B，B依赖C，C又依赖A。

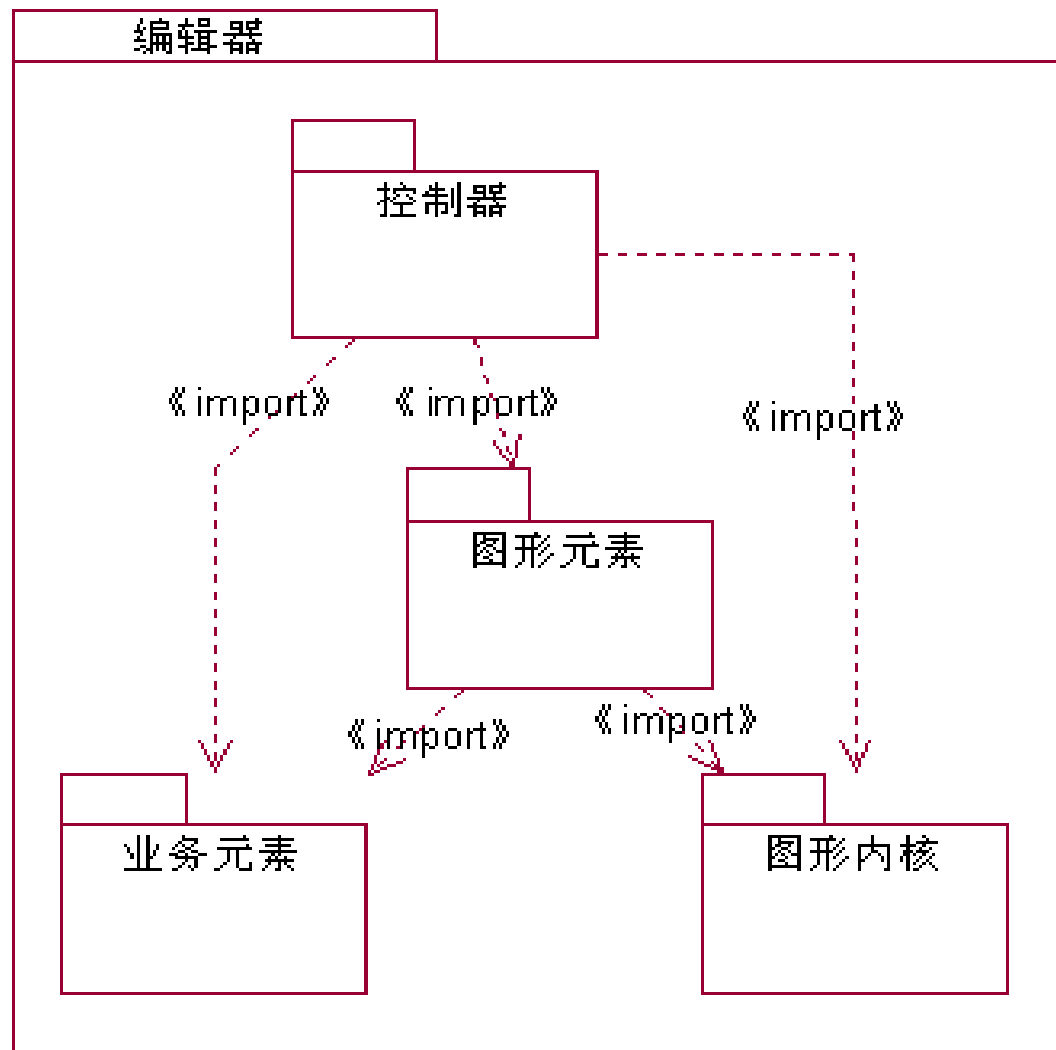
包图

1. 依赖关系：如果一个包要访问或导入另外一个包中的元素，则两个包之间存在依赖关系。包的依赖关系用UML中的虚线箭头来表示,由源包指向被依赖的独立包。在依赖关系上标注《import》导入的构造型。
2. 泛化关系：包之间也可以存在泛化关系。特殊包继承一般包的特性。

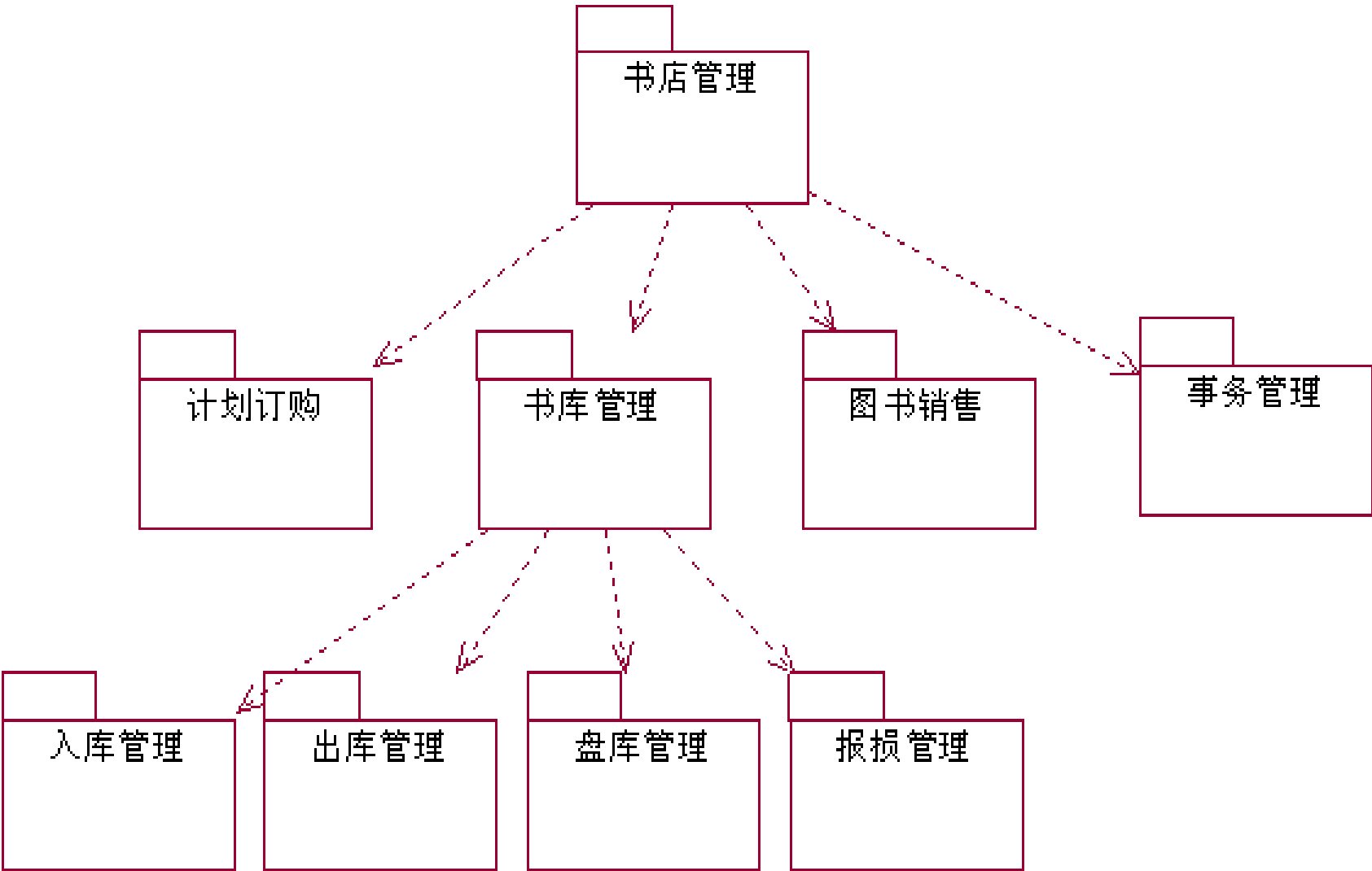


3. 包的嵌套

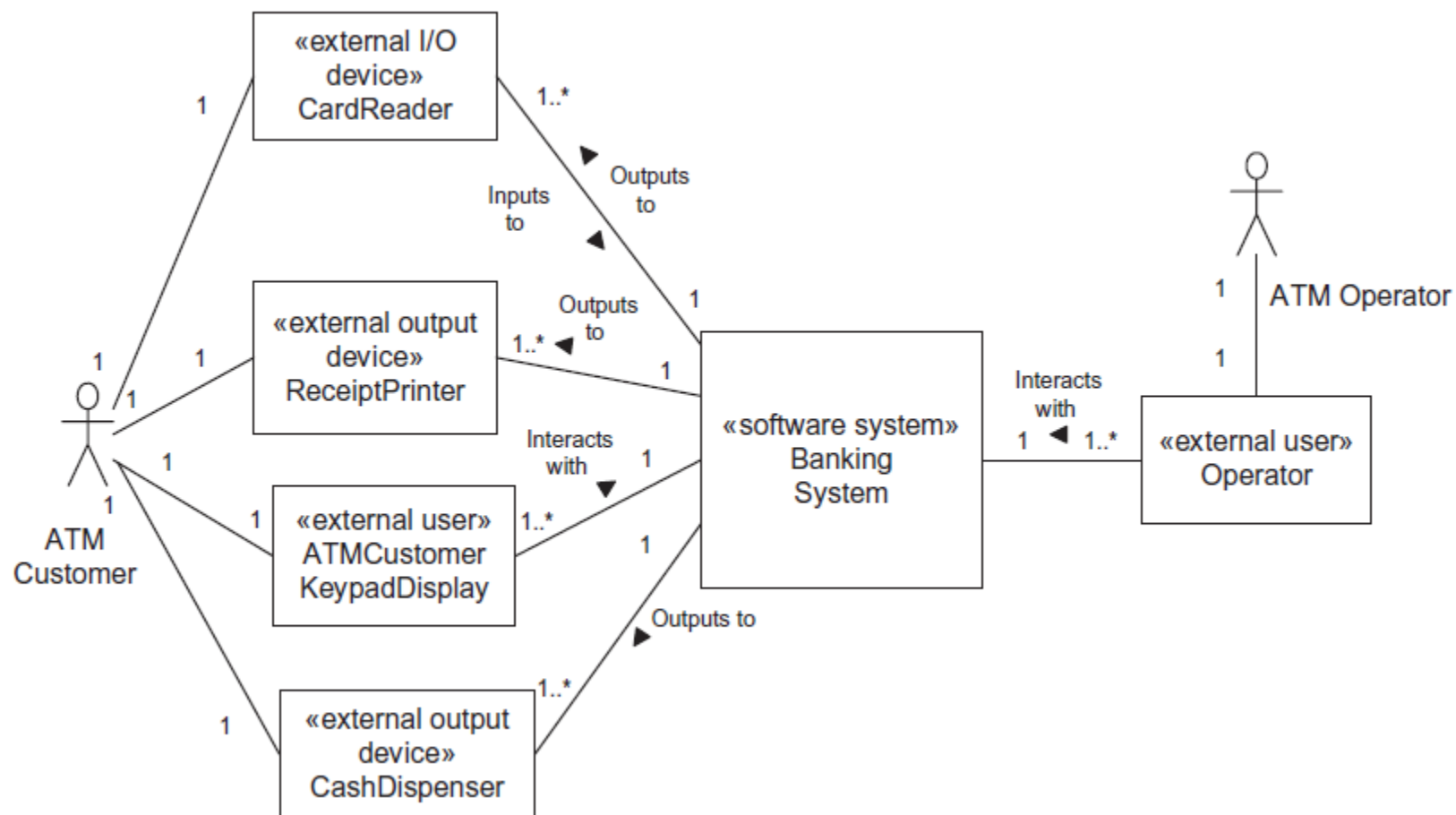
包之中可以嵌套子包。



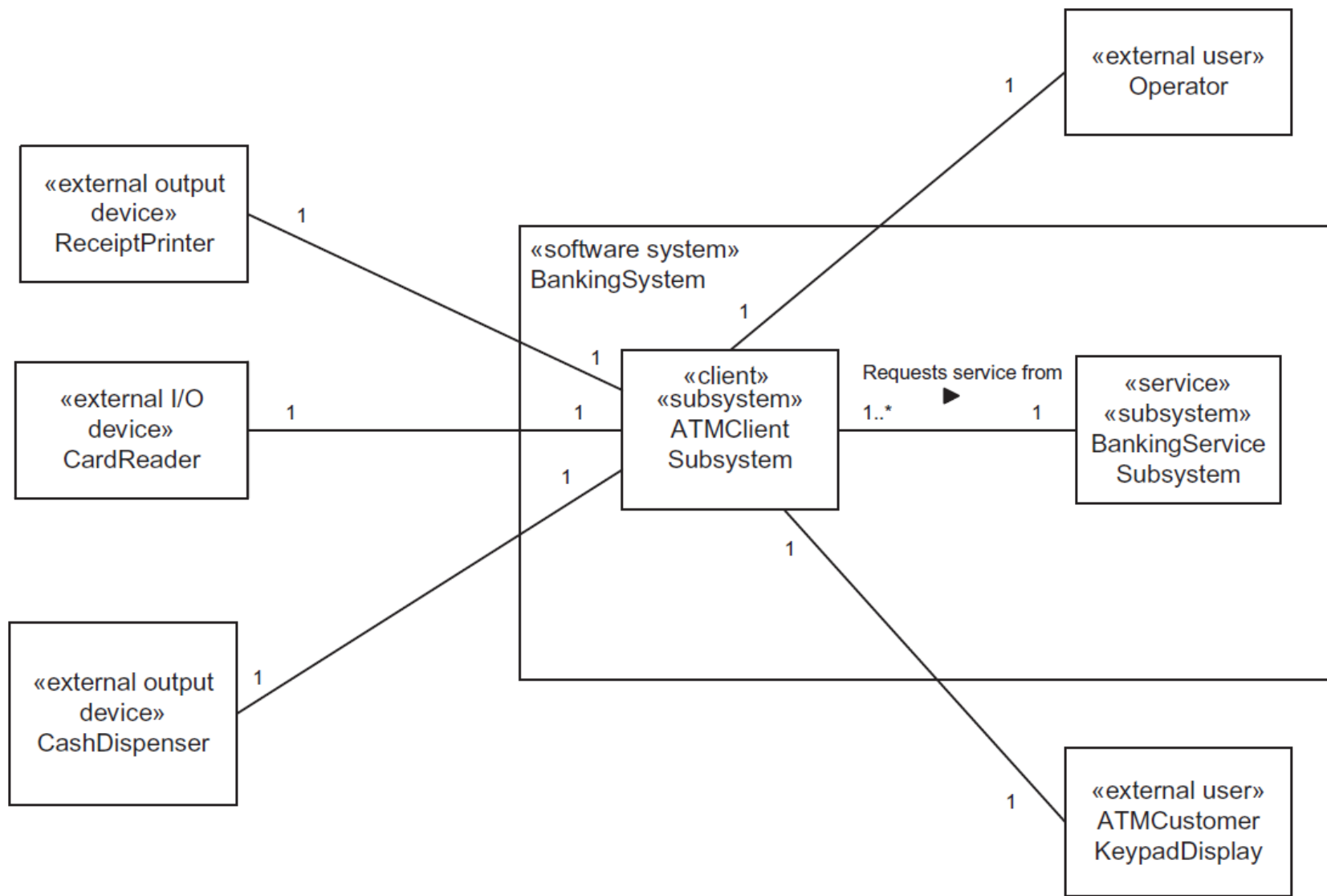
书店图书管理的包图：



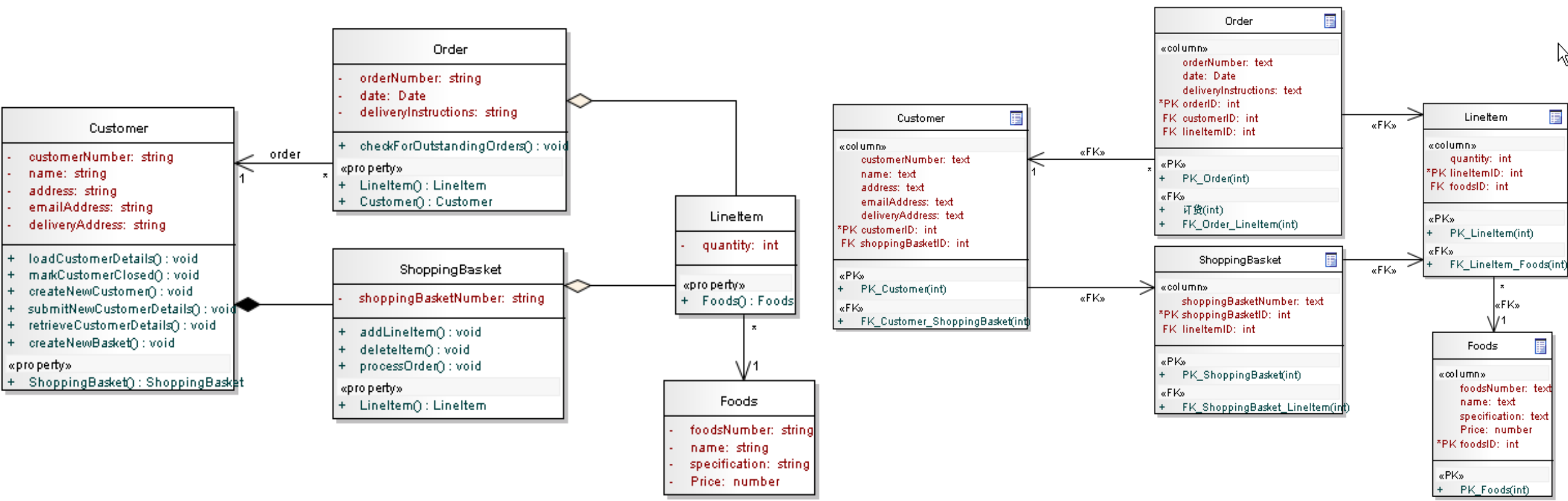
案例-ATM



案例-ATM子系统组织



UML 类图 (类模型, 数据模型)

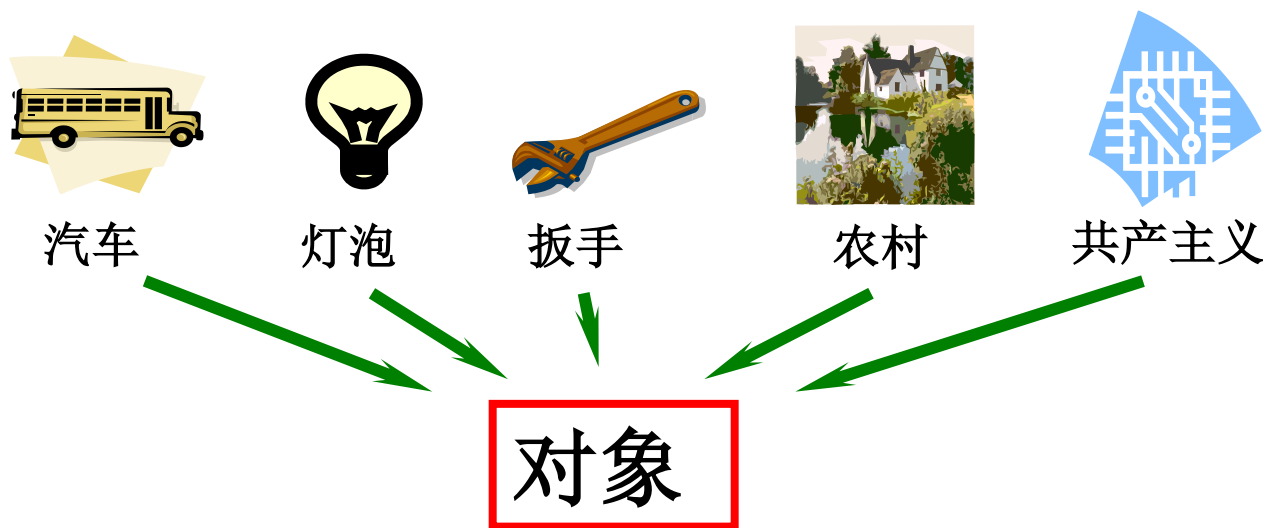


一、对象

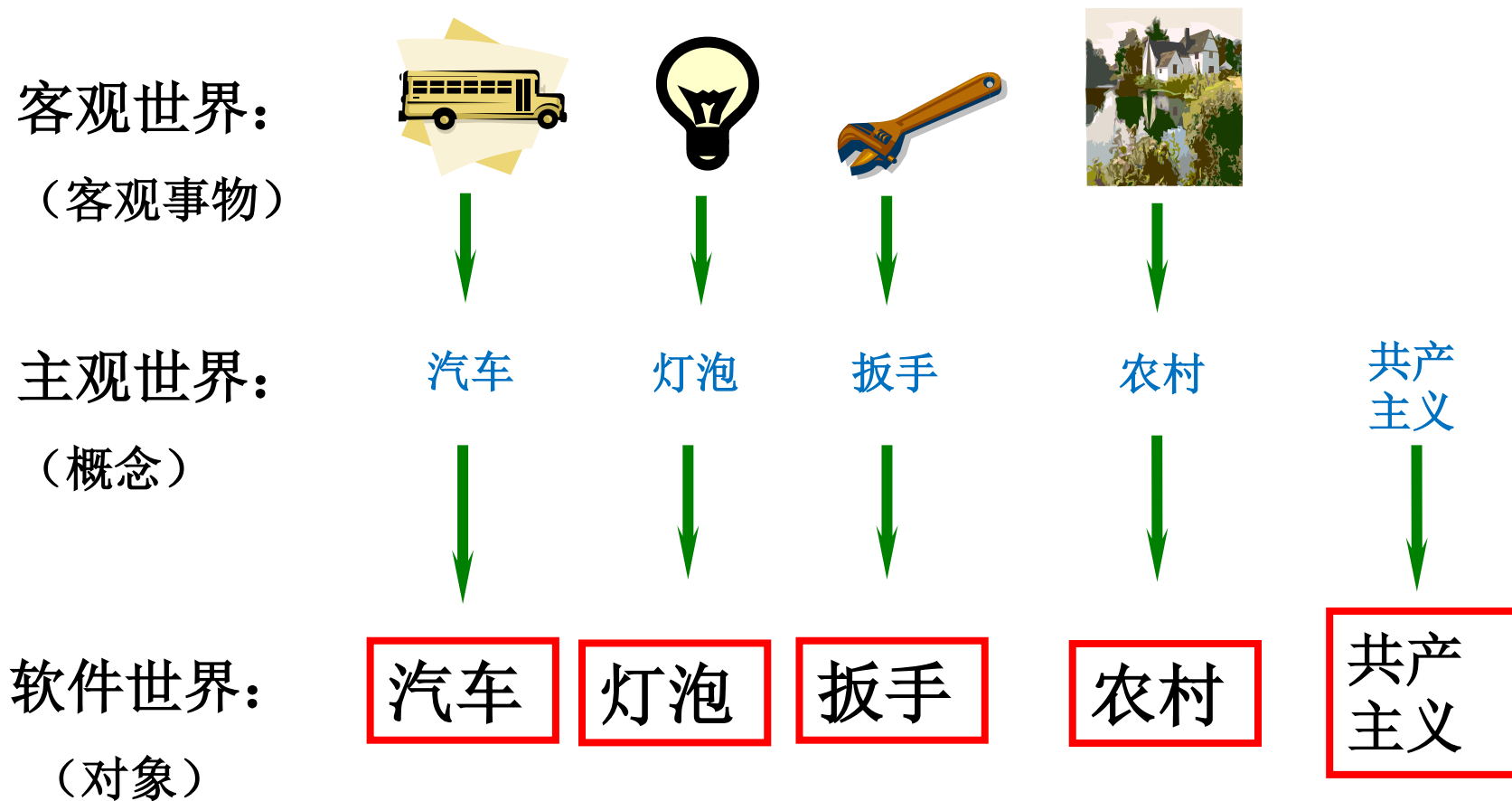
1、对象的概念

对象是系统中用来描述客观事物的一个实体，它是构成系统的一个基本单位。

对象既可以描述客观存在的事物，也可以表示由人主观构造的抽象观念



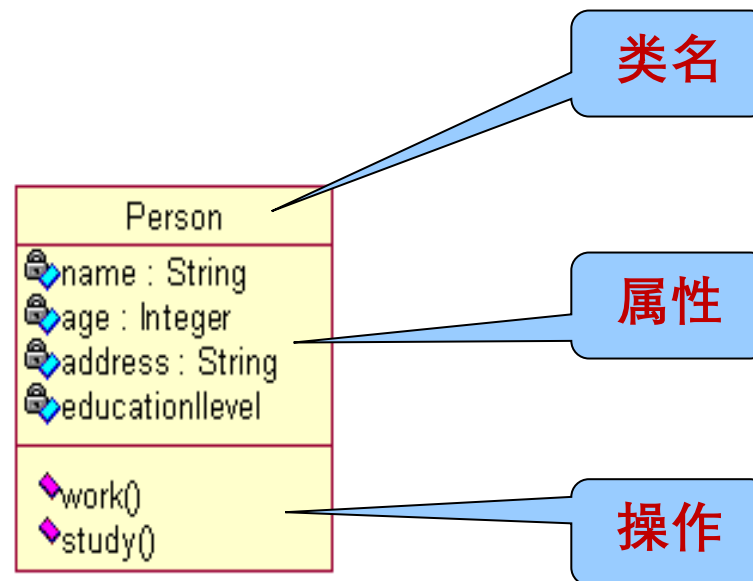
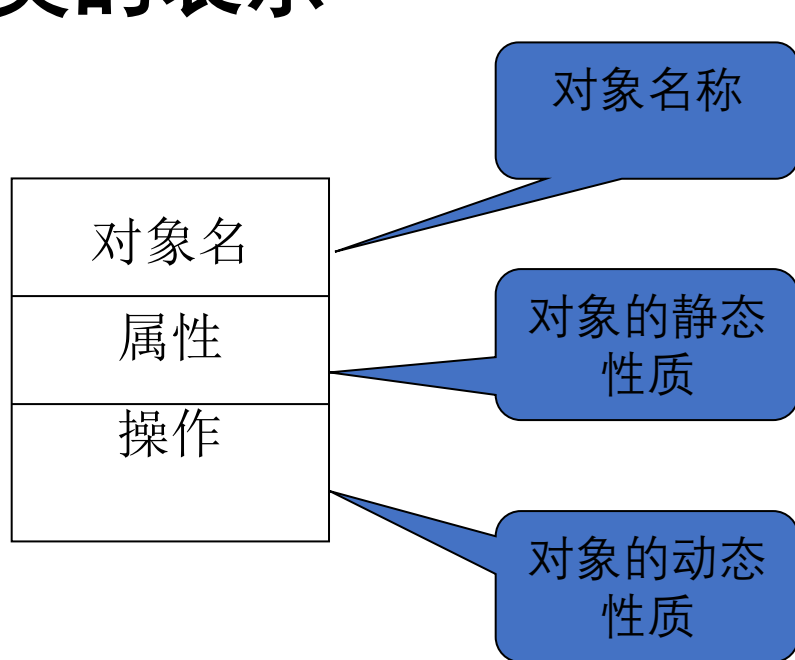
类的概念



1. 类的定义

类(class)：具有相似结构、行为和关系的一组对象。

2. 类的表示



三、封装

1、封装的概念

封装是把某些可以独立存在的内容封装起来，构成一个整体，并把内部细节隐藏起来，仅向外部提供可以提供的外在描述。

2、对象封装的作用

① 保护对象：

防止用户直接存取对象的内部细节。

② 保护客户：

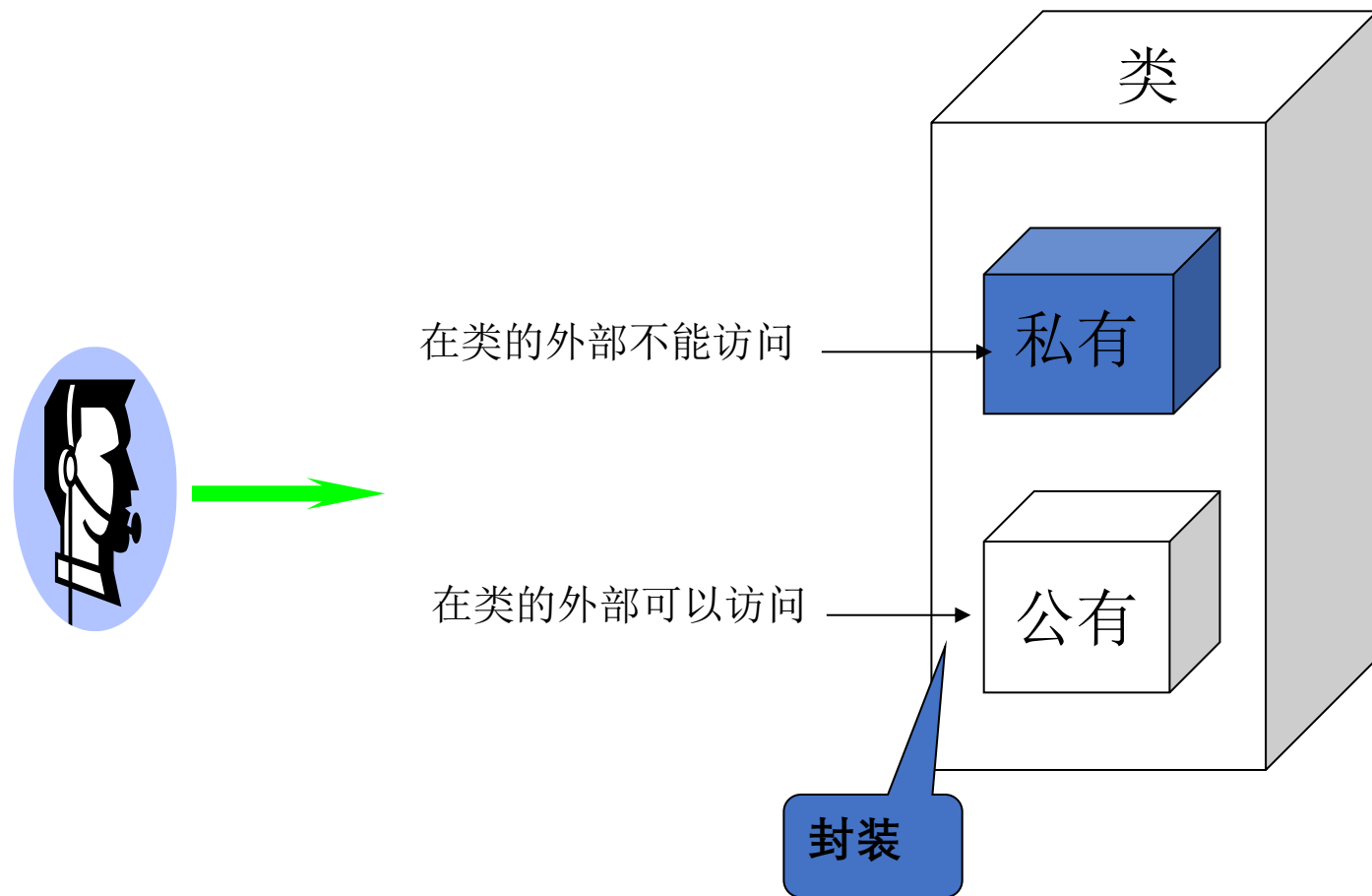
对象内部细节变化，接口不变，并不影响用户对对象的使用和访问。



封装的概念

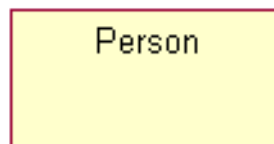
3、对象的封装

对象封装是把对象的属性和操作结合成一个独立的系统单位，并隐藏对象的内部细节，向外部提供对象的接口。

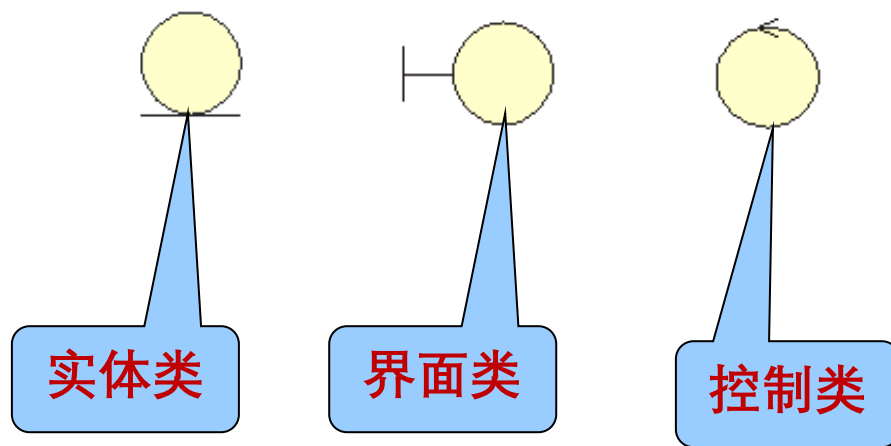


类的其他几种表示形式

① 简化表示



② 缩略表示



边界类(界面类)

①边界类位于系统与外界的交界处,承担系统与外界的信息功能。



边界类(图标)

`<<boundary>>`
边界类(标签)

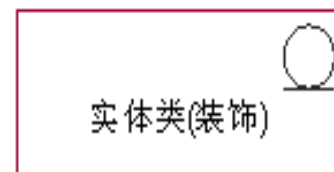
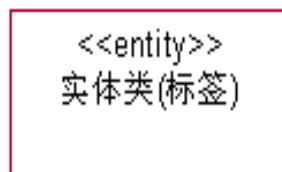


边界类(装饰)

②边界类处在用例图中,参与者与用例的关联处,可以根据用例图发现边界类。

实体类

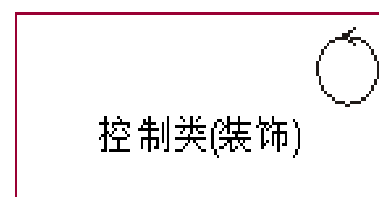
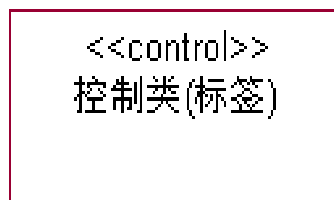
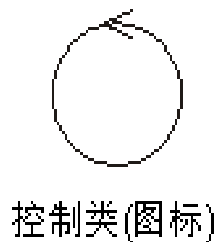
① 实体类对应着现实中的客观实物，用来保存信息，一般对应着数据表、文件等。



② 实体类可以从现实中共存在的客观事物，以及需要持久存放的信息两方面来发现。

控制类

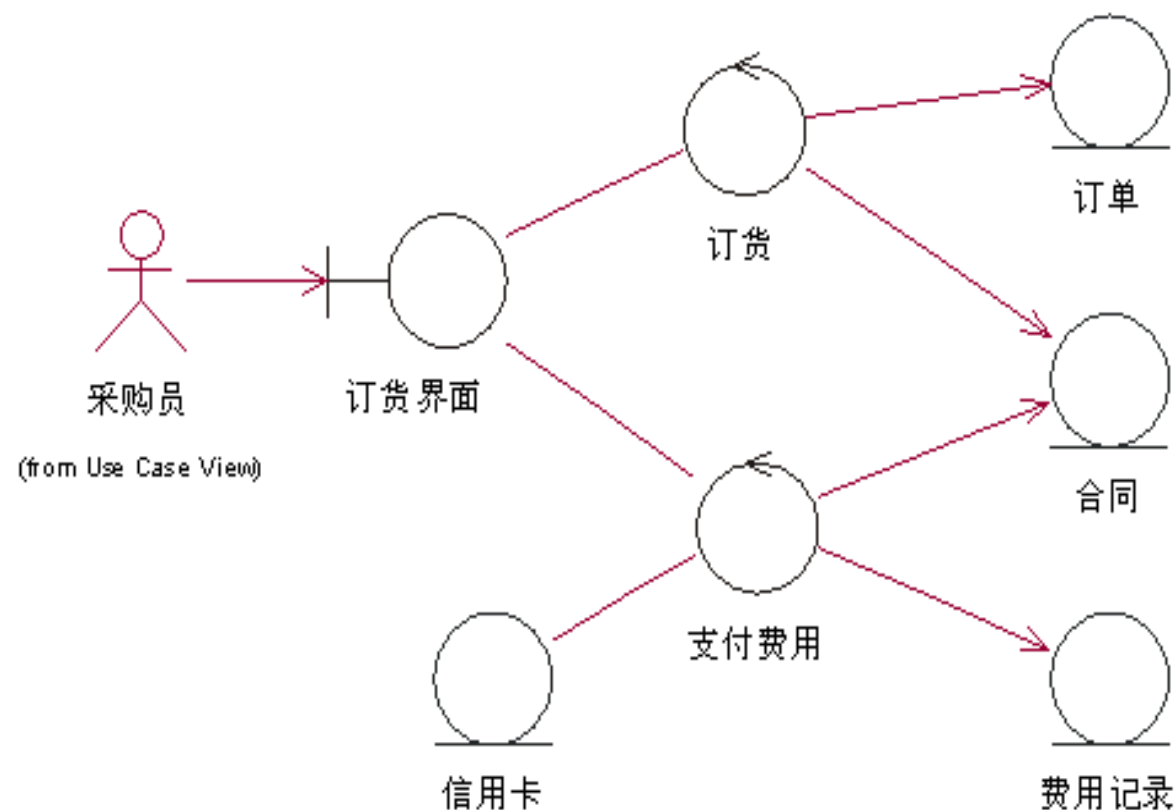
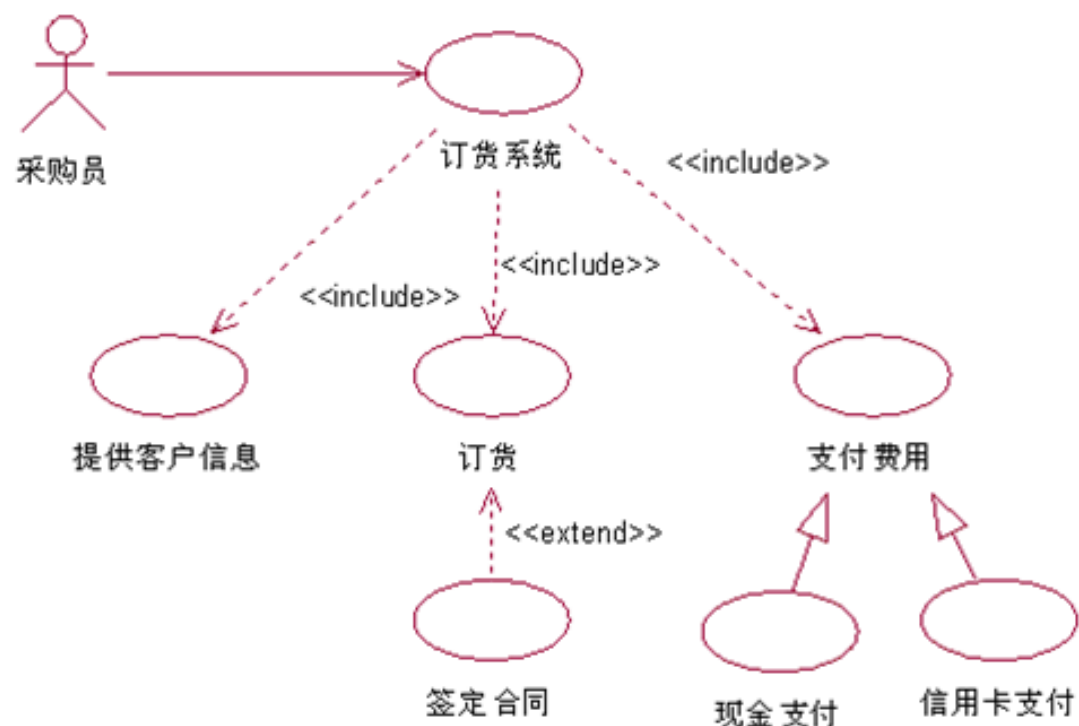
①控制类承担着事务处理，控制调控的控制作用。



②一个用例中最少会有一个控制类，用来控制用例中的事件顺序，也可以在多个用例之间协调用例之间的联系。

类图举例

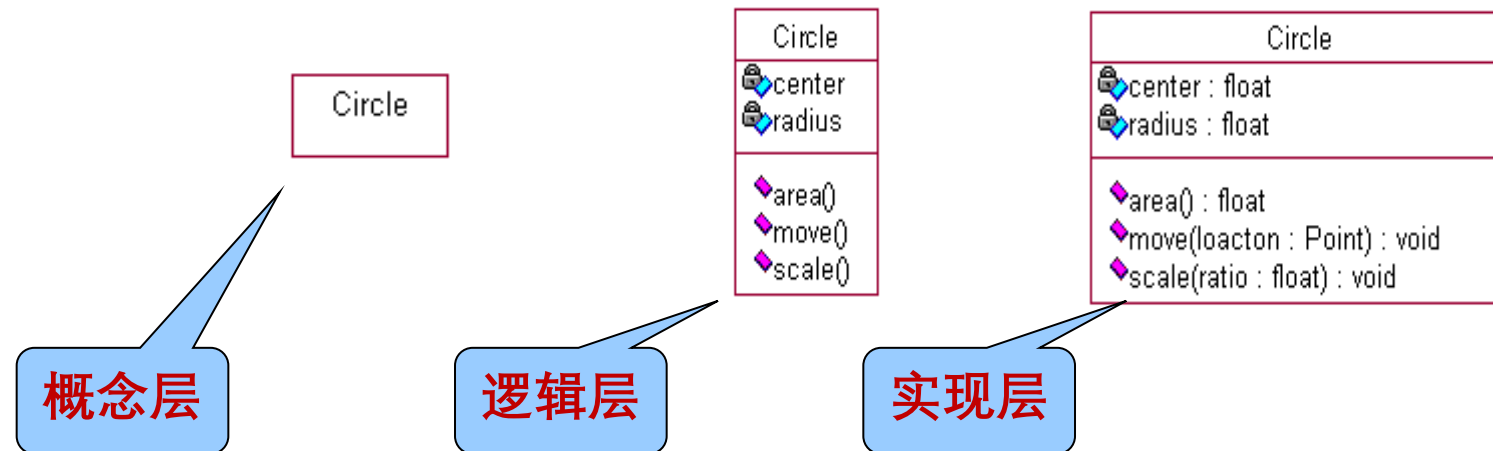
订货管理



类图的抽象层次

在系统的不同开发阶段，类图可以具有不同的抽象程度。
随着开发的深入，类图应该越来越详细、具体。

可以分为：概念层，逻辑层，实现层。



类的概念

- 类的**属性**(attribute): 描述类所表示事物的静态性质。

[可见性]属性名[:类型][‘[’多重性[次序]‘]’][=初始值][{特性}]

该属性对外部实体的显现程度.

可见public : +
受限protected: #
私有private : -

第1个英文单词首字母小写,其它单词首字母大写

contactName
credintLimit
isPrepaid

属性的数据类型:

字符串:String
日期:Date
布尔:Boolean
整型:int

类的概念

- 类的**属性**(attribute): 描述类所表示事物的静态性质。

[可见性]属性名[:类型][‘[’多重性[次序]‘]’][=初始值][{特性}]

表示属性值的取值的多寡,以及有序性:

例如: `name:String[0..1]`

表示属性"name"可能无值,也可能仅有一个值.

`points:Point[2..* ordered]`

表示有两个或多个值,有序

表示属性初始所取的值:

例如: `#visibility:Boolean=false`

表示属性"visibility"初始取"false"

类的概念

- 类的**操作**(operation): 描述类所表示事物的动态性质。

[可见性]操作名[(参数列表):返回类型][{特性}]

该操作对外部实体的显现程度.

可见public : +
受限protected: #
私有private : -

第1个英文单词首字母小写,其它单词首字母大写

close()
creditRecording()

该操作的返回值的类型.
例如: +display():Location

该操作的输入参数,可以为空.

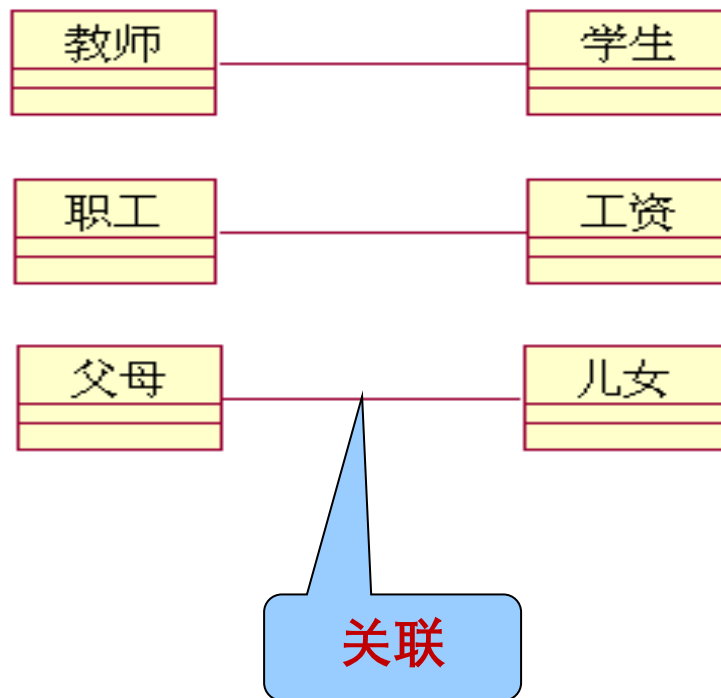
例如: #create()
+hide()
-ttachXWindow(xwin:Xwindow)

1. 关联的定义

● 关联的通俗含义

表示事物之间的一种牵连式的固有联系。

例如：



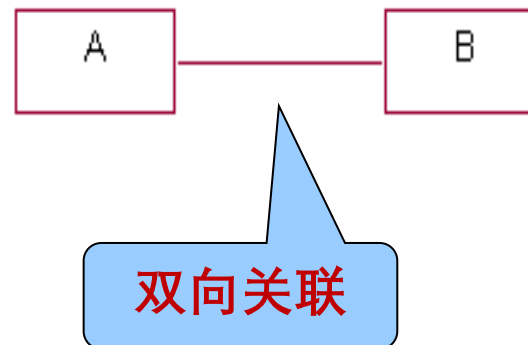
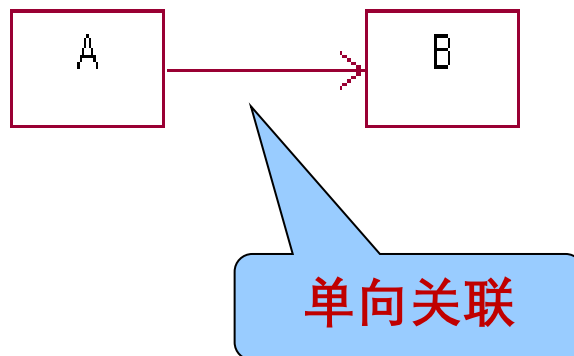
1. 关联的定义

● 关联的本质含义

关联(association): 模型元素之间的一种语义联系,它是对具有共同的结构特性、行为特性、关系和语义的链接的描述。

关联可以分为单向关联, 双向关联。

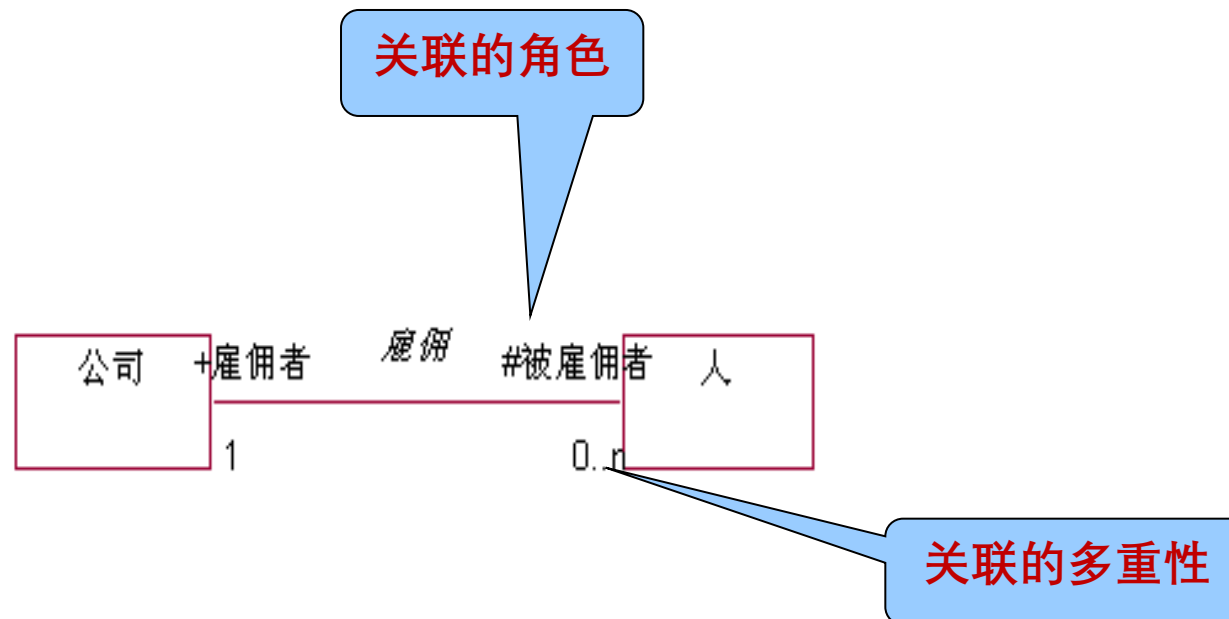
★ 如果关联关系已经清楚, 就无需关联名



2. 关联的特性

③ 关联的多重性

★如果关联名与角色名相同，则不标出角色名



0..1 : 0或1

0..n, 0..* : 0或多

1

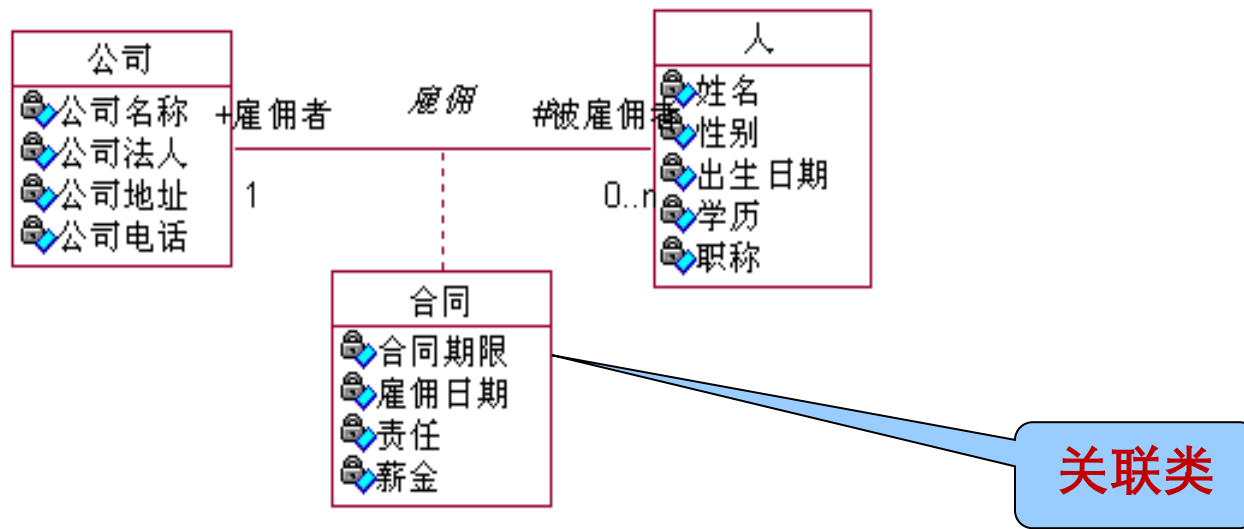
1..n : 1到多

* : 0到多

2. 关联的特性

④ 关联类

通过关联类描述关联的属性，操作，及其它信息。

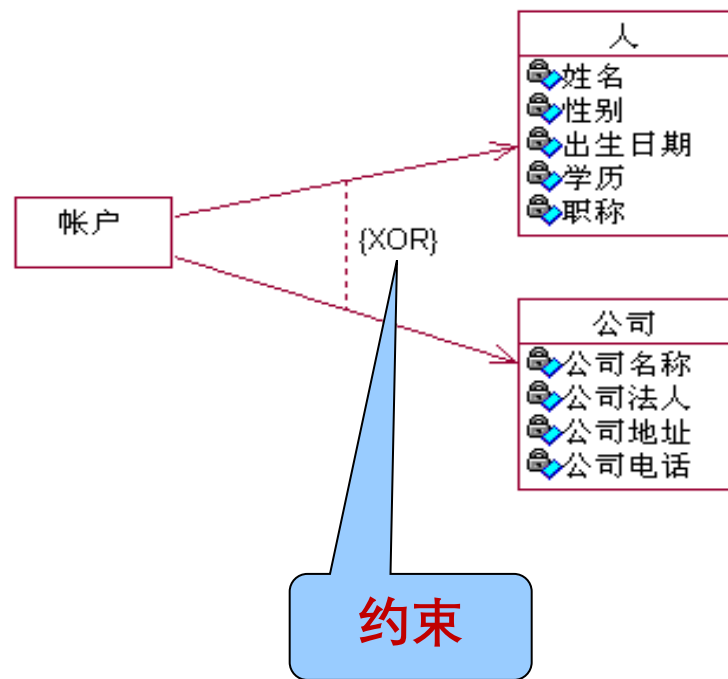


2. 关联的特性

⑤ 关联的约束

通过约束加强关联的含义。

例如，“帐户”不能同时与“人”和“公司”有关联。



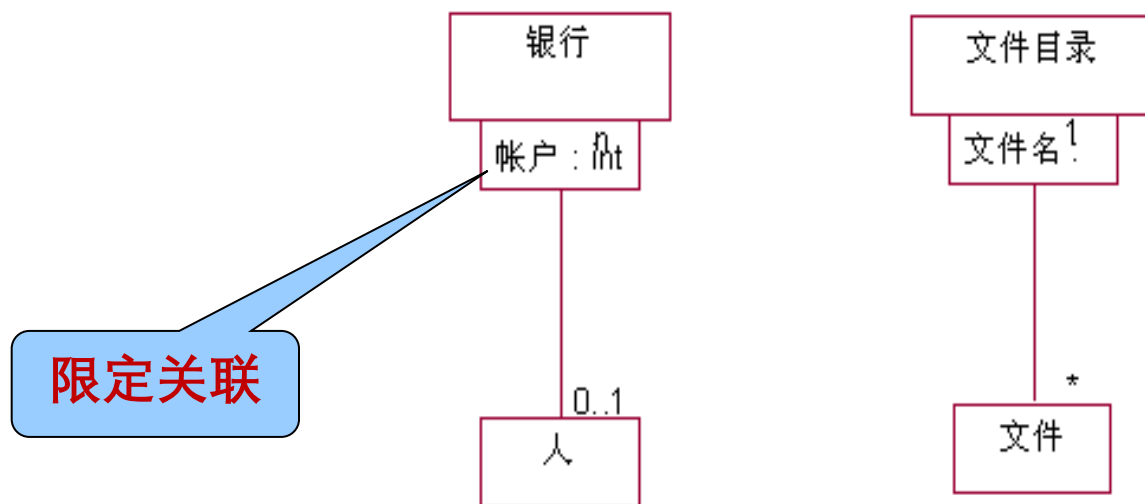
2. 关联的特性

⑥ 限定关联

通过限定符来规定关联的限定关系。

例如，“文件目录”通过“文件名”来与具体的文件对象进行关联。

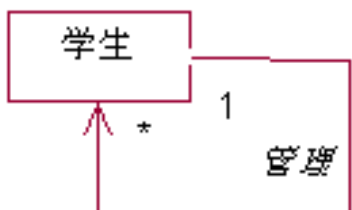
“银行”通过“帐户”与具体的“人”对象进行关联。



类的关系

3. 关联的种类

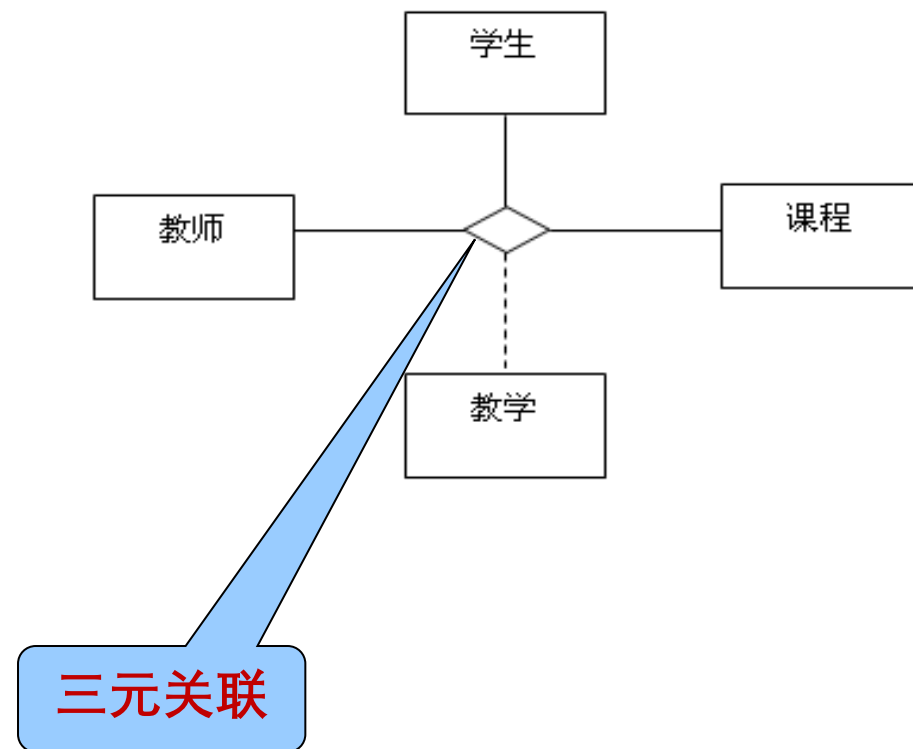
① 一元(自返)关联



② 二元关联

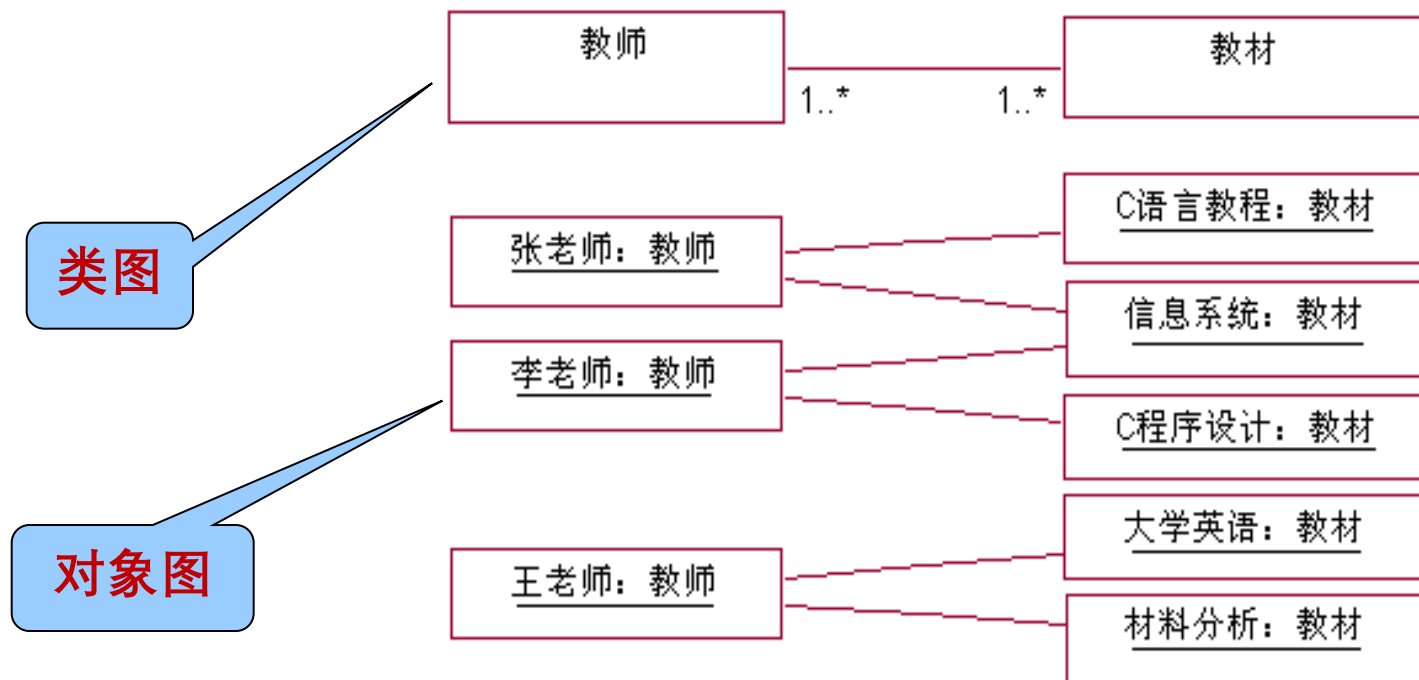


③ 多元关联



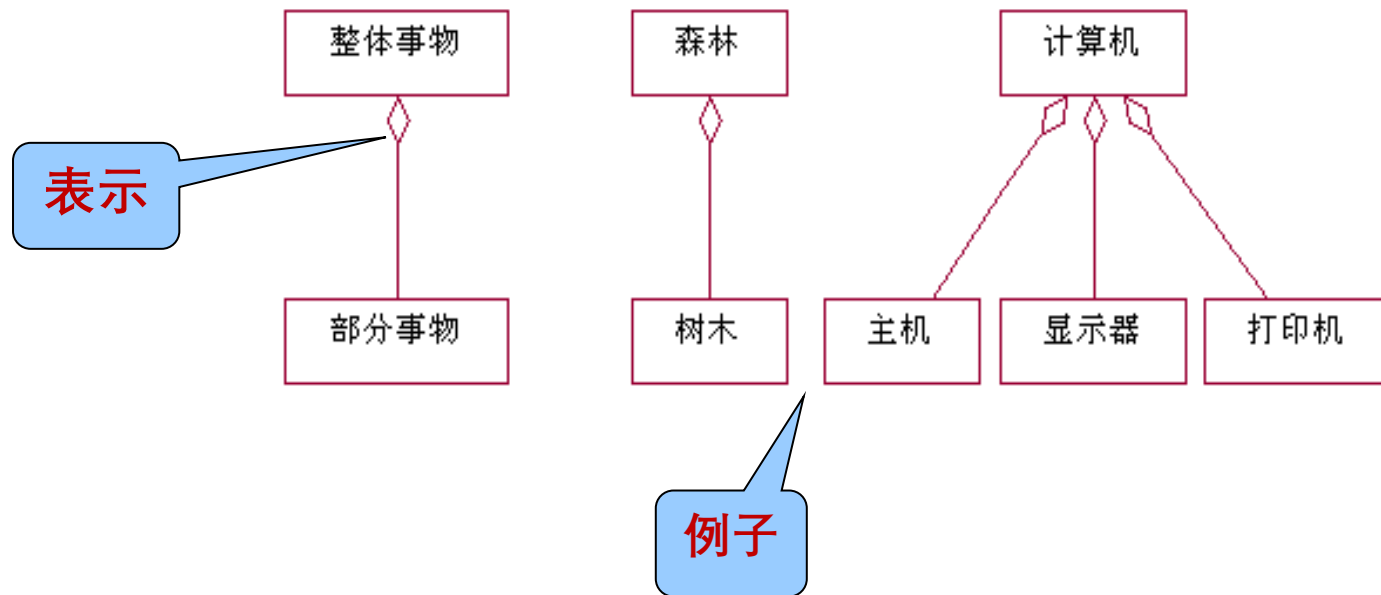
4. 对关联须注意的一个问题

关联一定指的是类中对象与类中的对象之间的连接对应关系。



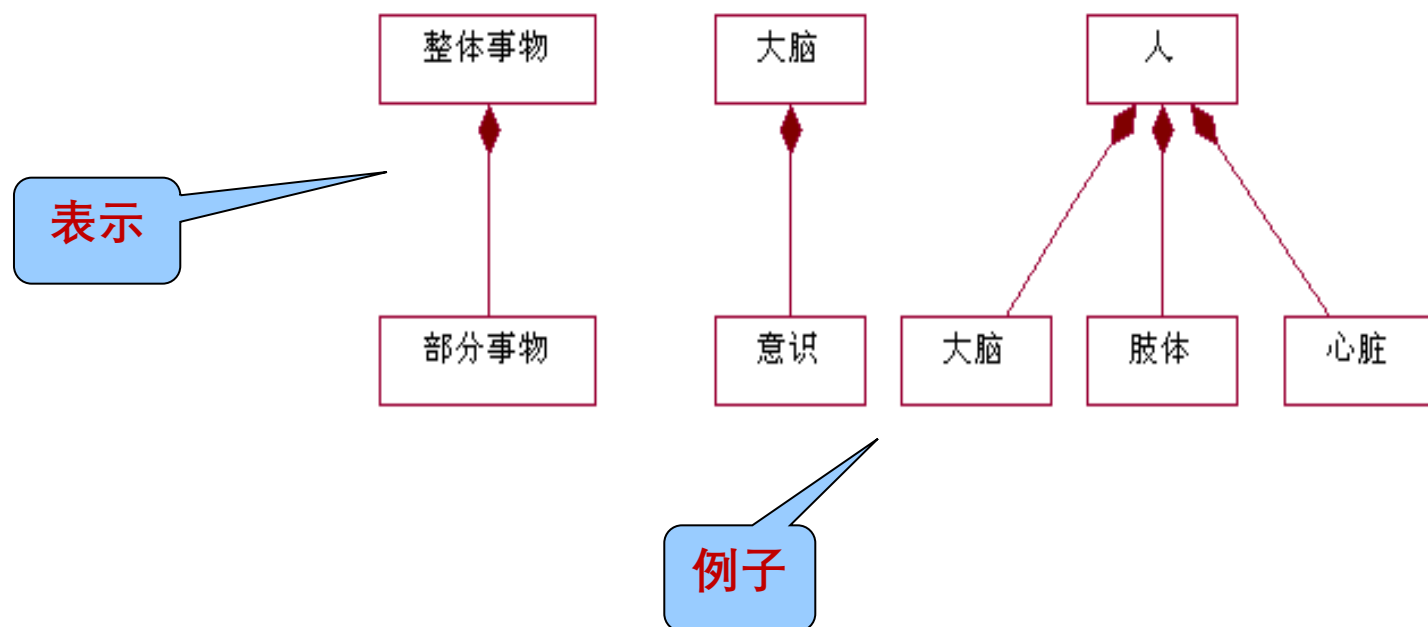
1. 聚集

聚集(aggregation): 表示类之间一种松散的整体与部分的组成关系，是一种特殊的关联。



2. 组合

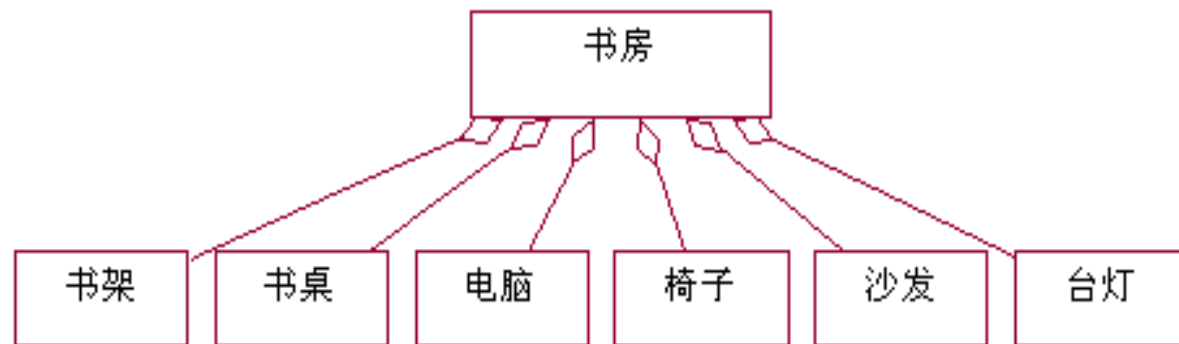
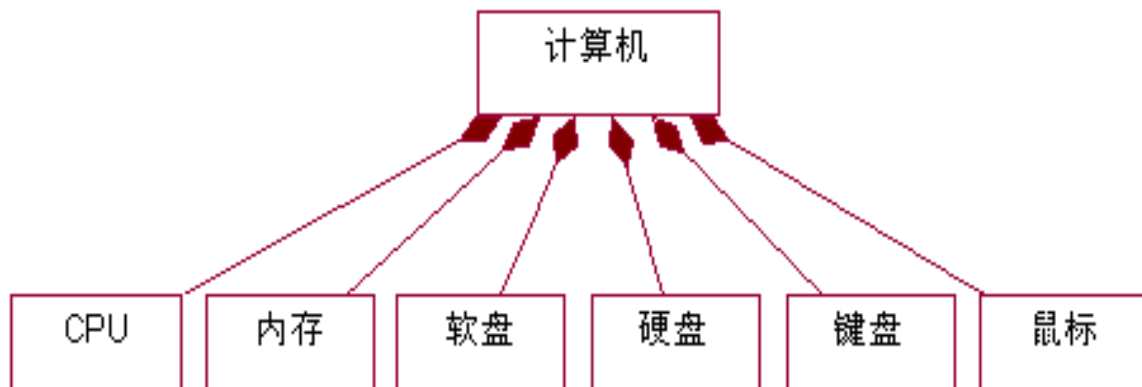
组合(composition): 表示类之间一种紧密的整体与部分的组成关系，也是一种特殊的关联。



类的关系

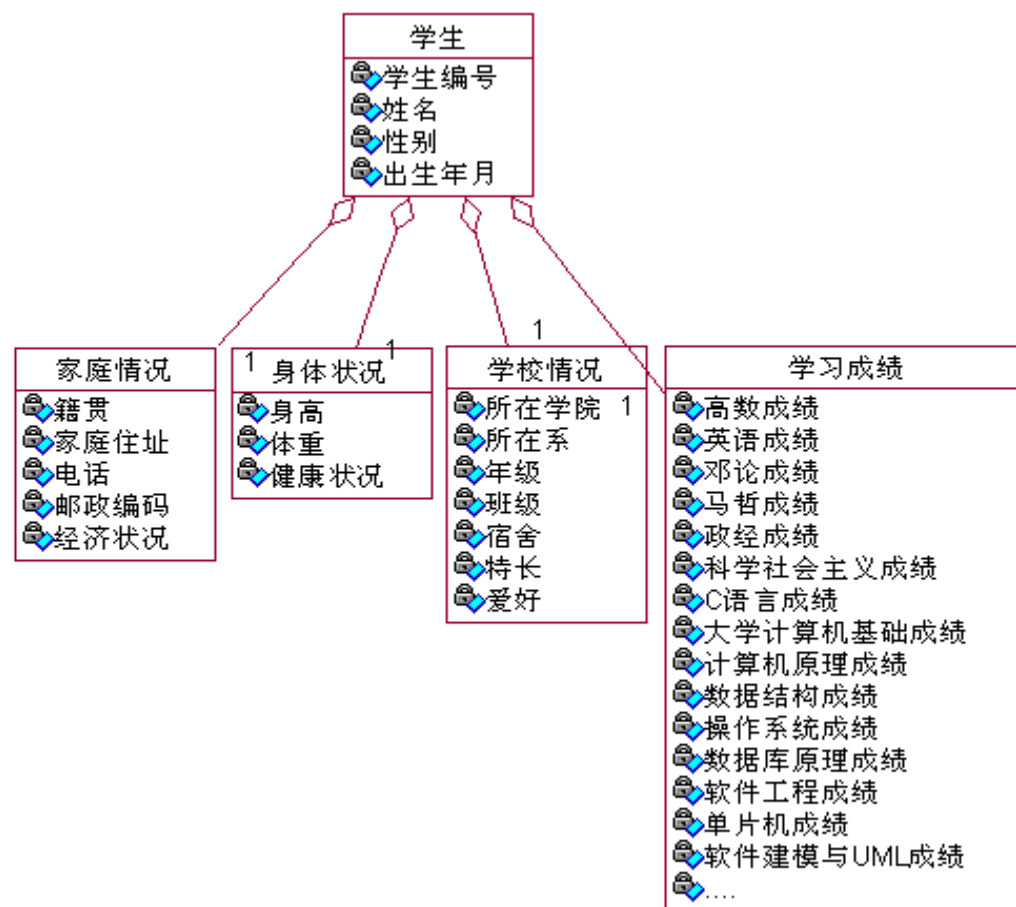
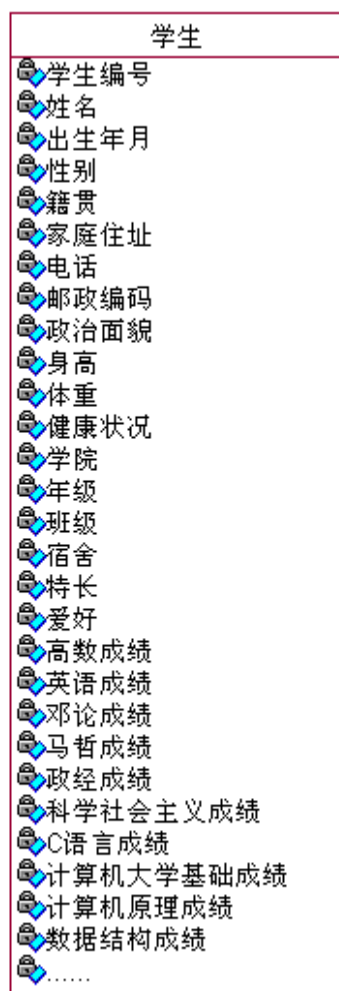
3. 聚集与组合的区别

- ① 聚集松散,组合紧密;
- ② 一个部分事物对象可以属于多个聚集对象,但一个部分事物对象仅能属于一个组合对象;
- ③ 聚集的对象生命周期可以不同,但组合对象则是同存同亡。



类的关系

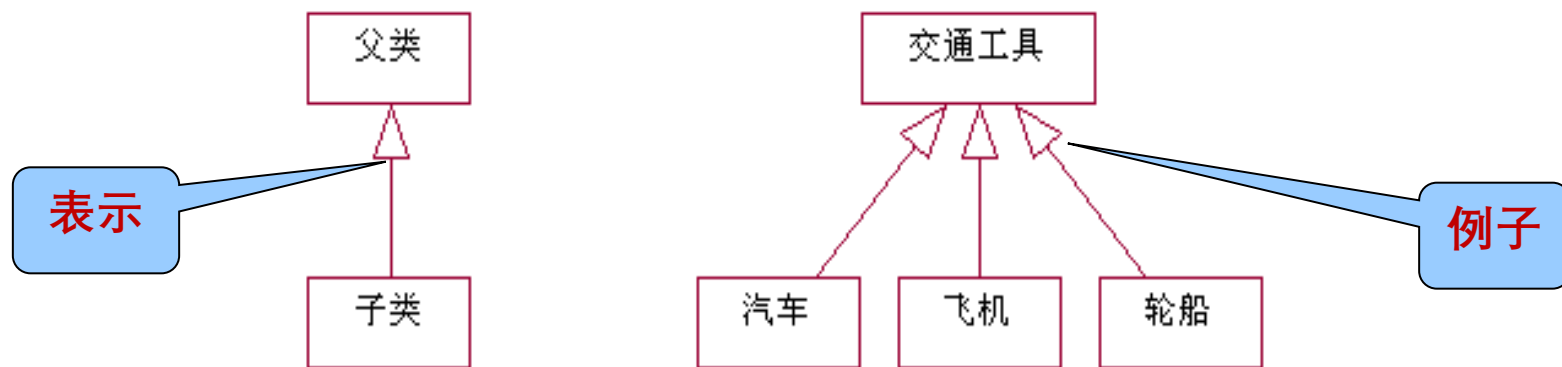
类可以优化，一个复杂类可以分解为多个简单类。



1. 泛化的含义

泛化(generalization): 表示事物之间的一般与特殊的关系。也可以称为继承关系。

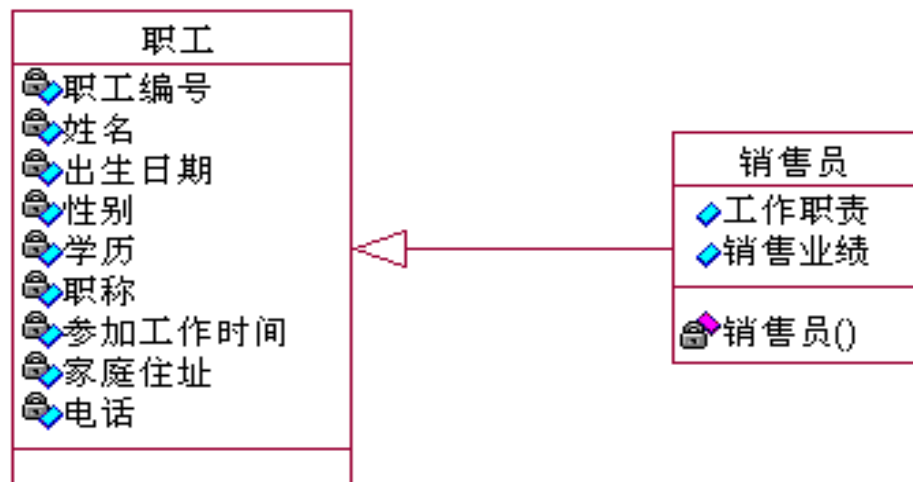
泛化关系除了表示类与类之间的关系之外，还可以表示参与者、用例、包、构件、接口等建模元素之间的关系。



四、继承

2、面向对象中的继承

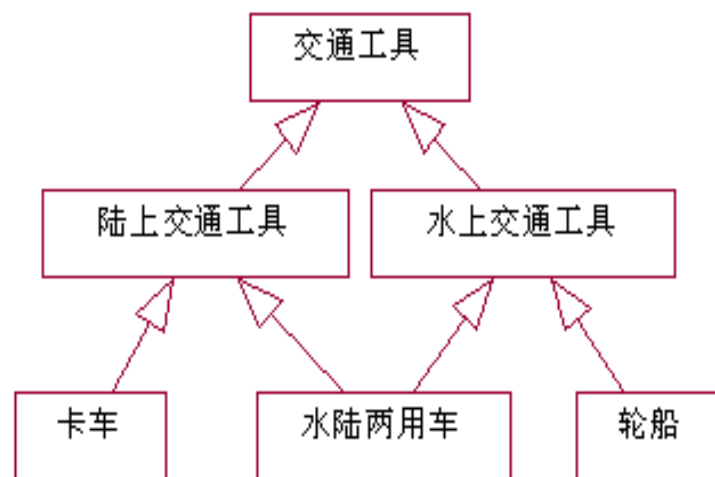
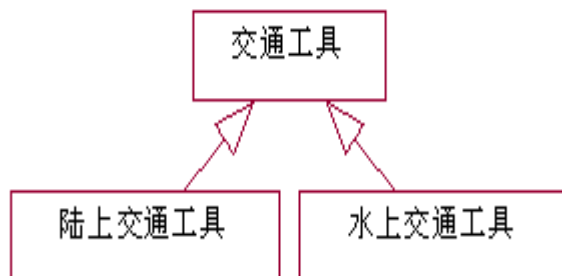
继承：子类无需定义，就可以直接继承父类的属性和操作。



3、继承的类型

单继承：子类只能继承一个父类；

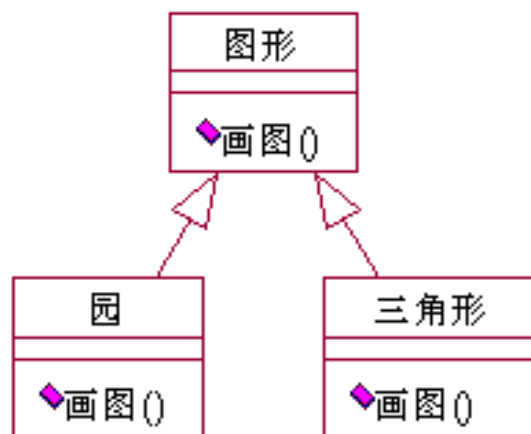
多继承：子类可以继承多个父类。



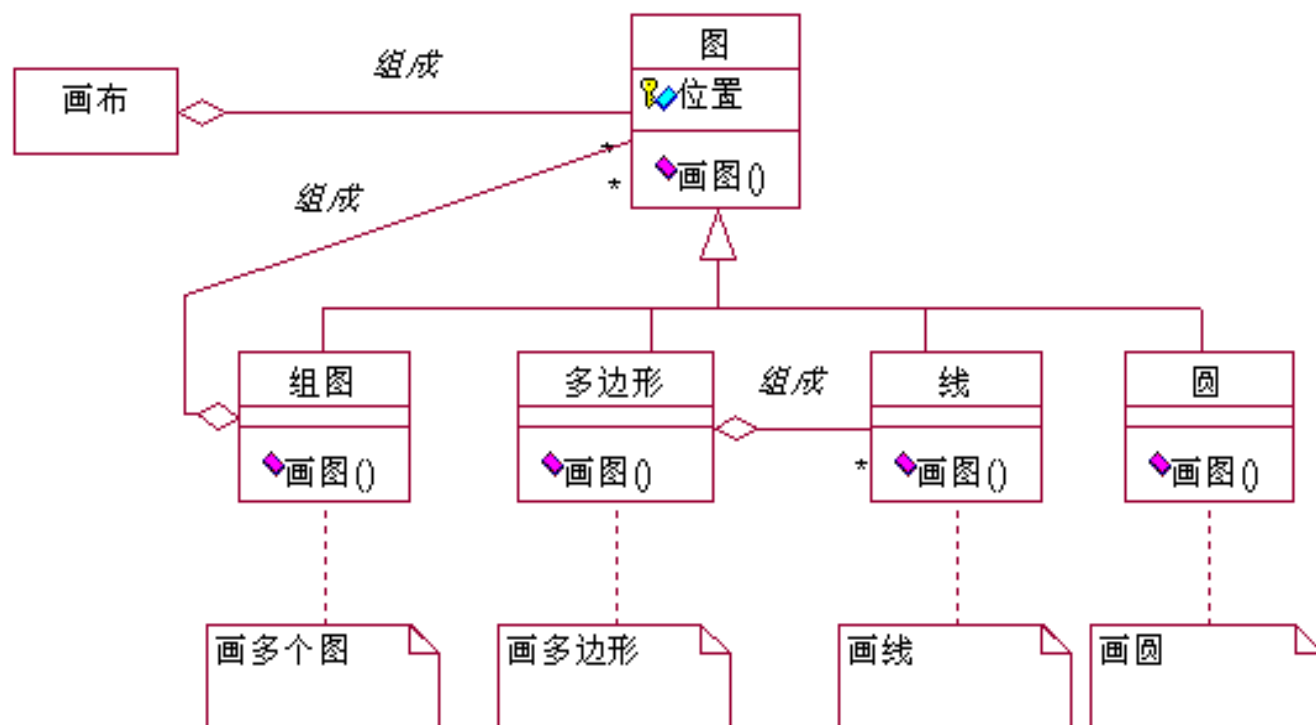
五、多态

1、多态的概念

多态描述子类在继承父类的属性和操作时，根据子类的特点，赋予子类的特殊性。



类的关系

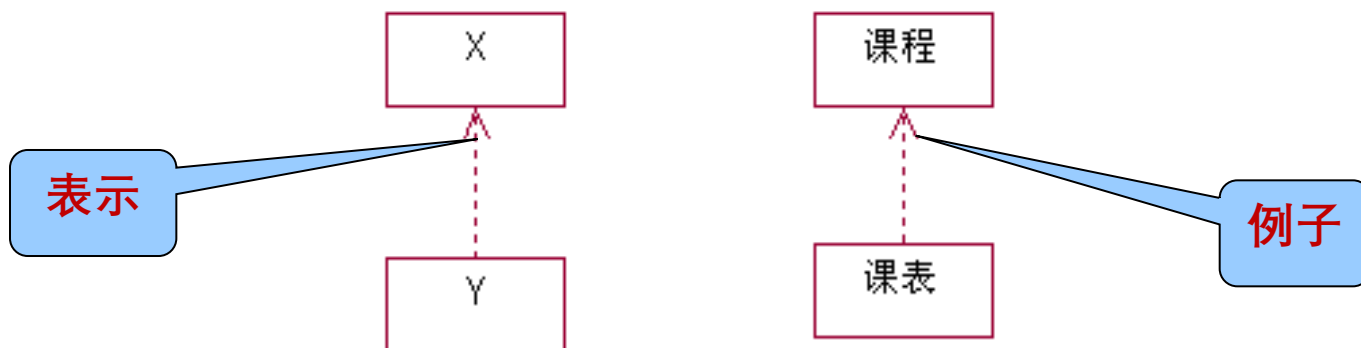


泛化与多态

1. 依赖的含义

依赖(dependency): 表示两个元素X、Y，如果X的变化必然导致Y的变化，则称Y依赖X。

依赖关系不仅限于类，用例、包、构件之间都可以存在依赖关系。

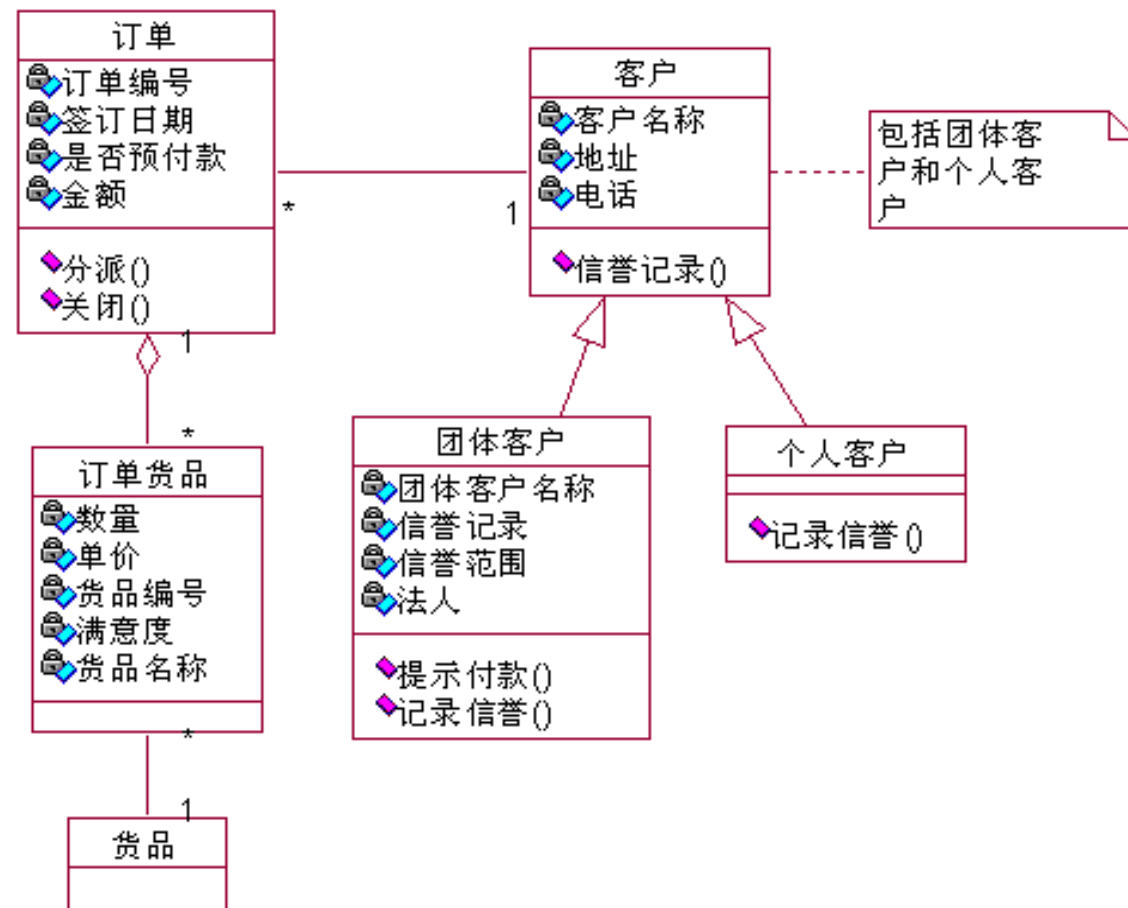


类图

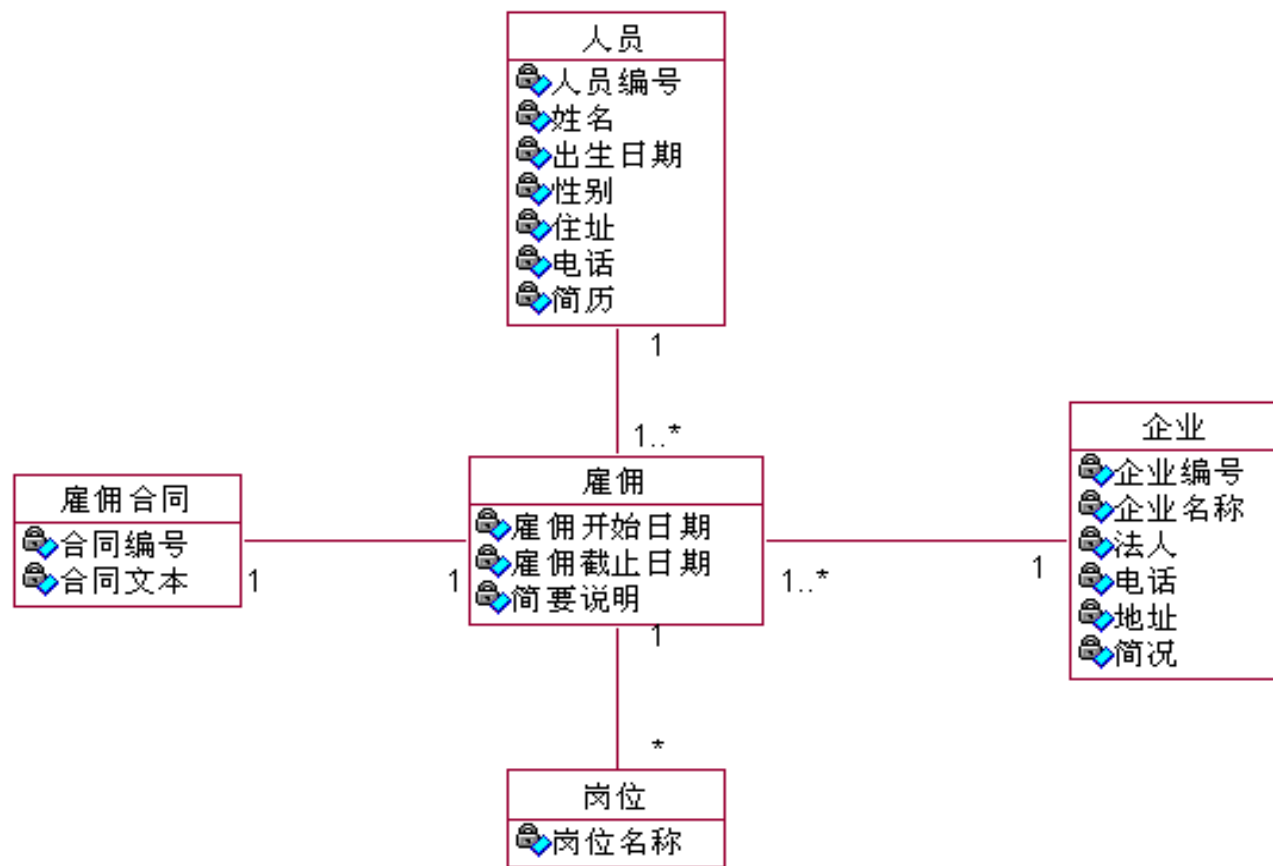
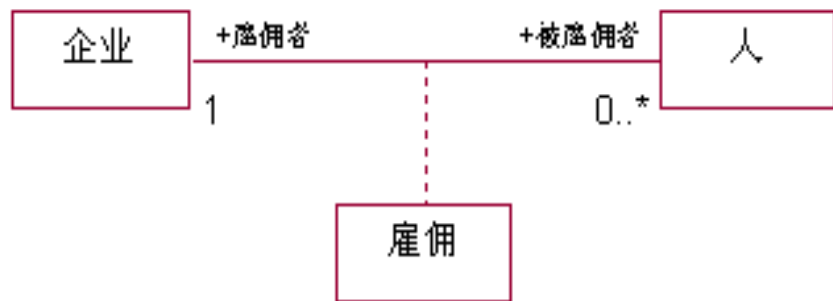
类图(Class Diagram): 是由类, 相关建模元素, 及其关系构成的图, 用来描述类之间的静态关系。

类图在系统中处在核心位置。也是UML中最为重要的一种图。

订货系统的类图

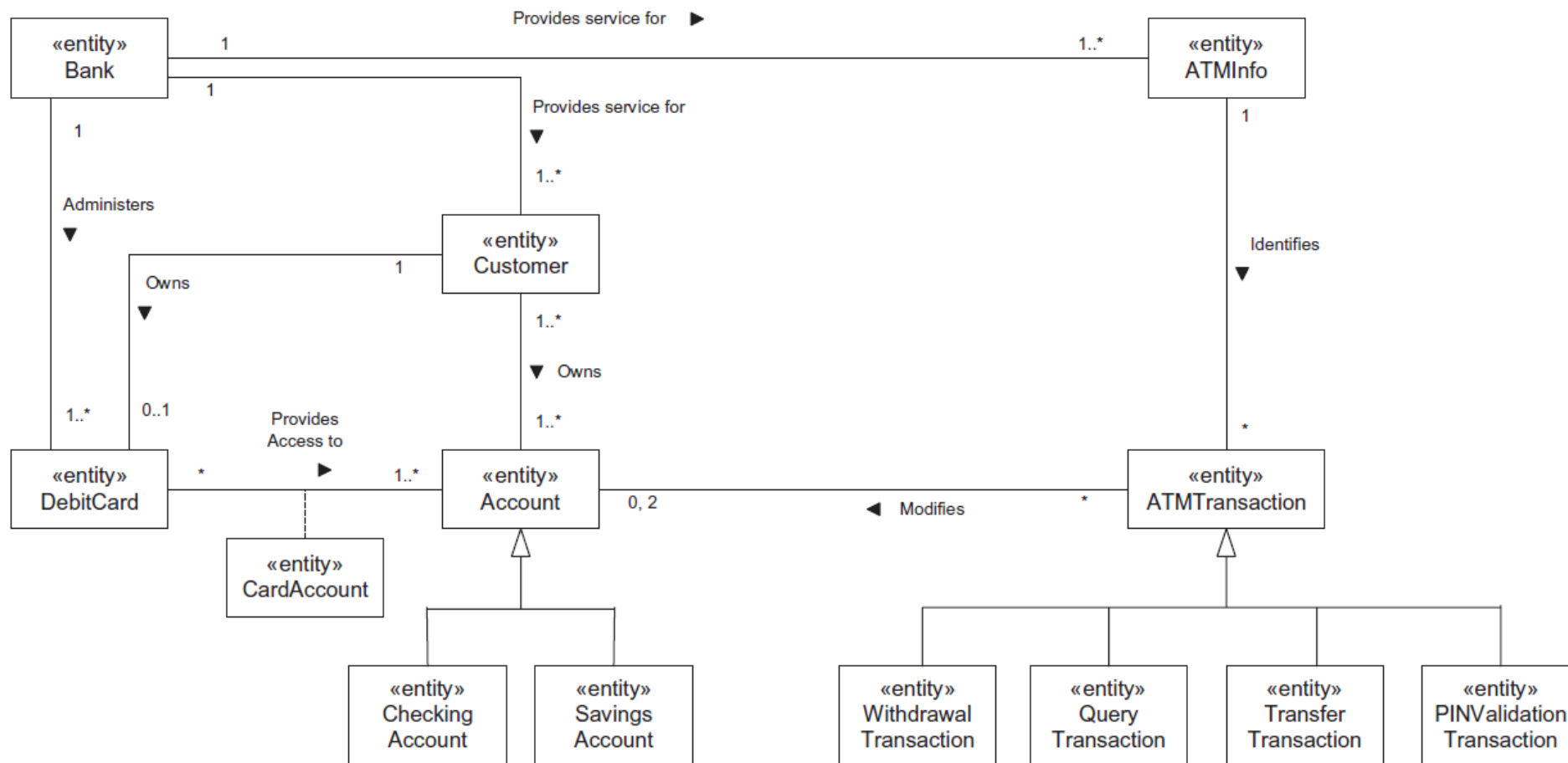


描绘类图时，应根据所要表现的应用需要来提取要素，绘制类图。



雇佣关系类图

案例-ATM实体类



案例-ATM实体类精化

«entity» Bank
bankName: String bankAddress: String bankId: Real

«entity» Customer
customerName: String customerId: String customerAddress: String

«entity» DebitCard
cardId: String PIN: String startDate: Date expirationDate: Date status: Integer limit: Real total: Real

«entity» Account
accountNumber: String accountType: String balance: Real

«entity» CheckingAccount
lastDepositAmount: Real

«entity» SavingsAccount
interest: Real

案例-ATM实体类精化

«entity» ATMTransaction
bankId: String ATMId: String date: Date time: Time transactionType: String cardId: String PIN: String status: Integer

«entity» WithdrawalTransaction
accountNumber: String amount: Real balance: Real

«entity» QueryTransaction
accountNumber: String balance: Real lastDepositAmount: Real

«entity» TransferTransaction
fromAccountNumber: String toAccountNumber: String amount: Real

«entity» PINValidationTransaction
startDate: Date expirationDate: Date

案例-ATM实体类精化

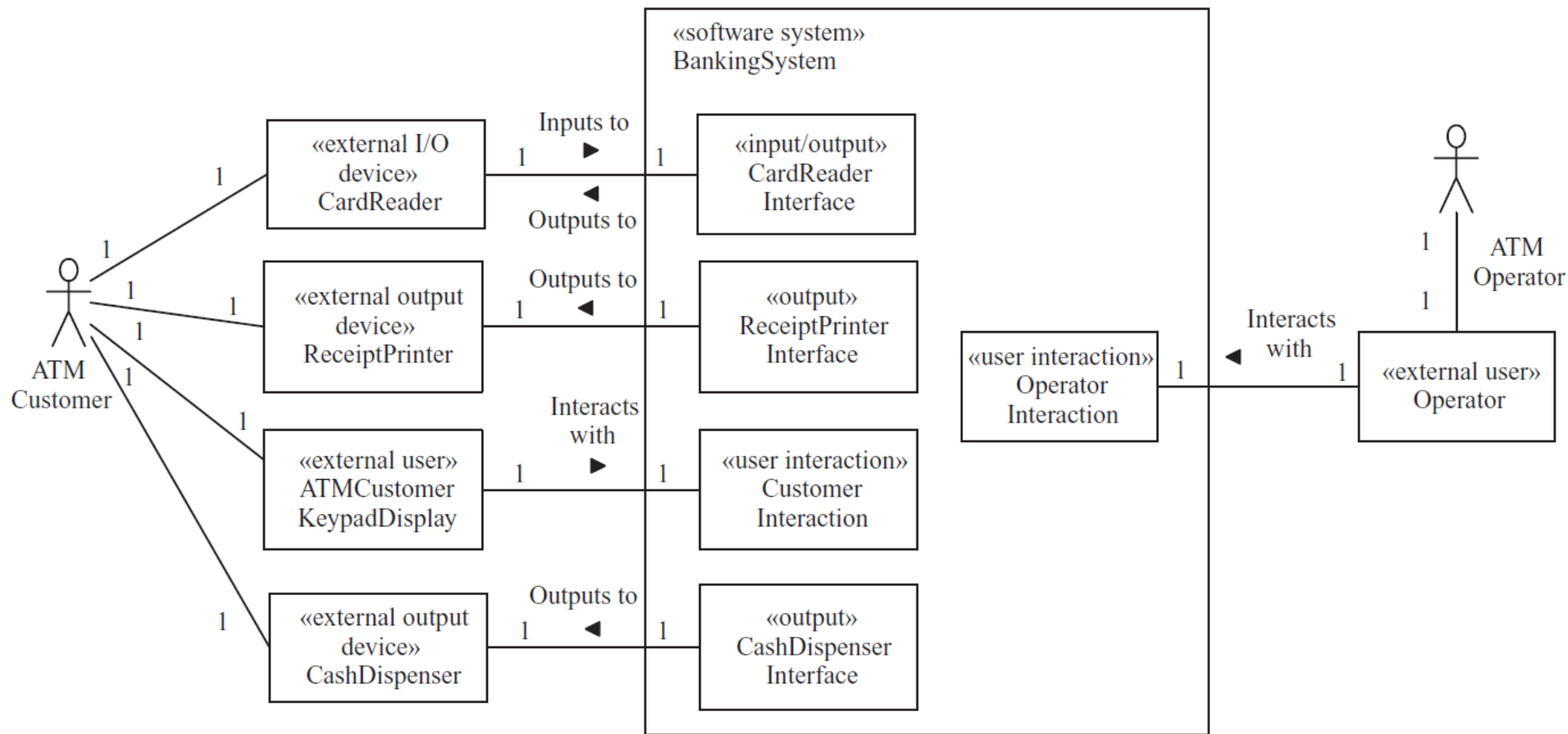
«entity» CardAccount
cardId: String accountNumber: String accountType: String

«entity» ATMInfo
bankId: String ATMId: String ATMLocation: String ATMAddress: String

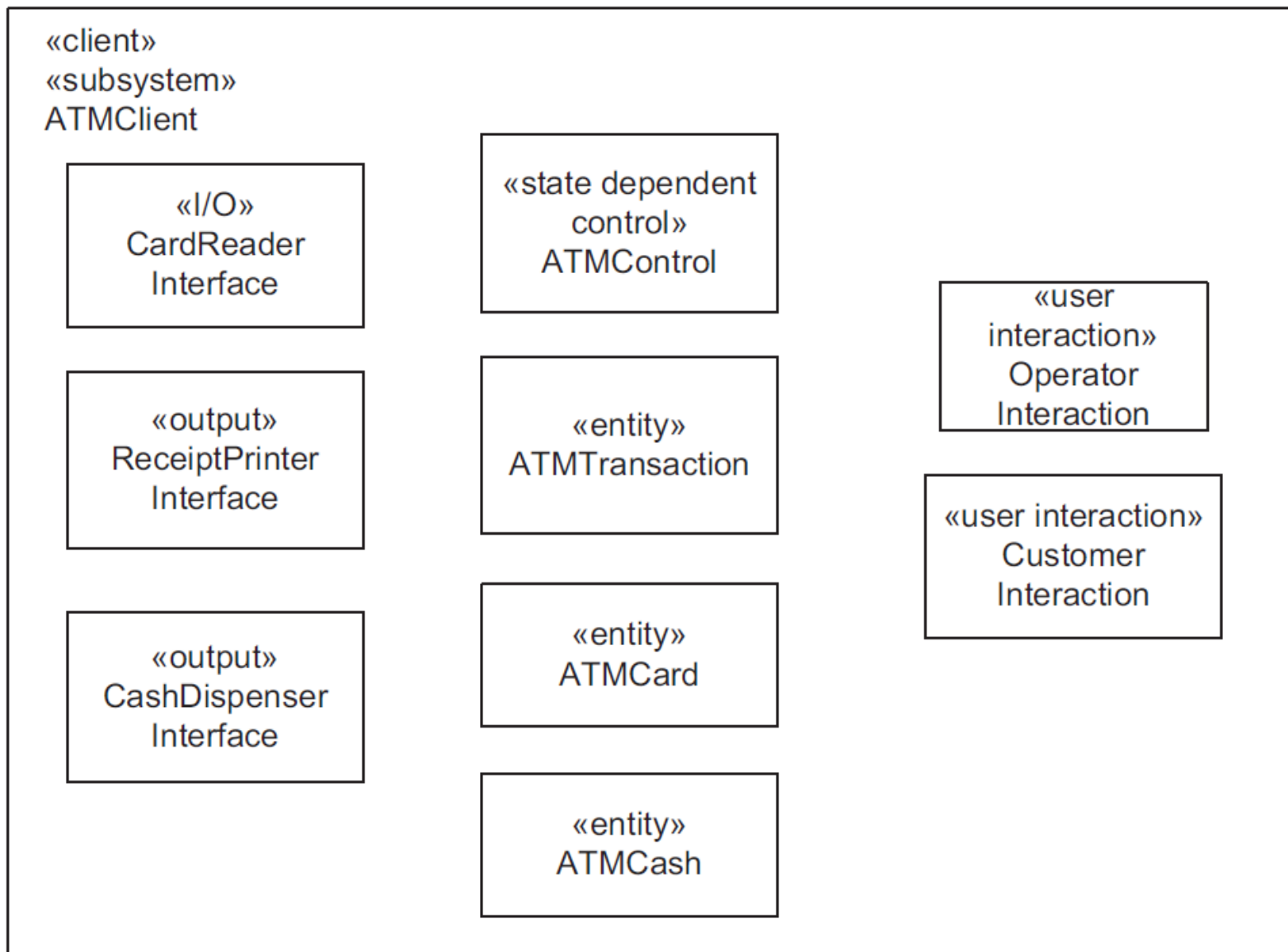
«entity» ATMCash
cashAvailable: Integer fives: Integer tens: Integer twenties: Integer

«entity» ATMCard
cardId: String startDate: Date expirationDate: Date

案例-ATM类的组织:边界



案例-ATM客户端类



案例-ATM客户端信息隐藏类

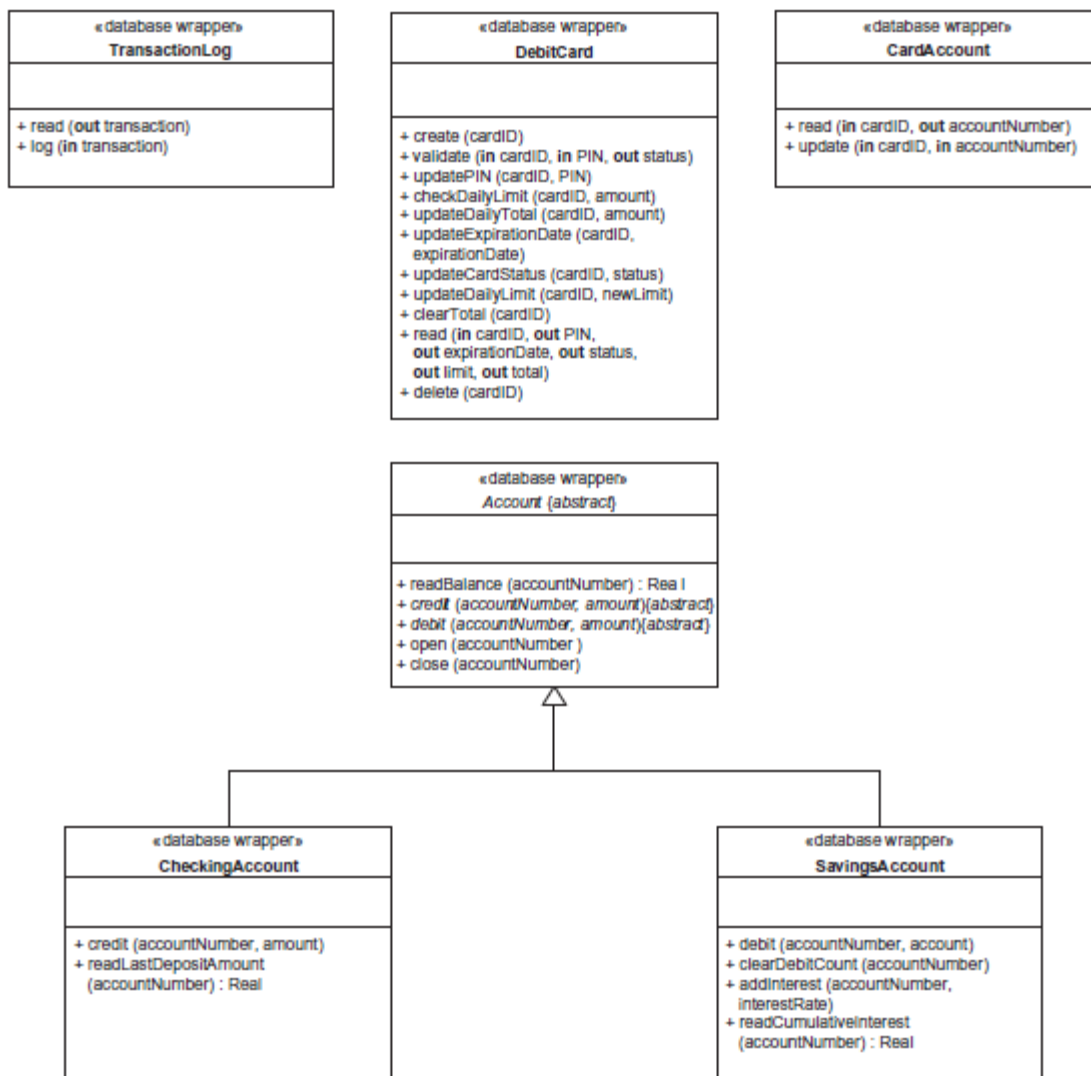
«data abstraction» ATMCard
- cardNumber: String - startDate: Date - expirationDate: Date
+ write (in cardId, in startDate, in expirationDate) + read (out cardId, out startDate, out expirationDate)

«data abstraction» ATMCash
- cashAvailable: Integer = 0 - fives: Integer = 0 - tens: Integer = 0 - twenties: Integer = 0
+ addCash (in fivesAdded, in tensAdded, in twentiesAdded) + withdrawCash (in cashAmount, out fivesToDispense, out tensToDispense, out twentiesToDispense)

«data abstraction» ATMTransaction
- transactionId: String - cardId: String - PIN: String - date: Date - time: Time - amount: Real - balance: Real - PINCount: Integer - status: Integer
+ createTransaction () + updateCustomerInfo (cardData, PIN) + updateCustomerSelection (in selection, out transactionData) + updatePINStatus (inout status) + updateTransactionStatus (amount, balance) + read (out transactionData)

«state machine» ATMStateMachine
+ processEvent (in event, out action) + currentState () : state

案例-ATM银行服务信息隐藏类



案例-ATM银行服务接口

«business logic» QueryTransactionManager
+ initialize () + query (in accountNumber, out q_response)

«business logic» PinValidationTransactionManager
+ initialize () + validatePIN (in cardID, in PIN, out v_response)

«business logic» TransferTransactionManager
+ initialize () + transfer (in fromAccountNumber, in toAccountNumber, in amount, out t_response)

«business logic» WithdrawalTransactionManager
+ initialize () + withdraw (in accountNumber, in amount, out w_response) + confirm (accountNumber, amount) + abort (accountNumber, amount)

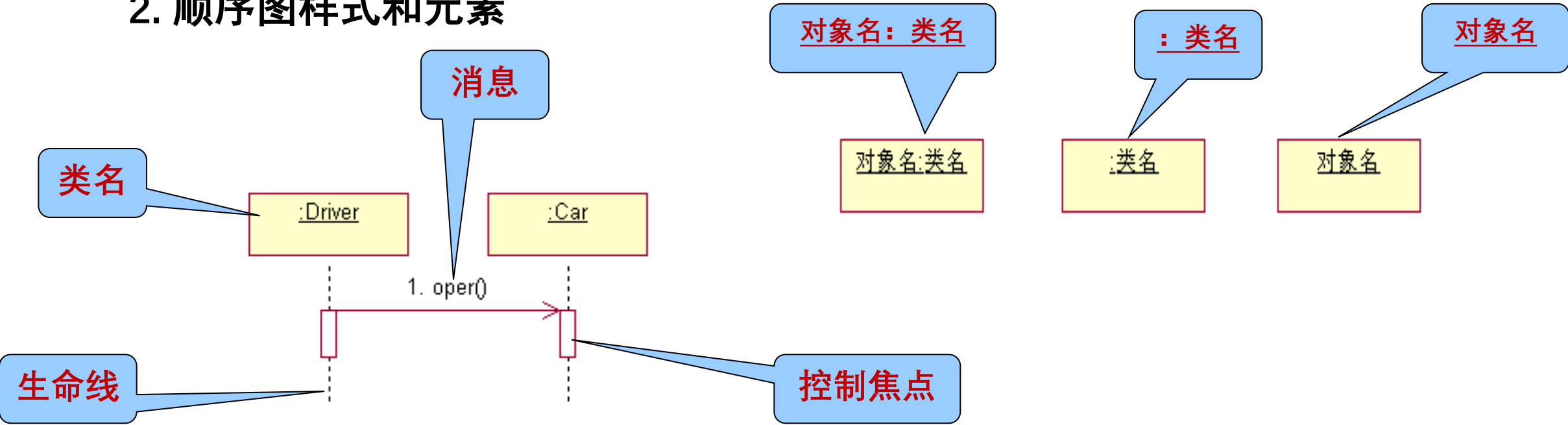
UML 顺序图 (交互模型)

顺序图

1. 顺序图的概念

顺序图(sequence diagram): 用来描述为了完成确定事务，对象之间按照时间消息交互的顺序关系。

2. 顺序图样式和元素

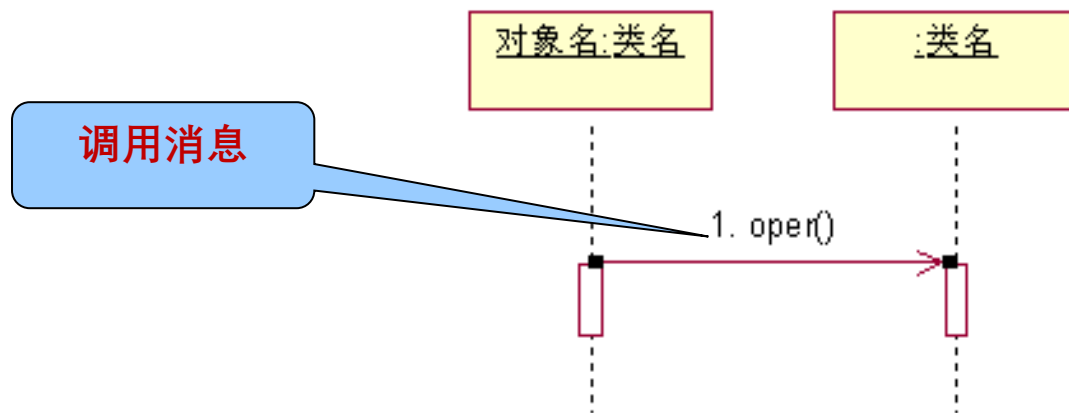


顺序图

消息：是一个对象向另外一个对象发出的服务请求。消息一般包括提供服务的对象标识，操作名，输入信息，输出信息等。消息有以下类型：

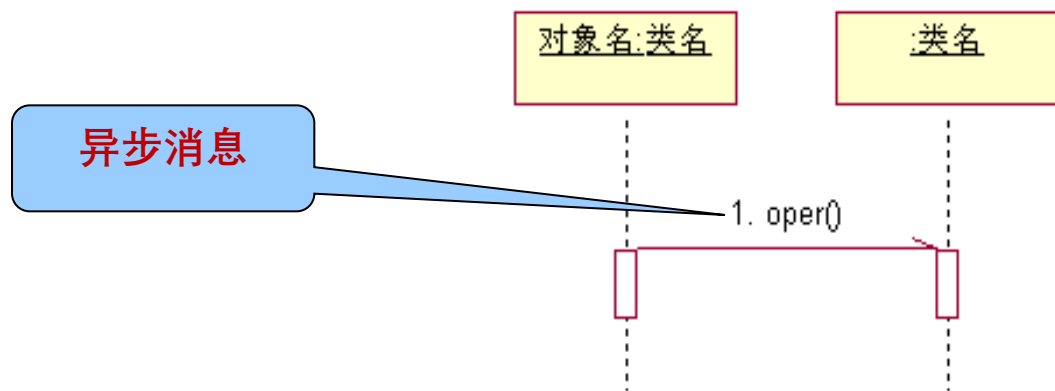
① 调用消息

调用消息(也称为同步消息)的发送者把控制传递给接收者，然后停止活动，等到消息接收者放弃或返回控制。



②异步消息

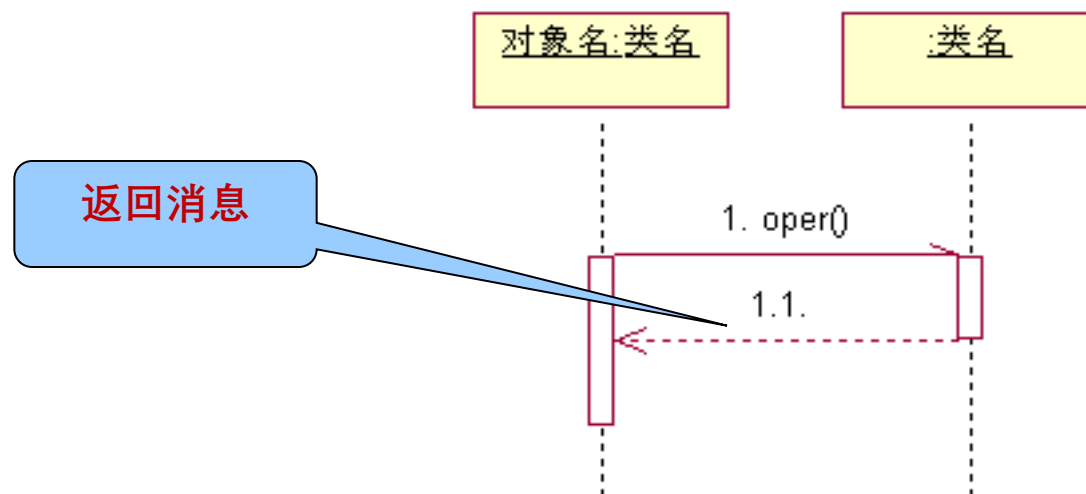
异步消息的发送者通过消息把信号传递给接收者，然后继续自己活动，不等待接收者返回消息或控制。



③返回消息

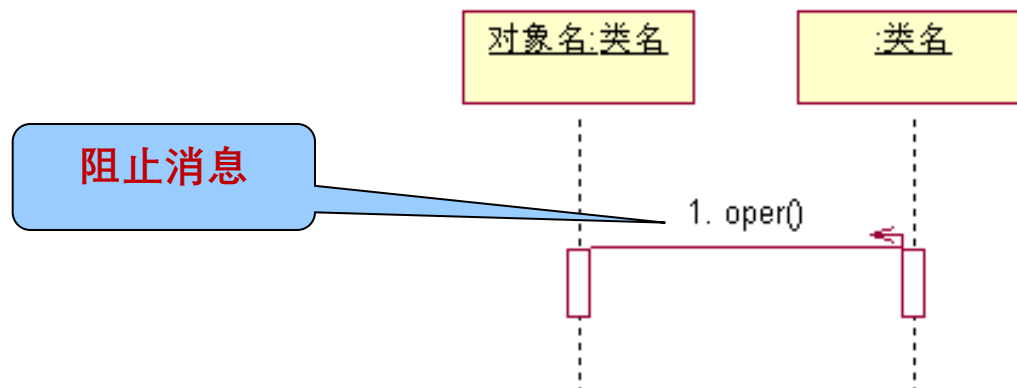
返回消息表示从过程调用返回。如果是从过程调用返回，则返回消息是隐含的，可以不画出来。

对于非过程调用，如果有返回消息，必须画出来。



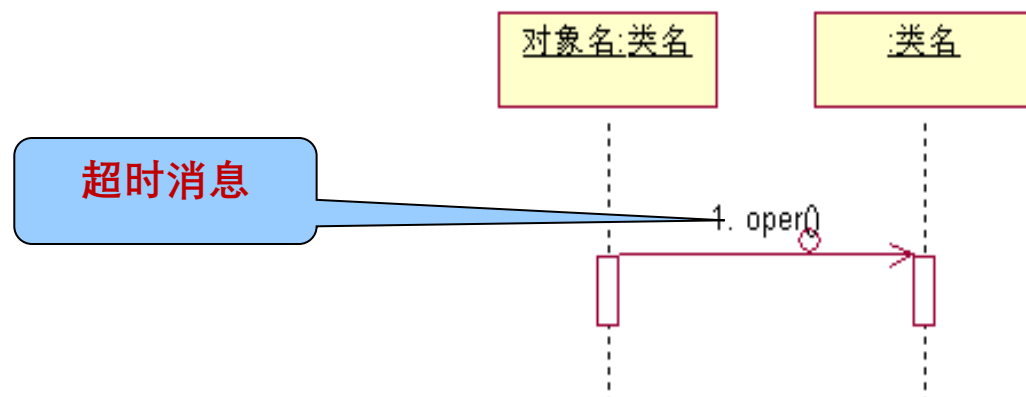
④阻止消息

阻止消息指消息发送者发出消息给接收者，如果接收者无法立即接收这个消息，则发送者放弃这个消息。



⑤超时消息

超时消息指消息发送者发出消息给接收者，并按指定时间等待，如果接收者无法在指定时间内接收这个消息，则发送者放弃这个消息。



消息类型

- 同步消息
- 异步消息
- 返回消息
- 阻止消息
- 超时消息

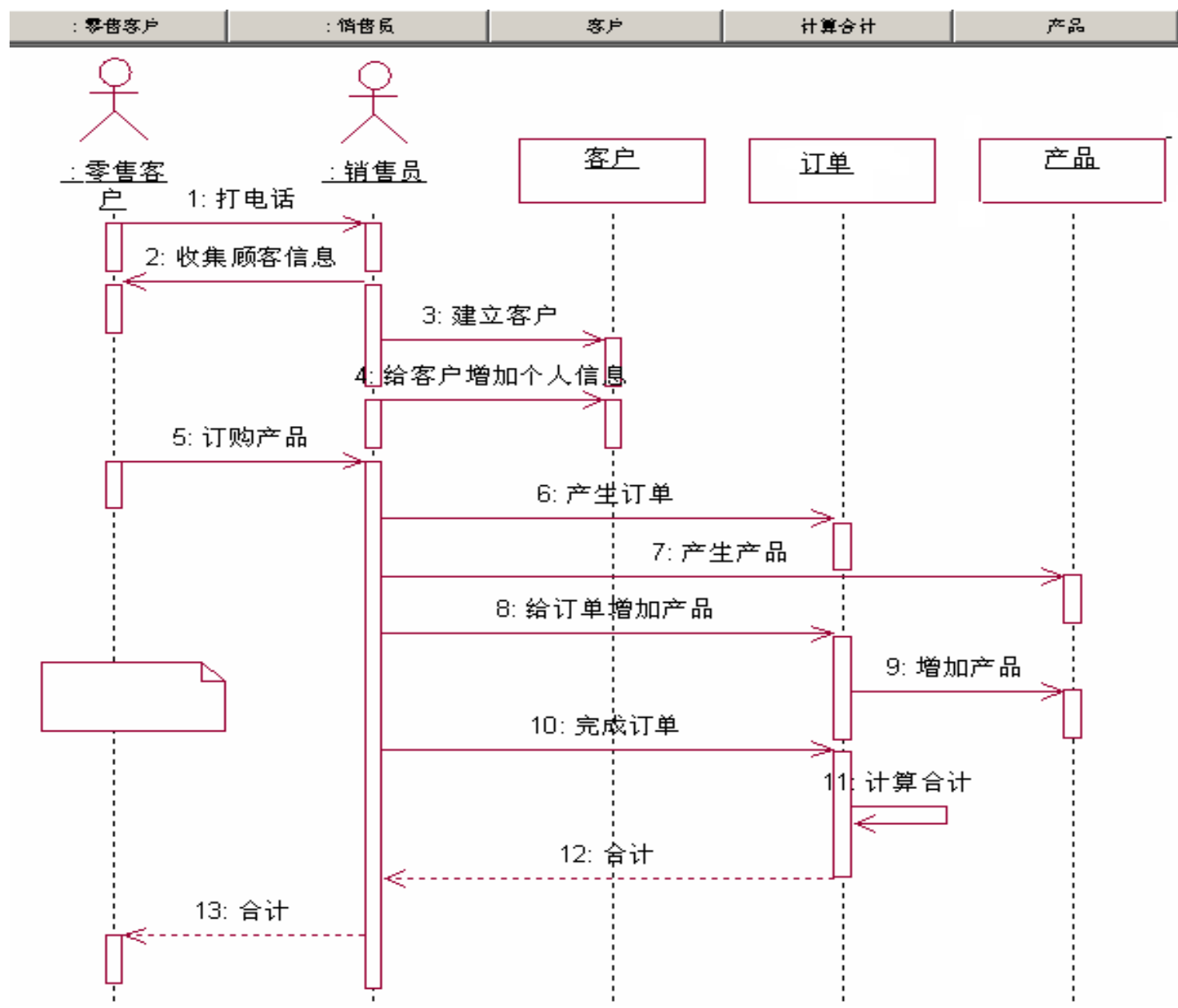
⑥消息的语法格式

[警戒条件][消息序号][返回值: =]消息名 ([参数列表])

例如:

2: display(x,y)	简单消息
1.2.1: p:=find(specs)	嵌套消息, 消息带返回值
[x<0]4: invert(x,color)	条件消息
3.1 * update()	循环消息

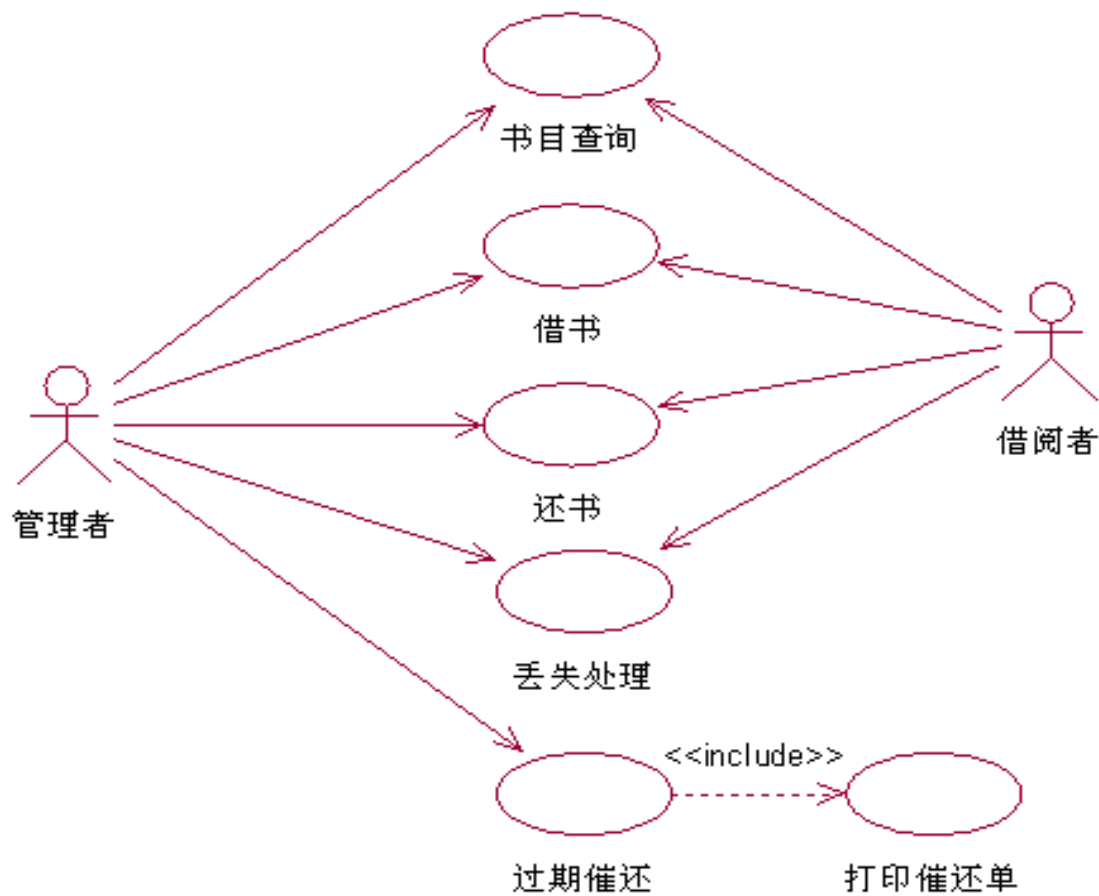
零售业务顺序图



建立顺序图

- ① 从用例和业务流程中识别交互过程;
- ② 通过类图识别参与交互过程的类;
- ③ 为每一个类设置生命线,并确定类的存在期限;
- ④ 从引发交互的初始消息开始,在类生命线上依次画出交互的消息;
- ⑤ 如果需要,可以给消息增加时间约束,以及前置条件和后置条件。

实例：图书馆借书处理的顺序图



借阅管理用例图

- 用例：借书

- 参与者：管理员, 借阅者

- 操作流：

- ① 管理员进入图书借阅界面，用例开始。

- ② 系统要求输入借阅者的借书证编码。

- ③ 系统检验借书证编码,如果正确,则显示借阅者的信息。

A1: 借书证编码有错。

A2: 如果该借阅者所借图书已经超期,则提示,本次拒借.

- ④ 系统要求输入所借图书的条码。

- ⑤ 系统显示所借图书的信息。

- ⑥ 确认借书。

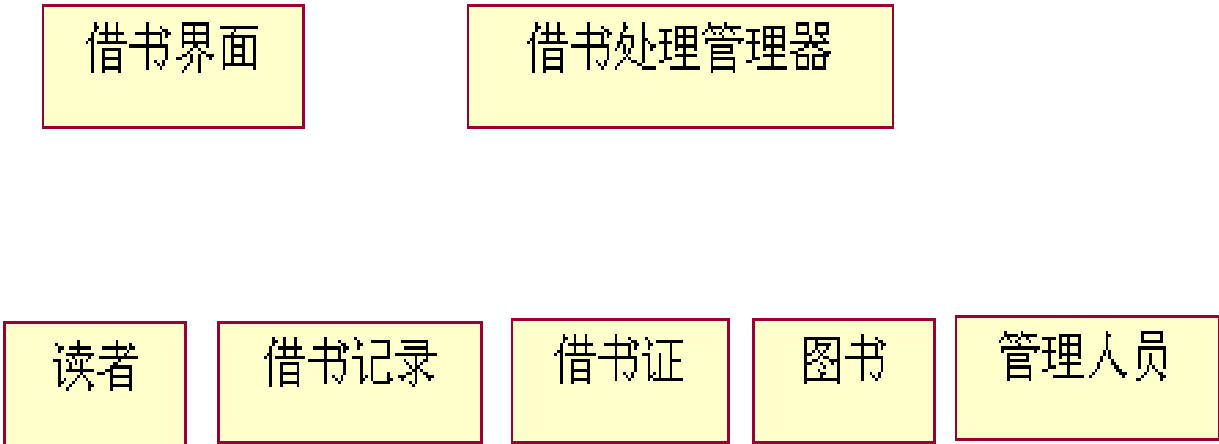
- ⑦ 系统回到上一界面，等待处理下一借书。

① 识别交互过程。

读者在借书时，先由管理员把借书证上的读者编号扫描给系统，系统检查这个读者的借书证合法信息,如果不合法,则系统给出提示,如果正确系统返回读者的身份信息，以及读者的借阅信息。

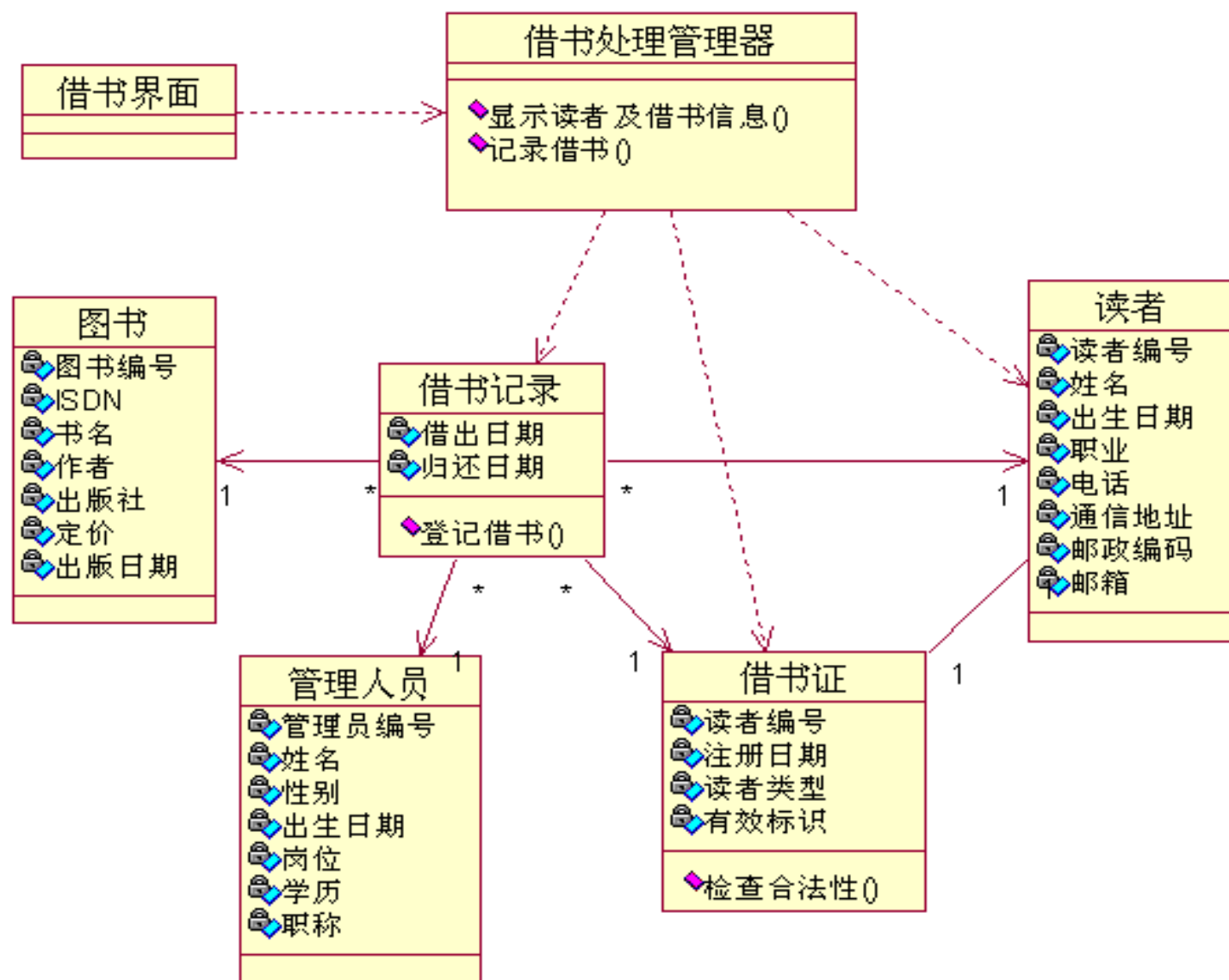
如果读者借书数量没有超过借书的上限，则把要借书的图书编号输入系统，系统登记借书信息，并返回借书成功信息，借书过程完成。

② 识别参与交互过程的对象；



实例：图书馆借书处理类图

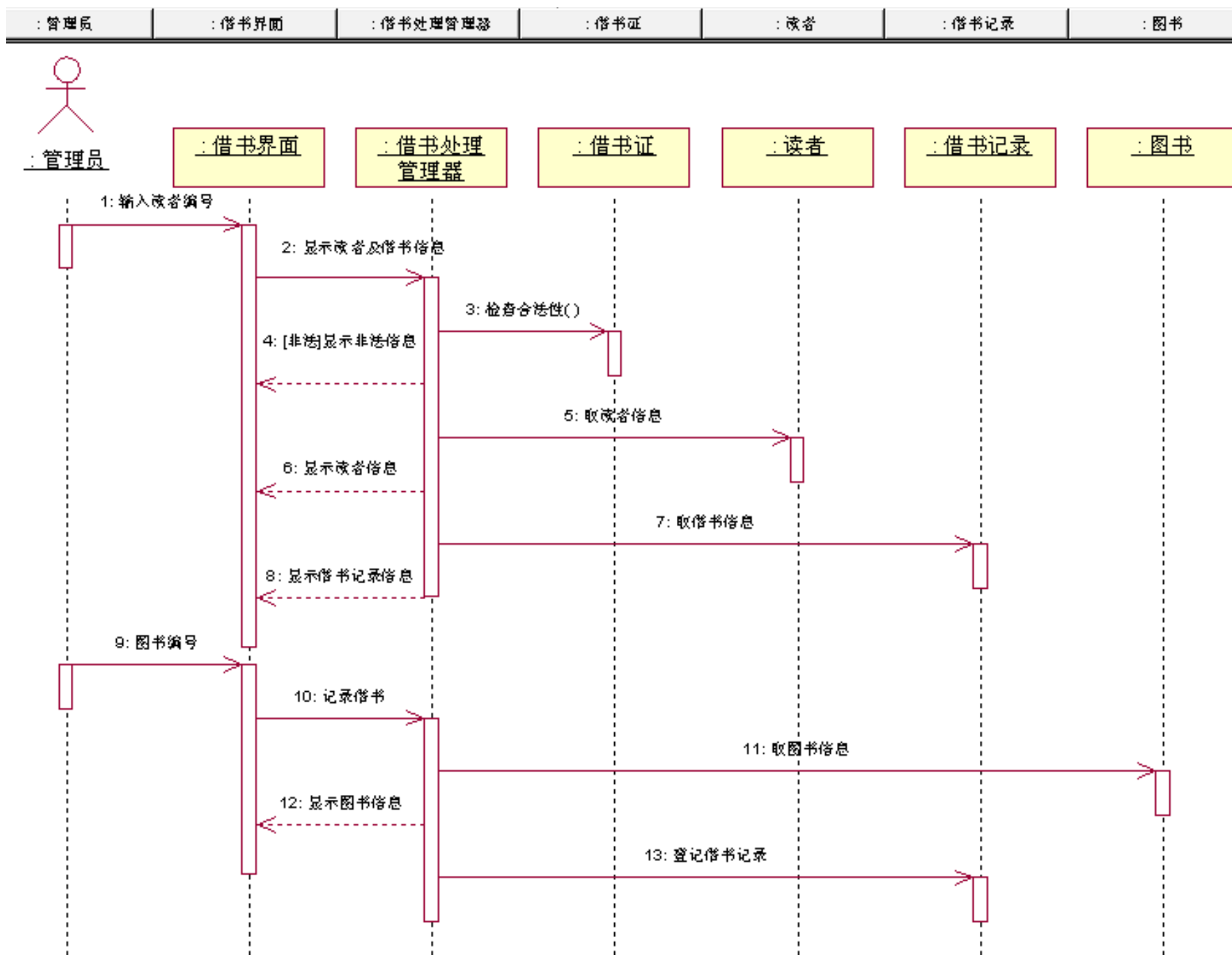
顺序图



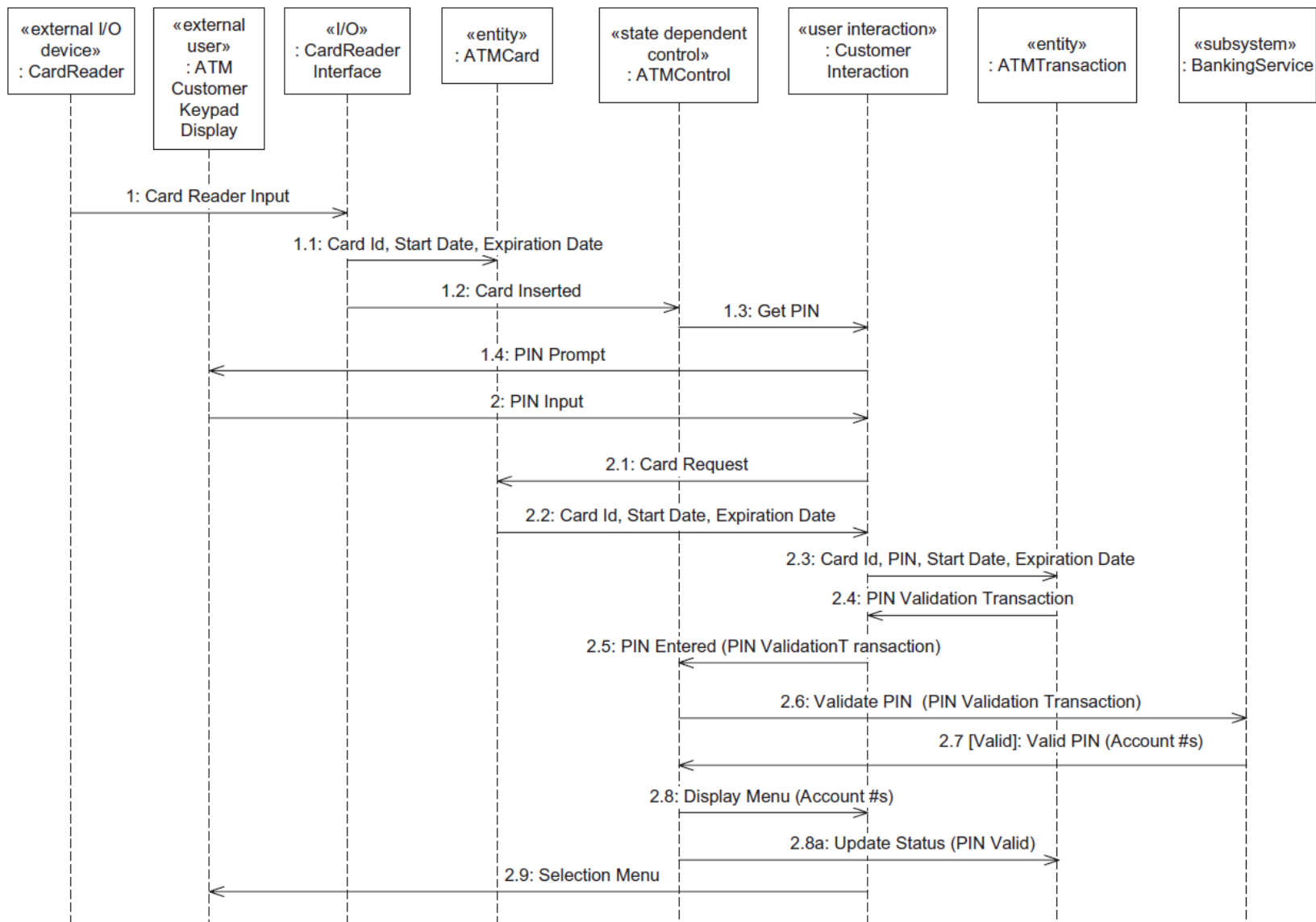
- ③ 为每一个类设置生命线,并确定类的存在期限;
- ④ 从引发交互的初始消息开始,在对象生命线上依次画出交互的消息
- ⑤ 绘制顺序图

实例:

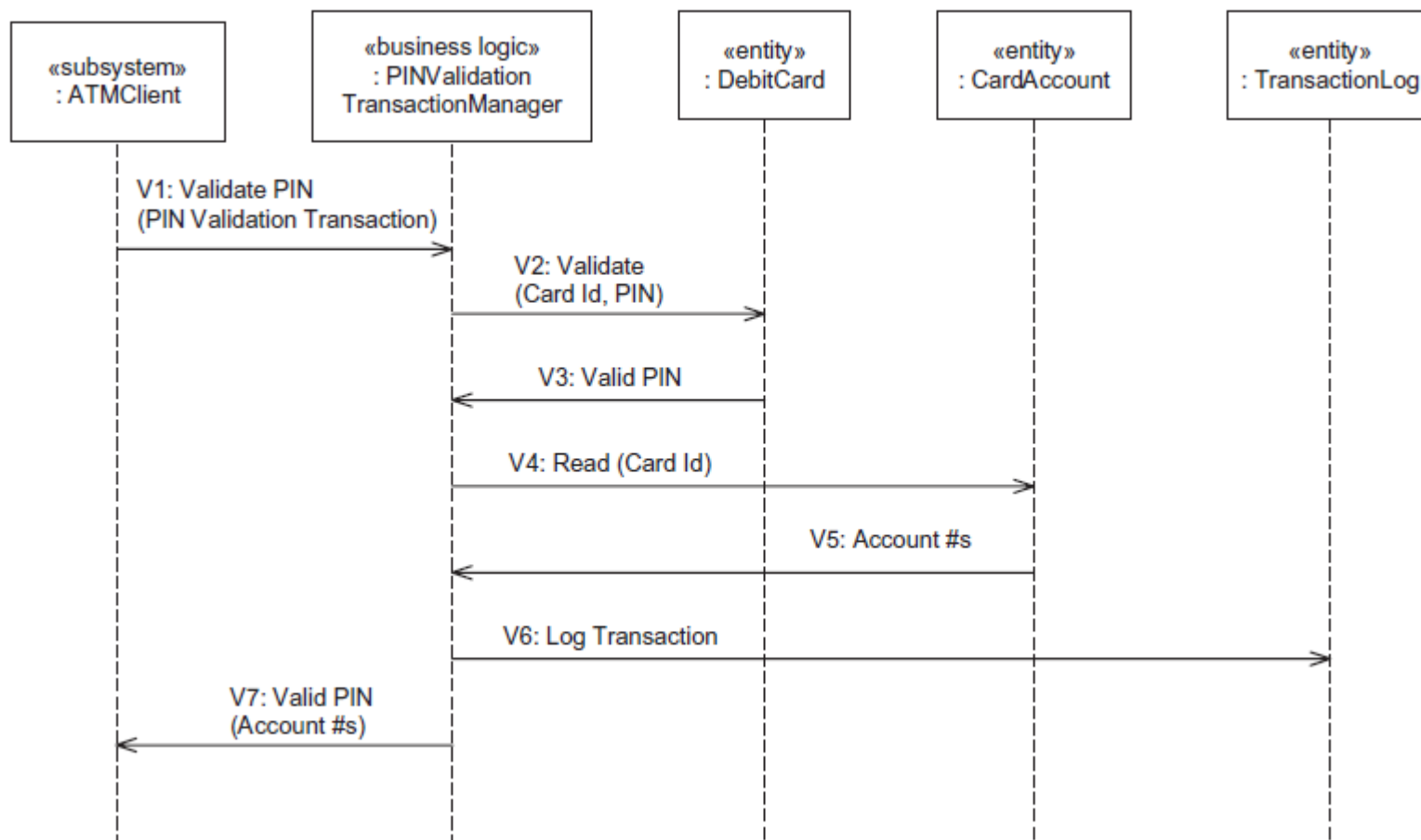
顺序图



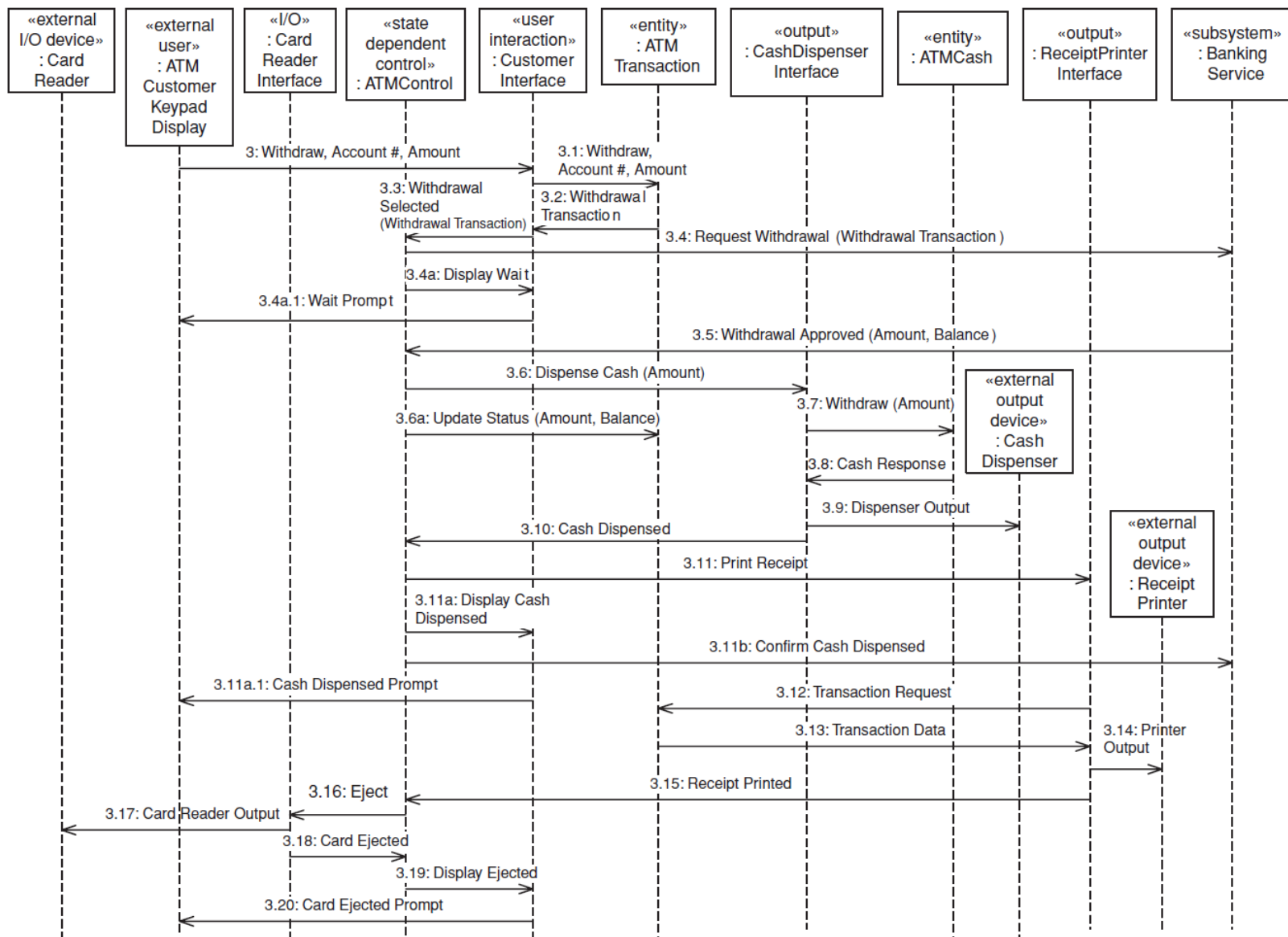
案例-ATM验证PIN码顺序图



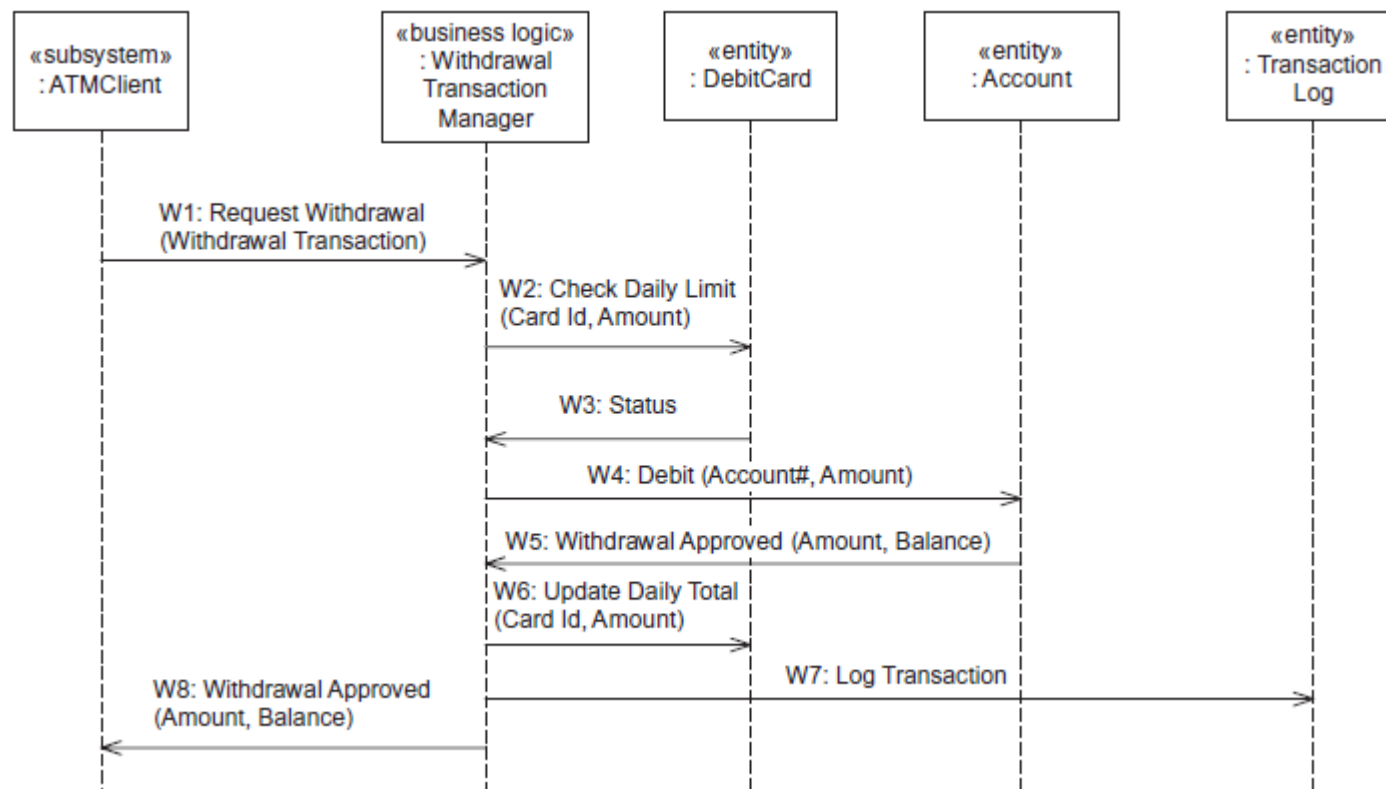
案例-ATM服务器端验证PIN码顺序图



案例-ATM客户端取款顺序图



案例-ATM服务端顺序图



谢谢!